

Advanced Python Constructs

Contents

- Iterators, generator expressions and generators
- Decorators

Author Zbigniew Jędrzejewski-Szmek

This section covers some features of the Python language which can be considered advanced — in the sense that not every language has them, and also in the sense that they are more useful in more complicated programs or libraries, but not in the sense of being particularly specialized, or particularly complicated.

It is important to underline that this chapter is purely about the language itself — about features supported through special syntax complemented by functionality of the Python stdlib, which could not be implemented through clever external modules.

The process of developing the Python programming language, its syntax, is very transparent; proposed changes are evaluated from various angles and discussed via *Python Enhancement Proposals* — [PEPs](#). As a result, features described in this chapter were added after it was shown that they indeed solve real problems and that their use is as simple as possible.

Iterators, generator expressions and generators

Iterators

An iterator is an object adhering to the [iterator protocol](#) — basically this means that it has a `next <iterator.next>` method, which, when called, returns the next item in the sequence, and when there's nothing to return, raises the `StopIteration <exceptions.StopIteration>` exception.

Simplicity

This duplication of effort is wasteful, and replacing the various home-grown approaches with a standard feature usually

An iterator object allows to loop just once. It holds the state (position) of a single iteration, or from the other side, each loop over a sequence requires a single iterator object. This means that we can iterate over the same sequence more than once concurrently. Separating the iteration logic from the sequence allows us to have more than one way of iteration.

ends up making things more readable, and interoperable as well. — *Guido van Rossum* in [Adding Optional Static Typing to Python](#)

Calling the `__iter__` <object.__iter__> method on a container to create an iterator object is the most straightforward way to get hold of an iterator. The `iter` function does that for us, saving a few keystrokes.

```
nums = [1, 2, 3]      # note that ... varies: these are different objects
iter(nums)
```

```
<list_iterator at 0x7fcd584f3cd0>
```

```
nums.__iter__()
```

```
<list_iterator at 0x7fcd44de0040>
```

```
nums.__reversed__()
```

```
<list_reverseiterator at 0x7fcd44df8ca0>
```

```
it = iter(nums)
next(it)
```

```
1
```

```
next(it)
```

```
2
```

```
next(it)
```

```
next(it)
```

```
-----
StopIteration                                Traceback (most recent call last)
Cell In[7], line 1
----> 1 next(it)

StopIteration:
```

When used in a loop, `StopIteration <exceptions.StopIteration>` is swallowed and causes the loop to finish. But with explicit invocation, we can see that once the iterator is exhausted, accessing it raises an exception.

Using the `for..in` loop also uses the `__iter__` method. This allows us to transparently start the iteration over a sequence. But if we already have the iterator, we want to be able to use it in an `for` loop in the same way. In order to achieve this, iterators in addition to `next` are also required to have a method called `__iter__` which returns the iterator (`self`).

Support for iteration is pervasive in Python: all sequences and unordered containers in the standard library allow this. The concept is also stretched to other things: e.g. `file` objects support iteration over lines.

```
with open("/etc/fstab") as f:
    assert f is f.__iter__()
```

The `file` is an iterator itself and its `__iter__` method doesn't create a separate object: only a single thread of sequential access is allowed.

Generator expressions

A second way in which iterator objects are created is through **generator expressions**, the basis for **list comprehensions**. To increase clarity, a generator expression must always be enclosed in parentheses or an expression. If round parentheses are used, then a generator iterator is created. If rectangular parentheses are used, the process is short-circuited and we get a `list`.

```
(i for i in nums)
```

```
<generator object <genexpr> at 0x7fcd44e4ce80>
```

```
[i for i in nums]
```

```
[1, 2, 3]
```

```
list(i for i in nums)
```

```
[1, 2, 3]
```

The list comprehension syntax also extends to **dictionary and set comprehensions**. A `set` is created when the generator expression is enclosed in curly braces. A `dict` is created when the generator expression contains “pairs” of the form `key:value`:

```
{i for i in range(3)}
```

```
{0, 1, 2}
```

```
{i:i**2 for i in range(3)}
```

```
{0: 0, 1: 1, 2: 4}
```

One *gotcha* should be mentioned: in old Pythons the index variable (`i`) would leak, and in versions `>= 3` this is fixed.

Generators

A third way to create iterator objects is to call a generator function. A **generator** is a function containing the keyword `yield`. It must be noted that the mere presence of this keyword completely changes the nature of the function: this `yield` statement doesn't have to be invoked, or even reachable, but causes the function to be marked as a generator. When a normal function is called, the instructions contained in the body start to be executed. When a generator is called, the execution stops before the first instruction in the body. An invocation of a generator function creates a generator object, adhering to the iterator protocol. As with normal function invocations, concurrent and recursive invocations are allowed.

When `next` is called, the function is executed until the first `yield`. Each encountered `yield` statement gives a value becomes the return value of `next`. After executing the `yield` statement, the execution of this function is suspended.

Generators

A generator is a function that produces a sequence of results instead of a single value. — *David Beazley* in the slides for [A Curious Course on Coroutines and Concurrency](#)

```
def f():  
    yield 1  
    yield 2  
  
f()
```

```
<generator object f at 0x7fcd447d0040>
```

```
gen = f()  
next(gen)
```

```
1
```

```
next(gen)
```

```
2
```

```
next(gen)
```

```
-----  
StopIteration                                Traceback (most recent call last)  
Cell In[16], line 1  
----> 1 next(gen)  
  
StopIteration:
```

Let's go over the life of the single invocation of the generator function.

```
def f():
    print("-- start --")
    yield 3
    print("-- finish --")
    yield 4

gen = f()
next(gen)
```

```
-- start --
```

```
3
```

```
next(gen)
```

```
-- finish --
```

```
4
```

```
next(gen)
```

```
-----
StopIteration                                Traceback (most recent call last)
Cell In[19], line 1
----> 1 next(gen)

StopIteration:
```

Contrary to a normal function, where executing `f()` would immediately cause the first `print` to be executed, `gen` is assigned without executing any statements in the function body. Only when `gen.__next__()` is invoked by `next`, the statements up to the first `yield` are executed. The second `next` prints `-- finish --` and execution halts on the second `yield`. The third `next` falls off the end of the function. Since no `yield` was reached, an exception is raised.

What happens with the function after a yield, when the control passes to the caller? The state of each generator is stored in the generator object. From the point of view of the generator function, it looks almost as if it was running in a separate thread, but this is just an illusion: execution is strictly single-threaded, but the interpreter keeps and restores the state in between the requests for the next value.

Why are generators useful? As noted in the parts about iterators, a generator function is just a different way to create an iterator object. Everything that can be done with `yield` statements, could also be done with `next` methods. Nevertheless, using a function and having the interpreter perform its magic to create an iterator has advantages. A function can be much shorter than the definition of a class with the required `next` and `__iter__` methods. What is more important, it is easier for the author of the generator to understand the state which is kept in local variables, as opposed to instance attributes, which have to be used to pass data between consecutive invocations of `next` on an iterator object.

A broader question is why are iterators useful? When an iterator is used to power a loop, the loop becomes very simple. The code to initialise the state, to decide if the loop is finished, and to find the next value is extracted into a separate place. This highlights the body of the loop — the interesting part. In addition, it is possible to reuse the iterator code in other places.

Bidirectional communication

Each `yield` statement causes a value to be passed to the caller. This is the reason for the introduction of generators by [PEP 255](#). But communication in the reverse direction is also useful. One obvious way would be some external state, either a global variable or a shared mutable object. Direct communication is possible thanks to [PEP 342](#). It is achieved by turning the previously boring `yield` statement into an expression. When the generator resumes execution after a `yield` statement, the caller can call a method on the generator object to either pass a value **into** the generator, which then is returned by the `yield` statement, or a different method to inject an exception into the generator.

The first of the new methods is `send(value) <generator.send>`, which is similar to `next() <generator.next>`, but passes `value` into the generator to be used for the value of the `yield` expression. In fact, `g.next()` and `g.send(None)` are equivalent.

The second of the new methods is `throw(type, value=None, traceback=None) <generator.throw>` which is equivalent to:

```
raise type, value, traceback
```

at the point of the `yield` statement.

Unlike `raise` (which immediately raises an exception from the current execution point), `throw()` first resumes the generator, and only then raises the exception. The word throw was picked

because it is suggestive of putting the exception in another location, and is associated with exceptions in other languages.

What happens when an exception is raised inside the generator? It can be either raised explicitly or when executing some statements or it can be injected at the point of a `yield` statement by means of the `throw()` method. In either case, such an exception propagates in the standard manner: it can be intercepted by an `except` or `finally` clause, or otherwise it causes the execution of the generator function to be aborted and propagates in the caller.

For completeness' sake, it's worth mentioning that generator iterators also have a `close()` `<generator.close>` method, which can be used to force a generator that would otherwise be able to provide more values to finish immediately. It allows the generator `__del__ <object.__del__>` method to destroy objects holding the state of generator. Let's define a generator which just prints what is passed in through send and throw.

```
import itertools

def g():
    print('--start--')
    for i in itertools.count():
        print('--yielding %i--' % i)
        try:
            ans = yield i
        except GeneratorExit:
            print('--closing--')
            raise
        except Exception as e:
            print('--yield raised %r--' % e)
        else:
            print('--yield returned %s--' % ans)
```

```
it = g()
next(it)
```

```
--start--
--yielding 0--
```

```
0
```

```
it.send(11)
```

```
--yield returned 11--  
--yielding 1--
```

1

```
it.throw(IndexError)
```

```
--yield raised IndexError--  
--yielding 2--
```

2

```
it.close()
```

```
--closing--
```

Chaining generators

Note

This is a preview of [PEP 380](#) (not yet implemented, but accepted for Python 3.3).

Let's say we are writing a generator and we want to yield a number of values generated by a second generator, a **subgenerator**. If yielding of values is the only concern, this can be performed without much difficulty using a loop such as

```
subgen = some_other_generator()  
for v in subgen:  
    yield v
```

However, if the subgenerator is to interact properly with the caller in the case of calls to `send()`, `throw()` and `close()`, things become considerably more difficult. The `yield` statement has to be guarded by a `try..except..finally` structure similar to the one defined in the previous section to “debug” the generator function. Such code is provided in [PEP 380#id13](#), here it suffices to say that new syntax to properly yield from a subgenerator is being introduced in Python 3.3:

```
yield from some_other_generator()
```

This behaves like the explicit loop above, repeatedly yielding values from `some_other_generator` until it is exhausted, but also forwards `send`, `throw` and `close` to the subgenerator.

Decorators

Since functions and classes are objects, they can be passed around. Since they are mutable objects, they can be modified. The act of altering a function or class object after it has been constructed but before it is bound to its name is called decorating.

There are two things hiding behind the name “decorator” — one is the function which does the work of decorating, i.e. performs the real work, and the other one is the expression adhering to the decorator syntax, i.e. an at-symbol and the name of the decorating function.

Function can be decorated by using the decorator syntax for functions:

```
@decorator          # ②
def function():      # ①
    pass
```

- A function is defined in the standard way. ①
- An expression starting with `@` placed before the function definition is the decorator ②. The part after `@` must be a simple expression, usually this is just the name of a function or class. This part is evaluated first, and after the function defined below is ready, the decorator is called with the newly defined function object as the single argument. The value returned by the decorator is attached to the original name of the function.

Decorators can be applied to functions and to classes. For classes the semantics are identical — the original class definition is used as an argument to call the decorator and whatever is returned is assigned under the original name.

Before the decorator syntax was implemented ([PEP 318](#)), it was possible to achieve the same effect by assigning the function or class object to a temporary variable and then invoking the

Summary

This amazing feature appeared in the language almost apologetically and with concern that it might not be that useful. — *Bruce Eckel* in [An Introduction to Python Decorators](#)

decorator explicitly and then assigning the return value to the name of the function. This sounds like more typing, and it is, and also the name of the decorated function doubling as a temporary variable must be used at least three times, which is prone to errors. Nevertheless, the example above is equivalent to:

```
def function():                # ①
    pass
function = decorator(function) # ②
```

Decorators can be stacked — the order of application is bottom-to-top, or inside-out. The semantics are such that the originally defined function is used as an argument for the first decorator, whatever is returned by the first decorator is used as an argument for the second decorator, ..., and whatever is returned by the last decorator is attached under the name of the original function.

The decorator syntax was chosen for its readability. Since the decorator is specified before the header of the function, it is obvious that its is not a part of the function body and its clear that it can only operate on the whole function. Because the expression is prefixed with `@` is stands out and is hard to miss (“in your face”, according to the PEP :)). When more than one decorator is applied, each one is placed on a separate line in an easy to read way.

Replacing or tweaking the original object

Decorators can either return the same function or class object or they can return a completely different object. In the first case, the decorator can exploit the fact that function and class objects are mutable and add attributes, e.g. add a docstring to a class. A decorator might do something useful even without modifying the object, for example register the decorated class in a global registry. In the second case, virtually anything is possible: when something different is substituted for the original function or class, the new object can be completely different. Nevertheless, such behaviour is not the purpose of decorators: they are intended to tweak the decorated object, not do something unpredictable. Therefore, when a function is “decorated” by replacing it with a different function, the new function usually calls the original function, after doing some preparatory work. Likewise, when a class is “decorated” by replacing it with a new class, the new class is usually derived from the original class. When the purpose of the decorator is to do something “every time”, like to log every call to a decorated function, only the second type of decorators can be used. On the other hand, if the first type is sufficient, it is better to use it, because it is simpler.

Decorators implemented as classes and as functions

The only *requirement* on decorators is that they can be called with a single argument. This means that decorators can be implemented as normal functions, or as classes with a `__call__` `<object.__call__>` method, or in theory, even as lambda functions.

Let's compare the function and class approaches. The decorator expression (the part after `@`) can be either just a name, or a call. The bare-name approach is nice (less to type, looks cleaner, etc.), but is only possible when no arguments are needed to customise the decorator. Decorators written as functions can be used in those two cases:

```
def simple_decorator(function):  
    print("doing decoration")  
    return function  
@simple_decorator  
def function():  
    print("inside function")
```

doing decoration

function()

inside function

```
def decorator_with_arguments(arg):  
    print("defining the decorator")  
    def _decorator(function):  
        # in this inner function, arg is available too  
        print("doing decoration, %r" % arg)  
        return function  
    return _decorator  
@decorator_with_arguments("abc")  
def function():  
    print("inside function")
```

defining the decorator
doing decoration, 'abc'

function()

inside function

The two trivial decorators above fall into the category of decorators which return the original function. If they were to return a new function, an extra level of nestedness would be required. In the worst case, three levels of nested functions.

```
def replacing_decorator_with_args(arg):
    print("defining the decorator")
    def _decorator(function):
        # in this inner function, arg is available too
        print("doing decoration, %r" % arg)
        def _wrapper(*args, **kwargs):
            print("inside wrapper, %r %r" % (args, kwargs))
            return function(*args, **kwargs)
        return _wrapper
    return _decorator
@replacing_decorator_with_args("abc")
def function(*args, **kwargs):
    print("inside function, %r %r" % (args, kwargs))
    return 14
```

```
defining the decorator
doing decoration, 'abc'
```

```
function(11, 12)
```

```
inside wrapper, (11, 12) {}
inside function, (11, 12) {}
```

```
14
```

The `_wrapper` function is defined to accept all positional and keyword arguments. In general we cannot know what arguments the decorated function is supposed to accept, so the wrapper function just passes everything to the wrapped function. One unfortunate consequence is that the apparent argument list is misleading.

Compared to decorators defined as functions, complex decorators defined as classes are simpler. When an object is created, the `__init__ <object.__init__>` method is only allowed to return `None`, and the type of the created object cannot be changed. This means that when a decorator is defined as a class, it doesn't make much sense to use the argument-less form: the final decorated object would just be an instance of the decorating class, returned by the constructor call, which is not very useful. Therefore it's enough to discuss class-based decorators where arguments are given in the decorator expression and the decorator `__init__` method is used for decorator construction.

```
class decorator_class(object):
    def __init__(self, arg):
        # this method is called in the decorator expression
        print("in decorator init, %s" % arg)
        self.arg = arg
    def __call__(self, function):
        # this method is called to do the job
        print("in decorator call, %s" % self.arg)
        return function
deco_instance = decorator_class('foo')
```

in decorator init, foo

```
@deco_instance
def function(*args, **kwargs):
    print("in function, %s %s" % (args, kwargs))
```

in decorator call, foo

function()

in function, () {}

Contrary to normal rules ([PEP 8](#)) decorators written as classes behave more like functions and therefore their name often starts with a lowercase letter.

In reality, it doesn't make much sense to create a new class just to have a decorator which returns the original function. Objects are supposed to hold state, and such decorators are more useful when the decorator returns a new object.

```
class replacing_decorator_class(object):
    def __init__(self, arg):
        # this method is called in the decorator expression
        print("in decorator init, %s" % arg)
        self.arg = arg
    def __call__(self, function):
        # this method is called to do the job
        print("in decorator call, %s" % self.arg)
        self.function = function
        return self._wrapper
    def _wrapper(self, *args, **kwargs):
        print("in the wrapper, %s %s" % (args, kwargs))
        return self.function(*args, **kwargs)
deco_instance = replacing_decorator_class('foo')
```

```
in decorator init, foo
```

```
@deco_instance
def function(*args, **kwargs):
    print("in function, %s %s" % (args, kwargs))
```

```
in decorator call, foo
```

```
function(11, 12)
```

```
in the wrapper, (11, 12) {}
in function, (11, 12) {}
```

A decorator like this can do pretty much anything, since it can modify the original function object and mangle the arguments, call the original function or not, and afterwards mangle the return value.

Copying the docstring and other attributes of the original function

When a new function is returned by the decorator to replace the original function, an unfortunate consequence is that the original function name, the original docstring, the original argument list are lost. Those attributes of the original function can partially be “transplanted” to the new function by setting `__doc__` (the docstring), `__module__` and `__name__` (the full name of the function), and `__annotations__` (extra information about arguments and the return value of the function available in Python 3). This can be done automatically by using `functools.update_wrapper`.



`functools.update_wrapper(wrapper, wrapped)` <functools.update_wrapper>

“Update a wrapper function to look like the wrapped function.”

```
import functools
def replacing_decorator_with_args(arg):
    print("defining the decorator")
    def _decorator(function):
        print("doing decoration, %r" % arg)
        def _wrapper(*args, **kwargs):
            print("inside wrapper, %r %r" % (args, kwargs))
            return function(*args, **kwargs)
        return functools.update_wrapper(_wrapper, function)
    return _decorator
@replacing_decorator_with_args("abc")
def function():
    "extensive documentation"
    print("inside function")
    return 14
```

```
defining the decorator
doing decoration, 'abc'
```

```
function
```

```
<function __main__.function()>
```

```
print(function.__doc__)
```

```
extensive documentation
```

One important thing is missing from the list of attributes which can be copied to the replacement function: the argument list. The default values for arguments can be modified through the `__defaults__`, `__kwdefaults__` attributes, but unfortunately the argument list itself cannot be set as an attribute. This means that `help(function)` will display a useless argument list which will be confusing for the user of the function. An effective but ugly way around this problem is to create the wrapper dynamically, using `eval`. This can be automated by using the external `decorator` module. It provides support for the `decorator` decorator, which takes a wrapper and turns it into a decorator which preserves the function signature.

To sum things up, decorators should always use `functools.update_wrapper` or some other means of copying function attributes.

Examples in the standard library

First, it should be mentioned that there's a number of useful decorators available in the standard library. There are three decorators which really form a part of the language:

- `classmethod` causes a method to become a “class method”, which means that it can be invoked without creating an instance of the class. When a normal method is invoked, the interpreter inserts the instance object as the first positional parameter, `self`. When a class method is invoked, the class itself is given as the first parameter, often called `cls`.

Class methods are still accessible through the class' namespace, so they don't pollute the module's namespace. Class methods can be used to provide alternative constructors:

```
class Array(object):
    def __init__(self, data):
        self.data = data

    @classmethod
    def fromfile(cls, file):
        data = numpy.load(file)
        return cls(data)
```

This is cleaner than using a multitude of flags to `__init__`.

- `staticmethod` is applied to methods to make them “static”, i.e. basically a normal function, but accessible through the class namespace. This can be useful when the function is only needed inside this class (its name would then be prefixed with `_`), or when we want the user to think of the method as connected to the class, despite an implementation which doesn't require this.
- `property` is the pythonic answer to the problem of getters and setters. A method decorated with `property` becomes a getter which is automatically called on attribute access.

```
class A(object):
    @property
    def a(self):
        "an important attribute"
        return "a value"
```

A.a

<property at 0x7fcd44dbdb70>

A().a

```
'a value'
```

In this example, `A.a` is an read-only attribute. It is also documented: `help(A)` includes the docstring for attribute `a` taken from the getter method. Defining `a` as a property allows it to be a calculated on the fly, and has the side effect of making it read-only, because no setter is defined.

To have a setter and a getter, two methods are required, obviously:

```
class Rectangle(object):
    def __init__(self, edge):
        self.edge = edge

    @property
    def area(self):
        """Computed area.

        Setting this updates the edge length to the proper value.
        """
        return self.edge**2

    @area.setter
    def area(self, area):
        self.edge = area ** 0.5
```

The way that this works, is that the `property` decorator replaces the getter method with a property object. This object in turn has three methods, `getter`, `setter`, and `deleter`, which can be used as decorators. Their job is to set the getter, setter and deleter of the property object (stored as attributes `fget`, `fset`, and `fdel`). The getter can be set like in the example above, when creating the object. When defining the setter, we already have the property object under `area`, and we add the setter to it by using the `setter` method. All this happens when we are creating the class.

Afterwards, when an instance of the class has been created, the property object is special. When the interpreter executes attribute access, assignment, or deletion, the job is delegated to the methods of the property object.

To make everything crystal clear, let's define a "debug" example:

```
class D(object):
    @property
    def a(self):
        print("getting 1")
        return 1
    @a.setter
    def a(self, value):
        print("setting %r" % value)
    @a.deleter
    def a(self):
        print("deleting")
D.a
```

<property at 0x7fcd44689300>

D.a.fget

<function __main__.D.a(self)>

D.a.fset

<function __main__.D.a(self, value)>

D.a.fdel

<function __main__.D.a(self)>

```
d = D()          # ... varies, this is not the same `a` function
d.a
```

getting 1

1

d.a = 2

setting 2

```
del d.a
```

```
deleting
```

```
d.a
```

```
getting 1
```

```
1
```

Properties are a bit of a stretch for the decorator syntax. One of the premises of the decorator syntax — that the name is not duplicated — is violated, but nothing better has been invented so far. It is just good style to use the same name for the getter, setter, and deleter methods.

Some newer examples include:

- `functools.lru_cache` memoizes an arbitrary function maintaining a limited cache of arguments:answer pairs (Python 3.2)
- `functools.total_ordering` is a class decorator which fills in missing ordering methods (`__lt__` `<object.__lt__>`, `__gt__` `<object.__gt__>`, `__le__` `<object.__le__>`, ...) based on a single available one.

Deprecation of functions

Let's say we want to print a deprecation warning on stderr on the first invocation of a function we don't like anymore. If we don't want to modify the function, we can use a decorator:

```

class deprecated(object):
    """Print a deprecation warning once on first use of the function.

    >>> @deprecated()
    ... def f():
    ...     pass
    >>> f()
    f is deprecated
    """
    def __call__(self, func):
        self.func = func
        self.count = 0
        return self._wrapper
    def _wrapper(self, *args, **kwargs):
        self.count += 1
        if self.count == 1:
            print(self.func.__name__, 'is deprecated')
        return self.func(*args, **kwargs)

```

It can also be implemented as a function:

```

def deprecated(func):
    """Print a deprecation warning once on first use of the function.

    >>> @deprecated
    ... def f():
    ...     pass
    >>> f()
    f is deprecated
    """
    count = [0]
    def wrapper(*args, **kwargs):
        count[0] += 1
        if count[0] == 1:
            print(func.__name__, 'is deprecated')
        return func(*args, **kwargs)
    return wrapper

```

A `while`-loop removing decorator

Let's say we have function which returns a lists of things, and this list created by running a loop. If we don't know how many objects will be needed, the standard way to do this is something like:

```
def find_answers():
    answers = []
    while True:
        ans = look_for_next_answer()
        if ans is None:
            break
        answers.append(ans)
    return answers
```

This is fine, as long as the body of the loop is fairly compact. Once it becomes more complicated, as often happens in real code, this becomes pretty unreadable. We could simplify this by using `yield` statements, but then the user would have to explicitly call `list(find_answers())`.

We can define a decorator which constructs the list for us:

```
def vectorized(generator_func):
    def wrapper(*args, **kwargs):
        return list(generator_func(*args, **kwargs))
    return functools.update_wrapper(wrapper, generator_func)
```

Our function then becomes:

```
@vectorized
def find_answers():
    while True:
        ans = look_for_next_answer()
        if ans is None:
            break
        yield ans
```

A plugin registration system

This is a class decorator which doesn't modify the class, but just puts it in a global registry. It falls into the category of decorators returning the original object:

```

class WordProcessor(object):
    PLUGINS = []
    def process(self, text):
        for plugin in self.PLUGINS:
            text = plugin().cleanup(text)
        return text

    @classmethod
    def plugin(cls, plugin):
        cls.PLUGINS.append(plugin)

@WordProcessor.plugin
class CleanMdashesExtension(object):
    def cleanup(self, text):
        return text.replace('&mdash;', u'\N{em dash}')

```

Here we use a decorator to decentralise the registration of plugins. We call our decorator with a noun, instead of a verb, because we use it to declare that our class is a plugin for `WordProcessor`. Method `plugin` simply appends the class to the list of plugins.

A word about the plugin itself: it replaces HTML entity for em-dash with a real Unicode em-dash character. It exploits the [unicode literal notation](#) to insert a character by using its name in the unicode database (“EM DASH”). If the Unicode character was inserted directly, it would be impossible to distinguish it from an en-dash in the source of a program.

See also

More examples and reading

- [PEP 318](#) (function and method decorator syntax)
- [PEP 3129](#) (class decorator syntax)
- <https://wiki.python.org/moin/PythonDecoratorLibrary>
- <https://docs.python.org/dev/library/functools.html>
- <https://pypi.org/project/decorator>
- Bruce Eckel
 - [Decorators I](#): Introduction to Python Decorators
 - [Python Decorators II](#): Decorator Arguments
 - [Python Decorators III](#): A Decorator-Based Build System

Context managers

A context manager is an object with `__enter__ <object.__enter__>` and `__exit__ <object.__exit__>` methods which can be used in the `with` statement:

```
with manager as var:  
    do_something(var)
```

is in the simplest case

equivalent to

```
var = manager.__enter__()  
try:  
    do_something(var)  
finally:  
    manager.__exit__()
```

In other words, the context manager protocol defined in [PEP 343](#) permits the extraction of the boring part of a `try..except..finally` structure into a separate class leaving only the interesting `do_something` block.

1. The `__enter__ <object.__enter__>` method is called first. It can return a value which will be assigned to `var`. The `as`-part is optional: if it isn't present, the value returned by

`__enter__` is simply ignored.

2. The block of code underneath `with` is executed. Just like with `try` clauses, it can either execute successfully to the end, or it can `break`, `continue` or `return`, or it can throw an exception. Either way, after the block is finished, the `__exit__` <object.__exit__> method is called. If an exception was thrown, the information about the exception is passed to `__exit__`, which is described below in the next subsection. In the normal case, exceptions can be ignored, just like in a `finally` clause, and will be rethrown after `__exit__` is finished.

Let's say we want to make sure that a file is closed immediately after we are done writing to it:

```
class closing(object):
    def __init__(self, obj):
        self.obj = obj
    def __enter__(self):
        return self.obj
    def __exit__(self, *args):
        self.obj.close()
with closing(open('/tmp/file', 'w')) as f:
    f.write('the contents\n')
```

Here we have made sure that the `f.close()` is called when the `with` block is exited. Since closing files is such a common operation, the support for this is already present in the `file` class. It has an `__exit__` method which calls `close` and can be used as a context manager itself:

```
with open('/tmp/file', 'a') as f:
    f.write('more contents\n')
```

The common use for `try..finally` is releasing resources. Various different cases are implemented similarly: in the `__enter__` phase the resource is acquired, in the `__exit__` phase it is released, and the exception, if thrown, is propagated. As with files, there's often a natural operation to perform after the object has been used and it is most convenient to have the support built in. With each release, Python provides support in more places:

- all file-like objects:
 - `file` → automatically closed
 - `fileinput`, `tempfile`
 - `bz2.BZ2File`, `gzip.GzipFile`, `tarfile.TarFile`, `zipfile.ZipFile`
 - `ftplib`, `nntplib` → close connection
- locks
 - `multiprocessing.RLock` → lock and unlock
 - `multiprocessing.Semaphore`

- `memoryview` → automatically release
- `decimal.localcontext` → modify precision of computations temporarily
- `_winreg.PyHKEY <_winreg.OpenKey>` → open and close hive key
- `warnings.catch_warnings` → kill warnings temporarily
- `contextlib.closing` → the same as the example above, call `close`
- parallel programming
 - `concurrent.futures.ThreadPoolExecutor` → invoke in parallel then kill thread pool
 - `concurrent.futures.ProcessPoolExecutor` → invoke in parallel then kill process pool
 - `nogil` → solve the GIL problem temporarily (cython only :())

Catching exceptions

When an exception is thrown in the `with`-block, it is passed as arguments to `__exit__`. Three arguments are used, the same as returned by `sys.exc_info()`: type, value, traceback. When no exception is thrown, `None` is used for all three arguments. The context manager can “swallow” the exception by returning a true value from `__exit__`. Exceptions can be easily ignored, because if `__exit__` doesn’t use `return` and just falls of the end, `None` is returned, a false value, and therefore the exception is rethrown after `__exit__` is finished.

The ability to catch exceptions opens interesting possibilities. A classic example comes from unit-tests — we want to make sure that some code throws the right kind of exception:

```
class assert_raises(object):
    # based on pytest and unittest.TestCase
    def __init__(self, type):
        self.type = type
    def __enter__(self):
        pass
    def __exit__(self, type, value, traceback):
        if type is None:
            raise AssertionError('exception expected')
        if isinstance(type, self.type):
            return True # swallow the expected exception
        raise AssertionError('wrong exception type')

with assert_raises(KeyError):
    {}['foo']
```

Using generators to define context managers

When discussing [generators], it was said that we prefer generators to iterators implemented as classes because they are shorter, sweeter, and the state is stored as local, not instance, variables. On the other hand, as described in [Bidirectional communication], the flow of data between the generator and its caller can be bidirectional. This includes exceptions, which can be thrown into the generator. We would like to implement context managers as special generator functions. In fact, the generator protocol was designed to support this use case.

```
@contextlib.contextmanager
def some_generator(<arguments>):
    <setup>
    try:
        yield <value>
    finally:
        <cleanup>
```

The `contextlib.contextmanager` helper takes a generator and turns it into a context manager. The generator has to obey some rules which are enforced by the wrapper function — most importantly it must `yield` exactly once. The part before the `yield` is executed from `__enter__`, the block of code protected by the context manager is executed when the generator is suspended in `yield`, and the rest is executed in `__exit__`. If an exception is thrown, the interpreter hands it to the wrapper through `__exit__` arguments, and the wrapper function then throws it at the point of the `yield` statement. Through the use of generators, the context manager is shorter and simpler.

Let's rewrite the `closing` example as a generator:

```
import contextlib

@contextlib.contextmanager
def closing(obj):
    try:
        yield obj
    finally:
        obj.close()
```

Let's rewrite the `assert_raises` example as a generator:

```
@contextlib.contextmanager
def assert_raises(type):
    try:
        yield
    except type:
        return
    except Exception as value:
        raise AssertionError('wrong exception type')
    else:
        raise AssertionError('exception expected')
```

Here we use a decorator to turn generator functions into context managers!

Advanced NumPy

Contents

- Life of ndarray
- Universal functions
- Interoperability features
- Array siblings: `chararray`, `MaskedArray`
- Summary
- Contributing to NumPy/SciPy

Author: *Pauli Virtanen*

NumPy is at the base of Python's scientific stack of tools. Its purpose to implement efficient operations on many items in a block of memory. Understanding how it works in detail helps in making efficient use of its flexibility, taking useful shortcuts.

This section covers:

- Anatomy of NumPy arrays, and its consequences. Tips and tricks.
- Universal functions: what, why, and what to do if you want a new one.
- Integration with other tools: NumPy offers several ways to wrap any data in an ndarray, without unnecessary copies.
- Recently added features, and what's in them: PEP 3118 buffers, generalized ufuncs, ...

Prerequisites

- NumPy
- Cython
- Pillow (Python imaging library, used in a couple of examples)

```
# Import Numpy module.
import numpy as np
# Import Matplotlib (for later).
import matplotlib.pyplot as plt
```

Life of ndarray

It's...

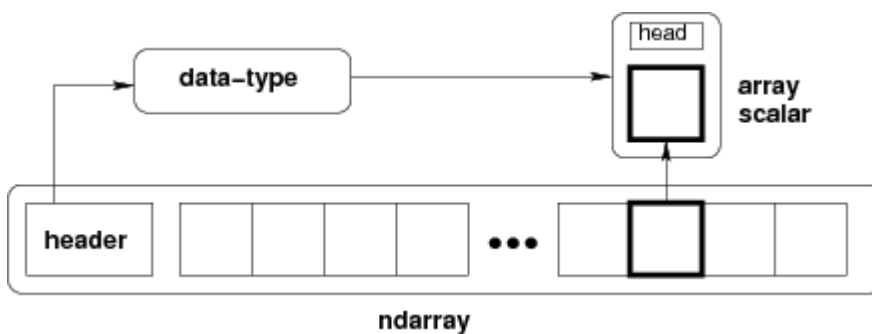
🔔 What is an ndarray

An **ndarray** is:

- A block of memory and
- an indexing scheme and
- a data type descriptor.

Put another way, an ndarray has **raw data**, and algorithms to:

- locate an element
- interpret an element



```
typedef struct PyArrayObject {
    PyObject_HEAD

    /* Block of memory */
    char *data;

    /* Data type descriptor */
    PyArray_Descr *descr;

    /* Indexing scheme */
    int nd;
    npy_intp *dimensions;
    npy_intp *strides;

    /* Other stuff */
    PyObject *base;
    int flags;
    PyObject *weakreflist;
} PyArrayObject;
```

Block of memory

```
x = np.array([1, 2, 3], dtype=np.int32)
x.data
```

```
<memory at 0x7f9df9f31d80>
```

```
bytes(x.data)
```

```
b'\x01\x00\x00\x00\x02\x00\x00\x00\x03\x00\x00\x00'
```

Memory address of the data:

```
x.__array_interface__['data'][0]
```

```
94910825589392
```

The whole `__array_interface__`:

```
x.__array_interface__
```

```
{'data': (94910825589392, False),  
  'strides': None,  
  'descr': [('', '<i4')],  
  'typestr': '<i4',  
  'shape': (3,),  
  'version': 3}
```

Reminder: two `ndarrays` may share the same memory:

```
x = np.array([1, 2, 3, 4])  
y = x[:-1]  
x[0] = 9  
y
```

```
array([9, 2, 3])
```

Memory does not need to be owned by an `ndarray`:

```
x = b'1234'
```

x is a string (in Python 3 a bytes), we can represent its data as an array of ints:

```
y = np.frombuffer(x, dtype=np.int8)  
y.data
```

```
<memory at 0x7fadb9f31f00>
```

```
y.base is x
```

```
True
```

```
y.flags
```

```
C_CONTIGUOUS : True  
F_CONTIGUOUS : True  
OWNDATA : False  
WRITEABLE : False  
ALIGNED : True  
WRITEBACKIFCOPY : False
```

The `owndata` and `writable` flags indicate status of the memory block.

Data types

The descriptor

`dtype` describes a single item in the array:

Table 6 **Dtypes**

type	scalar type of the data, one of: • int8, int16, float64, <i>et al.</i> (fixed size) • str, unicode, void (flexible size)
itemsize	size of the data block
byteorder	byte order : • big-endian <code>></code> • little-endian <code><</code> • not applicable <code> </code>
fields	sub-dtypes, if it's a structured data type
shape	shape of the array, if it's a sub-array

```
np.dtype(int).type
```

```
numpy.int64
```

```
np.dtype(int).itemsize
```

```
8
```

```
np.dtype(int).byteorder
```

```
'='
```

Example: reading `.wav` files

The `.wav` file header:

chunk_id	"RIFF"
chunk_size	4-byte unsigned little-endian integer
format	"WAVE"
fmt_id	"fmt "
fmt_size	4-byte unsigned little-endian integer
audio_fmt	2-byte unsigned little-endian integer
num_channels	2-byte unsigned little-endian integer
sample_rate	4-byte unsigned little-endian integer
byte_rate	4-byte unsigned little-endian integer
block_align	2-byte unsigned little-endian integer
bits_per_sample	2-byte unsigned little-endian integer
data_id	"data"
data_size	4-byte unsigned little-endian integer

- 44-byte block of raw data (in the beginning of the file)
- ... followed by `data_size` bytes of actual sound data.

The `.wav` file header as a NumPy *structured* data type:

```
wav_header_dtype = np.dtype([
    ("chunk_id", (bytes, 4)), # flexible-sized scalar type, item size 4
    ("chunk_size", "<u4"), # little-endian unsigned 32-bit integer
    ("format", "S4"), # 4-byte string
    ("fmt_id", "S4"),
    ("fmt_size", "<u4"),
    ("audio_fmt", "<u2"), #
    ("num_channels", "<u2"), # .. more of the same ...
    ("sample_rate", "<u4"), #
    ("byte_rate", "<u4"),
    ("block_align", "<u2"),
    ("bits_per_sample", "<u2"),
    ("data_id", ("S1", (2, 2))), # sub-array, just for fun!
    ("data_size", "u4"),
    #
    # the sound data itself cannot be represented here:
    # it does not have a fixed size
])
```

See also

[wavreader.py](#)

`wav_header_dtype['format']`

`dtype('S4')`

`wav_header_dtype.fields`

```
mappingproxy({'chunk_id': (dtype('S4'), 0),
              'chunk_size': (dtype('uint32'), 4),
              'format': (dtype('S4'), 8),
              'fmt_id': (dtype('S4'), 12),
              'fmt_size': (dtype('uint32'), 16),
              'audio_fmt': (dtype('uint16'), 20),
              'num_channels': (dtype('uint16'), 22),
              'sample_rate': (dtype('uint32'), 24),
              'byte_rate': (dtype('uint32'), 28),
              'block_align': (dtype('uint16'), 32),
              'bits_per_sample': (dtype('uint16'), 34),
              'data_id': (dtype(('S1', (2, 2))), 36),
              'data_size': (dtype('uint32'), 40)})
```

`wav_header_dtype.fields['format']`

```
(dtype('S4'), 8)
```

- The first element is the sub-dtype in the structured data, corresponding to the name `format`
- The second one is its offset (in bytes) from the beginning of the item

Exercise 43



Make a “sparse” dtype by using offsets, and only some of the fields:

```
wav_header_dtype = np.dtype(dict(  
    names=['format', 'sample_rate', 'data_id'],  
    offsets=[offset_1, offset_2, offset_3], # counted from start of  
    structure in bytes  
    formats=list of dtypes for each of the fields,  
))
```

```
Cell In[18], line 4  
    formats=list of dtypes for each of the fields,  
          ^  
SyntaxError: invalid syntax. Perhaps you forgot a comma?
```

and use that to read the sample rate, and `data_id` (as sub-array).

```
f = open('data/test.wav', 'r')  
wav_header = np.fromfile(f, dtype=wav_header_dtype, count=1)  
f.close()  
print(wav_header)
```

```
[(b'RIFF', 17402, b'WAVE', b'fmt ', 16, 1, 1, 16000, 32000, 2, 16, [[b'd',  
b'a'], [b't', b'a']], 17366)]
```

```
wav_header['sample_rate']
```

```
array([16000], dtype=uint32)
```

Let's try accessing the sub-array:

```
wav_header['data_id']
```

```
array([[b'd', b'a'],  
       [b't', b'a']], dtype='<S1')
```

```
wav_header.shape
```

```
(1,)
```

```
wav_header['data_id'].shape
```

```
(1, 2, 2)
```

When accessing sub-arrays, the dimensions get added to the end!

Note

There are existing modules such as `wavfile`, `audiolab`, etc. for loading sound data...

Casting and re-interpretation/views

casting

- on assignment
- on array construction
- on arithmetic
- etc.
- and manually: `.astype(dtype)`

data re-interpretation

- manually: `.view(dtype)`

Casting

- Casting in arithmetic, in nutshell:
 - only type (not value!) of operands matters
 - largest “safe” type able to represent both is picked
 - scalars can “lose” to arrays in some situations

- Casting in general copies data:

```
x = np.array([1, 2, 3, 4], dtype=float)
x
```

```
array([1., 2., 3., 4.])
```

```
y = x.astype(np.int8)
y
```

```
array([1, 2, 3, 4], dtype=int8)
```

```
y + 1
```

```
array([2, 3, 4, 5], dtype=int8)
```

```
y + 256
```

```
-----
OverflowError                                Traceback (most recent call last)
Cell In[27], line 1
----> 1 y + 256

OverflowError: Python integer 256 out of bounds for int8
```

```
y + 256.0
```

```
array([257., 258., 259., 260.])
```

```
y + np.array([256], dtype=np.int32)
```

```
array([257, 258, 259, 260], dtype=int32)
```

- Casting on setitem: dtype of the array is not changed on item assignment:

```
y[:] = y + 1.5
y
```

```
array([2, 3, 4, 5], dtype=int8)
```

Note

Exact rules: see [NumPy documentation](#)

Re-interpretation / viewing

Let's say we have a data block in memory (4 bytes). For the moment (as indicated by the bars between the values), we'll consider this to be four `uint8` values:



However, we can interpret this block as:

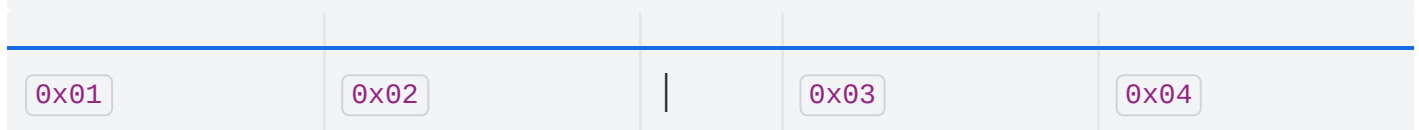
- 4 of `uint8` (as here), OR,
- 4 of `int8`, OR,
- 2 of `int16`, OR,
- 1 of `int32`, OR,
- 1 of `float32`, OR,
- ...

How to switch from one to another?

Option 1: Switch the dtype

```
x = np.array([1, 2, 3, 4], dtype=np.uint8)
x.dtype = "<i2"
x
```

```
array([ 513, 1027], dtype=int16)
```



Note

little-endian: least significant byte is on the *left* in memory

Option 2: Create a new view of type `uint32`, shorthand `i4`

```
y = x.view("<i4")  
y
```

```
array([67305985], dtype=int32)
```

0x01

0x02

0x03

0x04

On Views

- `.view()` makes *views*, does not copy (or alter) the memory block
- it only changes the dtype (and adjusts array shape):

```
x[1] = 5  
y
```

```
array([328193], dtype=int32)
```

```
y.base is x
```

True

Mini-exercise: data re-interpretation

See also

[view-colors.py](#)

Exercise 44



You have RGBA data in an array:

```
x = np.zeros((10, 10, 4), dtype=np.int8)
x[:, :, 0] = 1
x[:, :, 1] = 2
x[:, :, 2] = 3
x[:, :, 3] = 4
```

where the last three dimensions are the R, B, and G, and alpha channels.

How would you make a (10, 10) structured array with field names 'r', 'g', 'b', 'a' without copying data?

```
y = ...
```

```
assert (y['r'] == 1).all()
assert (y['g'] == 2).all()
assert (y['b'] == 3).all()
assert (y['a'] == 4).all()
```

```
-----
----
TypeError                                Traceback (most recent call
last)
Cell In[37], line 1
----> 1 assert (y['r'] == 1).all()
      2 assert (y['g'] == 2).all()
      3 assert (y['b'] == 3).all()

TypeError: 'ellipsis' object is not subscriptable
```

Solution to [Exercise 44](#)



```
y = x.view([('r', 'i1'),
            ('g', 'i1'),
            ('b', 'i1'),
            ('a', 'i1')])[:, :, 0]
```

A puzzle

Another two arrays, each occupying exactly 4 bytes of memory:

```
x = np.array([[1, 3], [2, 4]], dtype=np.uint8)
x
```

```
array([[1, 3],
       [2, 4]], dtype=uint8)
```

```
y = x.transpose()
y
```

```
array([[1, 2],
       [3, 4]], dtype=uint8)
```

We view the elements of `x` (1 byte each) as `int16` (2 bytes each):

```
x.view(np.int16)
```

```
array([[ 769],
       [1026]], dtype=int16)
```

What is happening here? Take a look at the bytes stored in memory by `x`:

```
x.tobytes()
```

```
b'\x01\x03\x02\x04'
```

The `\x` stands for hexadecimal, so what we are seeing is:

```
0x01 0x03 0x02 0x04
```

We ask NumPy to interpret these bytes as elements of dtype `int16`—each of which occupies *two* bytes in memory. Therefore, `0x01 0x03` becomes the first `uint16` and `0x02 0x04` the second.

You may then expect to see `0x0103` (259, when converting from hexadecimal to decimal) as the first result. But your computer likely stores most significant bytes first, and as such reads the number as `0x0301` or 769 (go on and type `0x0301` into your Python terminal to verify).

We can do the same on a copy of `y` (why doesn't it work on `y` directly?):

```
y.copy().view(np.int16)
```

```
array([[ 513],  
       [1027]], dtype=int16)
```

Can you explain these numbers, 513 and 1027, as well as the output shape of the resulting array?

Indexing scheme: strides

Main point

The question:

```
x = np.array([[1, 2, 3],  
              [4, 5, 6],  
              [7, 8, 9]], dtype=np.int8)  
x.tobytes('A')
```

```
b'\x01\x02\x03\x04\x05\x06\x07\x08\t'
```

The answer (in NumPy)

- **strides**: the number of bytes to jump to find the next element
- 1 stride per dimension

```
x.strides
```

```
(3, 1)
```

```
byte_offset = 3 * 1 + 1 * 2 # to find x[1, 2]  
x.flat[byte_offset]
```

```
np.int8(6)
```

```
x[1, 2]
```

```
np.int8(6)
```

simple, **flexible**

C and Fortran order

Note

The Python built-in `bytes` returns bytes in C-order by default which can cause confusion when trying to inspect memory layout. We use `numpy.ndarray.tobytes()` with `order=A` instead, which preserves the C or F ordering of the bytes in memory.

```
x = np.array([[1, 2, 3],  
              [4, 5, 6]], dtype=np.int16, order='C')  
x.strides
```

```
(6, 2)
```

```
x.tobytes('A')
```

```
b'\x01\x00\x02\x00\x03\x00\x04\x00\x05\x00\x06\x00'
```

- Need to jump 6 bytes to find the next row
- Need to jump 2 bytes to find the next column

```
y = np.array(x, order='F')  
y.strides
```

```
(2, 4)
```

```
y.tobytes('A')
```

```
b'\x01\x00\x04\x00\x02\x00\x05\x00\x03\x00\x06\x00'
```

- Need to jump 2 bytes to find the next row
- Need to jump 4 bytes to find the next column

Similarly for higher dimensions:

- C: last dimensions vary fastest (= smaller strides)
- F: first dimensions vary fastest

$$\begin{aligned}\text{shape} &= (d_1, d_2, \dots, d_n) \\ \text{strides} &= (s_1, s_2, \dots, s_n) \\ s_j^C &= d_{j+1}d_{j+2}\dots d_n \times \text{itemsize} \\ s_j^F &= d_1d_2\dots d_{j-1} \times \text{itemsize}\end{aligned}$$

Now we can understand the behavior of `.view()`

```
y = np.array([[1, 3], [2, 4]], dtype=np.uint8).transpose()
x = y.copy()
```

Transposition does not affect the memory layout of the data, only strides

```
x.strides
```

```
(2, 1)
```

```
y.strides
```

```
(1, 2)
```

```
x.tobytes('A')
```

```
b'\x01\x02\x03\x04'
```

```
y.tobytes('A')
```

```
b'\x01\x03\x02\x04'
```

- the results are different when interpreted as 2 of int16
- `.copy()` creates new arrays in the C order (by default)

Slicing with integers

- *Everything* can be represented by changing only `shape`, `strides`, and possibly adjusting the `data` pointer!
- Never makes copies of the data

```
x = np.array([1, 2, 3, 4, 5, 6], dtype=np.int32)
y = x[::-1]
y
```

```
array([6, 5, 4, 3, 2, 1], dtype=int32)
```

```
y.strides
```

```
(-4,)
```

```
y = x[2:]
y.__array_interface__['data'][0] - x.__array_interface__['data'][0]
```

```
8
```

```
x = np.zeros((10, 10, 10), dtype=float)
x.strides
```

```
(800, 80, 8)
```

```
x[:,::2,::3,::4].strides
```

```
(1600, 240, 32)
```

Similarly, transposes never make copies (it just swaps strides):

```
x = np.zeros((10, 10, 10), dtype=float)
x.strides
```

```
(800, 80, 8)
```

```
x.T.strides
```

```
(8, 80, 800)
```

But: not all reshaping operations can be represented by playing with strides:

```
a = np.arange(6, dtype=np.int8).reshape(3, 2)
b = a.T
b.strides
```

```
(1, 2)
```

So far, so good. However:

```
bytes(a.data)
```

```
b'\x00\x01\x02\x03\x04\x05'
```

```
b
```

```
array([[0, 2, 4],
       [1, 3, 5]], dtype=int8)
```

```
c = b.reshape(3*2)
c
```

```
array([0, 2, 4, 1, 3, 5], dtype=int8)
```

Here, there is no way to represent the array `c` given one stride and the block of memory for `a`. Therefore, the `reshape` operation needs to make a copy here.

Example: fake dimensions with strides

Stride manipulation

```
from numpy.lib.stride_tricks import as_strided
help(as_strided)
```

Help on function `as_strided` in module `numpy.lib.stride_tricks`:

`as_strided(x, shape=None, strides=None, subok=False, writeable=True)`
Create a view into the array with the given shape and strides.

.. warning:: This function has to be used with extreme care, see notes.

Parameters

`x` : ndarray

Array to create a new.

`shape` : sequence of int, optional

The shape of the new array. Defaults to ``x.shape``.

`strides` : sequence of int, optional

The strides of the new array. Defaults to ``x.strides``.

`subok` : bool, optional

If True, subclasses are preserved.

`writeable` : bool, optional

If set to False, the returned array will always be readonly.

Otherwise it will be writable if the original array was. It is advisable to set this to False if possible (see Notes).

Returns

`view` : ndarray

See also

`broadcast_to` : broadcast an array to a given shape.

`reshape` : reshape an array.

`lib.stride_tricks.sliding_window_view` :

userfriendly and safe function for a creation of sliding window views.

Notes

``as\_strided`` creates a view into the array given the exact strides and shape. This means it manipulates the internal data structure of ndarray and, if done incorrectly, the array elements can point to invalid memory and can corrupt results or crash your program. It is advisable to always use the original ``x.strides`` when calculating new strides to avoid reliance on a contiguous memory layout.

Furthermore, arrays created with this function often contain self overlapping memory, so that two elements are identical. Vectorized write operations on such arrays will typically be unpredictable. They may even give different results for small, large, or transposed arrays.

Since writing to these arrays has to be tested and done with great care, you may want to use ``writeable=False`` to avoid accidental write operations.

For these reasons it is advisable to avoid ``as\_strided`` when possible.

Warning

`as_strided` does **not** check that you stay inside the memory block bounds...

```
x = np.array([1, 2, 3, 4], dtype=np.int16)
as_strided(x, strides=(2*2, ), shape=(2, ))
```

```
array([1, 3], dtype=int16)
```

```
x[:, :2]
```

```
array([1, 3], dtype=int16)
```

See also

[stride-fakedims.py](#)

Exercise 45

Convert this:

```
in_arr = np.array([1, 2, 3, 4], dtype=np.int8)
in_arr
```

```
array([1, 2, 3, 4], dtype=int8)
```

to this:

```
array([[1, 2, 3, 4],
       [1, 2, 3, 4],
       [1, 2, 3, 4]], dtype=np.int8)
```

using only `as_strided` .:

Hint: `byte_offset = stride[0]*index[0] + stride[1]*index[1] + ...`

Spoiler for strides exercise

Stride can also be 0:

Solution to [Exercise 45](#)

```
x = np.array([1, 2, 3, 4], dtype=np.int8)
y = as_strided(x, strides=(0, 1), shape=(3, 4))
y
```

```
array([[1, 2, 3, 4],
       [1, 2, 3, 4],
       [1, 2, 3, 4]], dtype=int8)
```

```
y.base.base is x
```

```
True
```

Broadcasting

- Doing something useful with it: outer product of `[1, 2, 3, 4]` and `[5, 6, 7]`

```
x = np.array([1, 2, 3, 4], dtype=np.int16)
x2 = as_strided(x, strides=(0, 1*2), shape=(3, 4))
x2
```

```
array([[1, 2, 3, 4],
       [1, 2, 3, 4],
       [1, 2, 3, 4]], dtype=int16)
```

```
y = np.array([5, 6, 7], dtype=np.int16)
y2 = as_strided(y, strides=(1*2, 0), shape=(3, 4))
y2
```

```
array([[5, 5, 5, 5],
       [6, 6, 6, 6],
       [7, 7, 7, 7]], dtype=int16)
```

```
x2 * y2
```

```
array([[ 5, 10, 15, 20],  
       [ 6, 12, 18, 24],  
       [ 7, 14, 21, 28]], dtype=int16)
```

... seems somehow familiar ...

```
x = np.array([1, 2, 3, 4], dtype=np.int16)  
y = np.array([5, 6, 7], dtype=np.int16)  
x[np.newaxis,:] * y[:,np.newaxis]
```

```
array([[ 5, 10, 15, 20],  
       [ 6, 12, 18, 24],  
       [ 7, 14, 21, 28]], dtype=int16)
```

- Internally, array **broadcasting** is indeed implemented using 0-strides.

More tricks: diagonals

See also

[stride-diagonals.py](#)

Exercise 46

Pick diagonal entries of the matrix: (assume C memory order):

```
x = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]], dtype=np.int32)
```

Your task is to work out the correct strides for to get the diagonal of the array, as in:

```
x_diag = as_strided(x, shape=(3,), strides=(...,))
```

Next:

- Pick the first super-diagonal entries `[2, 6]`.
- And the sub-diagonals?

Hint to the last two: slicing first moves the point where striding starts from.

Solution to [Exercise 46](#)

Pick diagonals:

```
x_diag = as_strided(x, shape=(3, ), strides=((3+1)*x.itemsize,))
x_diag
```

```
array([1, 5, 9], dtype=int32)
```

Slice first, to adjust the data pointer:

```
as_strided(x[0, 1:], shape=(2, ), strides=((3+1)*x.itemsize, ))
```

```
array([2, 6], dtype=int32)
```

```
as_strided(x[1:, 0], shape=(2, ), strides=((3+1)*x.itemsize, ))
```

```
array([4, 8], dtype=int32)
```

Using np.diag

```
y = np.diag(x, k=1)  
y
```

```
array([2, 6], dtype=int32)
```

However,

```
y.flags.owndata
```

```
False
```

Challenge

Exercise 47

Compute the tensor trace:

```
x = np.arange(5*5*5*5).reshape(5, 5, 5, 5)
s = 0
for i in range(5):
    for j in range(5):
        s += x[j, i, j, i]
```

by striding, and using `sum()` on the result.

```
y = as_strided(x, shape=(5, 5), strides=(..., ...))
s2 = ...
assert s == s2
```

```
-----
----
TypeError                                Traceback (most recent call
last)
Cell In[85], line 1
----> 1 y = as_strided(x, shape=(5, 5), strides=(..., ...))
      2 s2 = ...
      3 assert s == s2

File /opt/hostedtoolcache/Python/3.12.12/x64/lib/python3.12/site-
packages/numpy/lib/_stride_tricks_impl.py:101, in as_strided(x, shape,
strides, subok, writeable)
      98 if strides is not None:
      99     interface['strides'] = tuple(strides)
--> 101 array = np.asarray(DummyArray(interface, base=x))
     102 # The route via `__interface__` does not preserve structured
     103 # dtypes. Since dtype should remain unchanged, we set it
explicitly.
     104 array.dtype = x.dtype

TypeError: 'ellipsis' object cannot be interpreted as an integer
```

Solution to [Exercise 47](#)

```
y = as_strided(x, shape=(5, 5), strides=((5*5*5 + 5)*x.itemsize,  
                                           (5*5 + 1)*x.itemsize))  
s2 = y.sum()  
s2
```

```
np.int64(7800)
```

CPU cache effects

Memory layout can affect performance:

```
x = np.zeros((20000,))  
y = np.zeros((20000*67,))[:,67]  
  
x.shape, y.shape
```

```
((20000,), (20000,))
```

```
%timeit np.median(x)
```

64.2 μ s $\pm$ 349 ns per loop (mean $\pm$ std. dev. of 7 runs, 10,000 loops each)

```
%timeit np.median(y)
```

93.4 μ s $\pm$ 813 ns per loop (mean $\pm$ std. dev. of 7 runs, 10,000 loops each)

```
x.strides, y.strides
```

```
((8,), (536,))
```

Note

**** Are smaller strides faster****

x



y



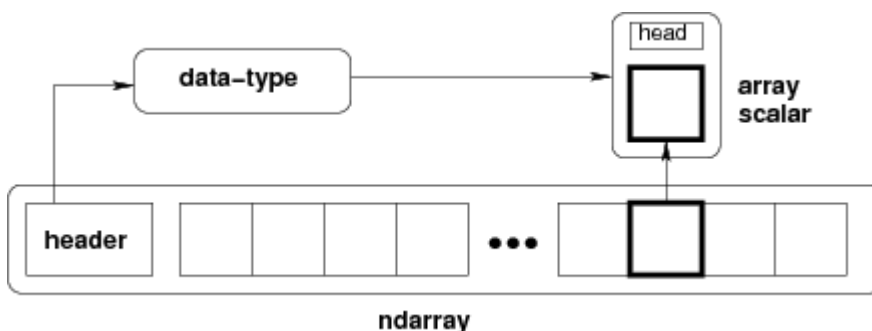
cache block size

- CPU pulls data from main memory to its cache in blocks
- If many array items consecutively operated on fit in a single block (small stride):
 - $\Rightarrow$ fewer transfers needed
 - $\Rightarrow$ faster

See also

- [numexpr](#) is designed to mitigate cache effects when evaluating array expressions.
- [numba](#) is a compiler for Python code, that is aware of numpy arrays.

Findings in dissection



- *memory block*: may be shared, `.base`, `.data`
- *data type descriptor*: structured data, sub-arrays, byte order, casting, viewing, `.astype()`, `.view()`

- *strided indexing*: strides, C/F-order, slicing w/ integers, `as_strided`, broadcasting, stride tricks, `diag`, CPU cache coherence

Universal functions

What are they?

- Ufunc performs an elementwise operation on all elements of an array.

Examples: `np.add`, `np.subtract`, `scipy.special.*`, ...

- Automatically support: broadcasting, casting, ...
- The author of an ufunc only has to supply the elementwise operation, NumPy takes care of the rest.
- The elementwise operation needs to be implemented in C (or, e.g., Cython)

Parts of an Ufunc

Part 1: provided by user

```
void ufunc_loop(void **args, int *dimensions, int *steps, void *data)
{
    /*
     * int8 output = elementwise_function(int8 input_1, int8 input_2)
     *
     * This function must compute the ufunc for many values at once,
     * in the way shown below.
     */
    char *input_1 = (char*)args[0];
    char *input_2 = (char*)args[1];
    char *output = (char*)args[2];
    int i;

    for (i = 0; i < dimensions[0]; ++i) {
        *output = elementwise_function(*input_1, *input_2);
        input_1 += steps[0];
        input_2 += steps[1];
        output += steps[2];
    }
}
```

Part 2. The NumPy part, built by

```

char types[3]

types[0] = NPY_BYTE    /* type of first input arg */
types[1] = NPY_BYTE    /* type of second input arg */
types[2] = NPY_BYTE    /* type of third input arg */

PyObject *python_ufunc = PyUFunc_FromFuncAndData(
    ufunc_loop,
    NULL,
    types,
    1, /* ntypes */
    2, /* num_inputs */
    1, /* num_outputs */
    identity_element,
    name,
    docstring,
    unused)

```

A ufunc can also support multiple different input-output type combinations.

Making it easier

`ufunc_loop` is of very generic form, and NumPy provides pre-made ones

<code>PyUfunc_f_f</code>	<code>float elementwise_func(float input_1)</code>
<code>PyUfunc_ff_f</code>	<code>float elementwise_func(float input_1, float input_2)</code>
<code>PyUfunc_d_d</code>	<code>double elementwise_func(double input_1)</code>
<code>PyUfunc_dd_d</code>	<code>double elementwise_func(double input_1, double input_2)</code>
<code>PyUfunc_D_D</code>	<code>elementwise_func(np_cdouble *input, np_cdouble* output)</code>
<code>PyUfunc_DD_D</code>	<code>elementwise_func(np_cdouble *in1, np_cdouble *in2, np_cdouble* out)</code>

- Only `elementwise_func` needs to be supplied
- ... except when your elementwise function is not in one of the above forms

Exercise: building an ufunc from scratch

Exercise 48

The Mandelbrot fractal is defined by the iteration

$$z \leftarrow z^2 + c$$

where $c = x + iy$ is a complex number. This iteration is repeated – if z stays finite no matter how long the iteration runs, c belongs to the Mandelbrot set.

First — make a ufunc called `mandel(z0, c)` that computes:

```
z = z0
for k in range(iterations):
    z = z*z + c
```

Run for, say, 100 iterations or until `z.real**2 + z.imag**2 > 1000`. Use it to determine which `c` are in the Mandelbrot set.

Our function is a simple one, so make use of the `PyUFunc_*` helpers.

Write it in Cython

See also

mandel.pyx, [mandelplot.py](#)

Reminder: some pre-made Ufunc loops:

PyUfunc_f_f	float elementwise_func(float input_1)
PyUfunc_ff_f	float elementwise_func(float input_1, float input_2)
PyUfunc_d_d	double elementwise_func(double input_1)
PyUfunc_dd_d	double elementwise_func(double input_1, double input_2)
PyUfunc_D_D	elementwise_func(complex_double *input, complex_double* output)
PyUfunc_DD_D	elementwise_func(complex_double *in1, complex_double *in2, complex_double* out)

Type codes:

```
NPY_BOOL, NPY_BYTE, NPY_UBYTE, NPY_SHORT, NPY_USHORT, NPY_INT,
NPY_UINT,
NPY_LONG, NPY_ULONG, NPY_LONGLONG, NPY_ULONGLONG, NPY_FLOAT,
NPY_DOUBLE,
NPY_LONGDOUBLE, NPY_CFLOAT, NPY_CDOUBLE, NPY_CLONGDOUBLE, NPY_DATETIME,
NPY_TIMEDELTA, NPY_OBJECT, NPY_STRING, NPY_UNICODE, NPY_VOID
```

Solution to [Exercise 48](#)

```

# The elementwise function
# -----

cdef void mandel_single_point(double complex *z_in,
                             double complex *c_in,
                             double complex *z_out) noexcept nogil:

    #
    # The Mandelbrot iteration
    #

    #
    # Some points of note:
    #
    # - It's *NOT* allowed to call any Python functions here.
    #
    # The Ufunc loop runs with the Python Global Interpreter Lock
    released.
    # Hence, the ``nogil``.
    #
    # - And so all local variables must be declared with ``cdef``
    #
    # - Note also that this function receives *pointers* to the data;
    # the "traditional" solution to passing complex variables around
    #

    cdef double complex z = z_in[0]
    cdef double complex c = c_in[0]
    cdef int k # the integer we use in the for loop

    # Straightforward iteration

    for k in range(100):
        z = z*z + c
        if z.real**2 + z.imag**2 > 1000:
            break

    # Return the answer for this point
    z_out[0] = z

# Boilerplate Cython definitions
#
# Pulls definitions from the NumPy C headers.
# -----

from numpy cimport import_array, import_ufunc
from numpy cimport (PyUFunc_FromFuncAndData,
                    PyUFuncGenericFunction)
from numpy cimport NPY_CDOUBLE
from numpy cimport PyUFunc_DD_D

# Required module initialization
# -----

import_array()
import_ufunc()

```

```

# The actual ufunc declaration
# -----

cdef PyUFuncGenericFunction loop_func[1]
cdef char input_output_types[3]
cdef void *elementwise_funcs[1]

loop_func[0] = PyUFunc_DD_D

input_output_types[0] = NPY_CDOUBLE
input_output_types[1] = NPY_CDOUBLE
input_output_types[2] = NPY_CDOUBLE

elementwise_funcs[0] = <void*>mandel_single_point

mandel = PyUFunc_FromFuncAndData(
    loop_func,
    elementwise_funcs,
    input_output_types,
    1, # number of supported input types
    2, # number of input args
    1, # number of output args
    0, # `identity` element, never mind this
    "mandel", # function name
    "mandel(z, c) -> computes iterated z*z + c", # docstring
    0 # unused
)

```

```

"""
Plot Mandelbrot
=====

Plot the Mandelbrot ensemble.

"""

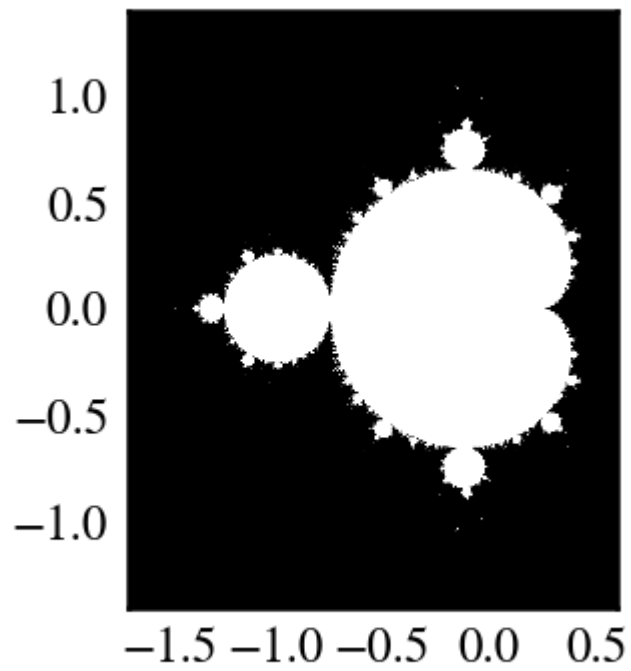
import numpy as np
import mandel

x = np.linspace(-1.7, 0.6, 1000)
y = np.linspace(-1.4, 1.4, 1000)
c = x[None, :] + 1j * y[:, None]
z = mandel.mandel(c, c)

import matplotlib.pyplot as plt

plt.imshow(abs(z) ** 2 < 1000, extent=[-1.7, 0.6, -1.4, 1.4])
plt.gray()
plt.show()

```



Note

Most of the boilerplate could be automated by these Cython modules:

[cython/cython](https://github.com/cython/cython)

Several accepted input types

E.g. supporting both single- and double-precision versions

```

cdef void mandel_single_point(double complex *z_in,
                             double complex *c_in,
                             double complex *z_out) nogil:
    ...

cdef void mandel_single_point_singleprec(float complex *z_in,
                                         float complex *c_in,
                                         float complex *z_out) nogil:
    ...

cdef PyUFuncGenericFunction loop_funcs[2]
cdef char input_output_types[3*2]
cdef void *elementwise_funcs[1*2]

loop_funcs[0] = PyUFunc_DD_D
input_output_types[0] = NPY_CDOUBLE
input_output_types[1] = NPY_CDOUBLE
input_output_types[2] = NPY_CDOUBLE
elementwise_funcs[0] = <void*>mandel_single_point

loop_funcs[1] = PyUFunc_FF_F
input_output_types[3] = NPY_CFLOAT
input_output_types[4] = NPY_CFLOAT
input_output_types[5] = NPY_CFLOAT
elementwise_funcs[1] = <void*>mandel_single_point_singleprec

mandel = PyUFunc_FromFuncAndData(
    loop_func,
    elementwise_funcs,
    input_output_types,
    2, # number of supported input types  <-----
    2, # number of input args
    1, # number of output args
    0, # `identity` element, never mind this
    "mandel", # function name
    "mandel(z, c) -> computes iterated  $z*z + c$ ", # docstring
    0 # unused
)

```

Generalized ufuncs

ufunc

```
output = elementwise_function(input)
```

Both `output` and `input` can be a single array element only.

generalized ufunc

```
output and input can be arrays with a fixed number of dimensions
```

For example, matrix trace (sum of diag elements):

```
input shape = (n, n)
output shape = ()      # i.e.  scalar

(n, n) -> ()
```

Matrix product:

```
input_1 shape = (m, n)
input_2 shape = (n, p)
output shape  = (m, p)

(m, n), (n, p) -> (m, p)
```

- This is called the “*signature*” of the generalized ufunc
- The dimensions on which the g-ufunc acts, are “*core dimensions*”

Status in NumPy

- g-ufuncs are in NumPy already ...
- new ones can be created with `PyUFunc_FromFuncAndDataAndSignature`
- most linear-algebra functions are implemented as g-ufuncs to enable working with stacked arrays:

```
import numpy as np
rng = np.random.default_rng(27446968)
np.linalg.det(rng.random((3, 5, 5)))
```

```
array([ 0.01829761, -0.0077266 , -0.05336566])
```

```
np.linalg._umath_linalg.det.signature
```

```
'(m,m)->()'
```

- matrix multiplication this way could be useful for operating on many small matrices at once
- Also see `tensordot` and `einsum`

Generalized ufunc loop

Matrix multiplication `(m,n), (n,p) -> (m,p)`

```

void gufunc_loop(void **args, int *dimensions, int *steps, void *data)
{
    char *input_1 = (char*)args[0]; /* these are as previously */
    char *input_2 = (char*)args[1];
    char *output = (char*)args[2];

    int input_1_stride_m = steps[3]; /* strides for the core dimensions */
    int input_1_stride_n = steps[4]; /* are added after the non-core */
    int input_2_strides_n = steps[5]; /* steps */
    int input_2_strides_p = steps[6];
    int output_strides_n = steps[7];
    int output_strides_p = steps[8];

    int m = dimension[1]; /* core dimensions are added after */
    int n = dimension[2]; /* the main dimension; order as in */
    int p = dimension[3]; /* signature */

    int i;

    for (i = 0; i < dimensions[0]; ++i) {
        matmul_for_strided_matrices(input_1, input_2, output,
                                    strides for each array...);

        input_1 += steps[0];
        input_2 += steps[1];
        output += steps[2];
    }
}

```

Interoperability features

Sharing multidimensional, typed data

Suppose you

1. Write a library than handles (multidimensional) binary data,
2. Want to make it easy to manipulate the data with NumPy, or whatever other library,
3. ... but would **not** like to have NumPy as a dependency.

Currently, 3 solutions:

1. the “old” buffer interface
2. the array interface
3. the “new” buffer interface ([PEP 3118](#))

The old buffer protocol

- Only 1-D buffers
- No data type information
- C-level interface; `PyBufferProcs tp_as_buffer` in the type object
- But it's integrated into Python (e.g. strings support it)

Mini-exercise using [Pillow](#) (Python Imaging Library):

See also

[pilbuffer.py](#)

Exercise 49

```
from PIL import Image
data = np.zeros((200, 200, 4), dtype=np.uint8)
data[:, :] = [255, 0, 0, 255] # Red
# In PIL, RGBA images consist of 32-bit integers whose bytes are
# [RR, GG, BB, AA]
data = data.view(np.int32).squeeze()
img = Image.frombuffer("RGBA", (200, 200), data, "raw", "RGBA", 0, 1)
img.save('test.png')
```

The question

What happens if `data` is now modified, and `img` saved again?

The old buffer protocol

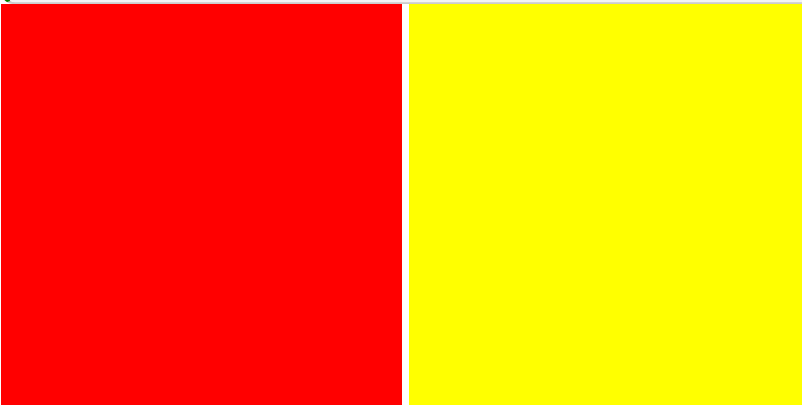
Show how to exchange data between numpy and a library that only knows the buffer interface:

```
# Make a sample image, RGBA format
x = np.zeros((200, 200, 4), dtype=np.uint8)
x[:, :, 0] = 255 # red
x[:, :, 3] = 255 # opaque

data_i32 = x.view(np.int32) # Check that you understand why this is OK!

img = Image.frombuffer("RGBA", (200, 200), data_i32)
img.save("test_red.png")

# Modify the original data, and save again.
x[:, :, 1] = 255
img.save("test_recolored.png")
```



Array interface protocol

- Multidimensional buffers
- Data type information present
- NumPy-specific approach; slowly deprecated (but not going away)
- Not integrated in Python otherwise

See also

Documentation: <https://numpy.org/doc/stable/reference/arrays.interface.html>

```
x = np.array([[1, 2], [3, 4]])
x.__array_interface__
```

```
{'data': (94910831978128, False),  
  'strides': None,  
  'descr': [('', '<i8')],  
  'typestr': '<i8',  
  'shape': (2, 2),  
  'version': 3}
```

► Show code cell source

```
from PIL import Image
img = Image.open('data/test.png')
img.__array_interface__
```

[illegible]

```
x = np.asarray(img)
x.shape
```

(200, 200, 4)

Note

A more C-friendly variant of the array interface is also defined.

Array siblings: `chararray`, `MaskedArray`

`chararray`: vectorized string operations

```
x = np.char.asarray(['a', '  bbb', '  ccc'])  
x
```

```
chararray(['a', '  bbb', '  ccc'], dtype='<U5')
```

```
x.upper()
```

```
chararray(['A', '  BBB', '  CCC'], dtype='<U5')
```

`MaskedArray` missing data

Masked arrays are arrays that may have missing or invalid entries.

For example, suppose we have an array where the fourth entry is invalid:

```
x = np.array([1, 2, 3, -99, 5])
```

One way to describe this is to create a masked array:

```
mx = np.ma.MaskedArray(x, mask=[0, 0, 0, 1, 0])  
mx
```

```
masked_array(data=[1, 2, 3, --, 5],  
             mask=[False, False, False,  True, False],  
             fill_value=999999)
```

Masked mean ignores masked data:

```
mx.mean()
```

```
np.float64(2.75)
```

```
np.mean(mx)
```

```
np.float64(2.75)
```

⚠ Warning

Not all NumPy functions respect masks, for instance `np.dot`, so check the return types.

The `MaskedArray` returns a **view** to the original array:

```
mx[1] = 9  
x
```

```
array([ 1,  9,  3, -99,  5])
```

The mask

You can modify the mask by assigning:

```
mx[1] = np.ma.masked  
mx
```

```
masked_array(data=[1, --, 3, --, 5],  
             mask=[False,  True, False,  True, False],  
             fill_value=999999)
```

The mask is cleared on assignment:

```
mx[1] = 9  
mx
```

```
masked_array(data=[1, 9, 3, --, 5],  
             mask=[False, False, False,  True, False],  
             fill_value=999999)
```

The mask is also available directly:

```
mx.mask
```

```
array([False, False, False,  True, False])
```

The masked entries can be filled with a given value to get an usual array back:

```
x2 = mx.filled(-1)
x2
```

```
array([ 1,  9,  3, -1,  5])
```

The mask can also be cleared:

```
mx.mask = np.ma.nomask
mx
```

```
masked_array(data=[1, 9, 3, -99, 5],
             mask=[False, False, False, False, False],
             fill_value=999999)
```

Domain-aware functions

The masked array package also contains domain-aware functions:

```
np.ma.log(np.array([1, 2, -1, -2, 3, -5]))
```

```
masked_array(data=[0.0, 0.6931471805599453, --, --, 1.0986122886681098,
                  --],
             mask=[False, False,  True,  True, False,  True],
             fill_value=1e+20)
```

Note

Streamlined and more seamless support for dealing with missing data in arrays is making its way into NumPy 1.7. Stay tuned!

Example: Masked statistics

Canadian rangers were distracted when counting hares and lynxes in 1903-1910 and 1917-1918, and got the numbers are wrong. (Carrot farmers stayed alert, though.) Compute the mean populations over time, ignoring the invalid numbers.

```
data = np.loadtxt('data/populations.txt')
populations = np.ma.MaskedArray(data[:,1:])
year = data[:, 0]
```

```
bad_years = (((year >= 1903) & (year <= 1910))
              | ((year >= 1917) & (year <= 1918)))
# '&' means 'and' and '|' means 'or'
populations[bad_years, 0] = np.ma.masked
populations[bad_years, 1] = np.ma.masked
```

```
populations.mean(axis=0)
```

```
masked_array(data=[40472.72727272727, 18627.272727272728, 42400.0],
              mask=[False, False, False],
              fill_value=1e+20)
```

```
populations.std(axis=0)
```

```
masked_array(data=[21087.656489006717, 15625.799814240254,
                  3322.5062255844787],
              mask=[False, False, False],
              fill_value=1e+20)
```

Note that Matplotlib knows about masked arrays:

```
plt.plot(year, populations, 'o-')
```

```
[<matplotlib.lines.Line2D at 0x7fadb9da7290>,
 <matplotlib.lines.Line2D at 0x7fadb9d06cc0>,
 <matplotlib.lines.Line2D at 0x7fadb9db2f60>]
```

`np.recarray`: purely convenience

```
arr = np.array([('a', 1), ('b', 2)], dtype=[('x', 'S1'), ('y', int)])
arr2 = arr.view(np.recarray)
arr2.x
```

```
array([b'a', b'b'], dtype='<S1')
```

```
arr2.y
```

```
array([1, 2])
```

Summary

- Anatomy of the ndarray: data, dtype, strides.
- Universal functions: elementwise operations, how to make new ones
- Numpy subclasses
- Various buffer interfaces for integration with other tools
- Recent additions: PEP 3118, generalized ufuncs

Contributing to NumPy/SciPy

Get this tutorial: <https://www.euroscipy.org/talk/882>

Why

- “There’s a bug?”
- “I don’t understand what this is supposed to do?”
- “I have this fancy code. Would you like to have it?”
- “I’d like to help! What can I do?”

Reporting bugs

- Bug tracker (prefer **this**)
 - [🐍 numpy/numpy#issues](https://github.com/numpy/numpy/issues)
 - [🐍 scipy/scipy#issues](https://github.com/scipy/scipy/issues)
 - Click the “Sign up” link to get an account
- Mailing lists (<https://numpy.org/community/>)

- If you're unsure
- No replies in a week or so? Just file a bug ticket.

Good bug report

Title: `numpy.random.permutations` fails for non-integer arguments

I'm trying to generate random permutations, using `numpy.random.permutations`

When calling `numpy.random.permutation` with non-integer arguments it fails with a cryptic error message::

```
>>> rng.permutation(12)
array([ 2,  6,  4,  1,  8, 11, 10,  5,  9,  3,  7,  0])
>>> rng.permutation(12.)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "_generator.pyx", line 4844, in
numpy.random._generator.Generator.permutation
numpy.exceptions.AxisError: axis 0 is out of bounds for array of dimension
0
```

This also happens with long arguments, and so `np.random.permutation(X.shape[0])` where `X` is an array fails on 64 bit windows (where shape is a tuple of longs).

It would be great if it could cast to integer or at least raise a proper error for non-integer types.

I'm using NumPy 1.4.1, built from the official tarball, on Windows 64 with Visual studio 2008, on Python.org 64-bit Python.

0. What are you trying to do?

1. **Small code snippet reproducing the bug** (if possible)

- What actually happens
- What you'd expect

2. Platform (Windows / Linux / OSX, 32/64 bits, x86/PPC, ...)

3. Version of NumPy/SciPy

```
print(np.__version__)
```

2.4.1

Check that the following is what you expect

```
print(np.__file__)
```

```
/opt/hostedtoolcache/Python/3.12.12/x64/lib/python3.12/site-packages/numpy/__init__.py
```

In case you have old/broken NumPy installations lying around.

If unsure, try to remove existing NumPy installations, and reinstall...

Contributing to documentation

1. Documentation editor

- <https://numpy.org/doc/stable/>
- Registration
 - Register an account
 - Subscribe to `scipy-dev` mailing list (subscribers-only)
 - Problem with mailing lists: you get mail
 - But: **you can turn mail delivery off**
 - “change your subscription options”, at the bottom of <https://mail.python.org/mailman3/lists/scipy-dev.python.org/>
 - Send a mail @ `scipy-dev` mailing list; ask for activation:

```
To: scipy-dev@scipy.org
```

```
Hi,
```

```
I'd like to edit NumPy/SciPy docstrings. My account is XXXXX
```

```
Cheers,  
N. N.
```

- Check the style guide:
 - <https://numpy.org/doc/stable/>
 - Don't be intimidated; to fix a small thing, just fix it
- Edit

2. Edit sources and send patches (as for bugs)

3. Complain on the mailing list

Contributing features

The contribution of features is documented on <https://numpy.org/doc/stable/dev/>

How to help, in general

- Bug fixes always welcome!
 - What irks you most
 - Browse the tracker
- Documentation work
 - API docs: improvements to docstrings
 - Know some SciPy module well?
 - *User guide*
 - <https://numpy.org/doc/stable/user/>
- Ask on communication channels:
 - `numpy-discussion` list
 - `scipy-dev` list

Debugging code

Contents

- Avoiding bugs
- Debugging workflow
- Using the Python debugger
- Debugging segmentation faults using gdb

Author: *Gaël Varoquaux*

This section explores tools to understand better your code base: debugging, to find and fix bugs.

It is not specific to the scientific Python community, but the strategies that we will employ are tailored to its needs.

Prerequisites

- NumPy
- IPython
- [nosetests](#)
- [pyflakes](#)
- gdb for the C-debugging part.

Avoiding bugs

Coding best practices to avoid getting in trouble

- We all write buggy code. Accept it. Deal with it.
- Write your code with testing and debugging in mind.
- Keep It Simple, Stupid (KISS).

Brian Kernighan

“Everyone knows that debugging is twice as hard as writing a program in the first place. So if you’re as

- What is the simplest thing that could possibly work?
- Don't Repeat Yourself (DRY).
 - Every piece of knowledge must have a single, unambiguous, authoritative representation within a system.
 - Constants, algorithms, etc...
- Try to limit interdependencies of your code. (Loose Coupling)
- Give your variables, functions and modules meaningful names (not mathematics names)

clever as you can be when you write it, how will you ever debug it?"

pyflakes: fast static analysis

There are several static analysis tools in Python; to name a few:

- [pylint](#)
- [pychecker](#)
- [pyflakes](#)
- [flake8](#)

Here we focus on `pyflakes`, which is the simplest tool.

- **Fast, simple**
- Detects syntax errors, missing imports, typos on names.

Another good recommendation is the `flake8` tool which is a combination of pyflakes and pep8. Thus, in addition to the types of errors that pyflakes catches, flake8 detects violations of the recommendation in [PEP8](#) style guide.

Integrating pyflakes (or flake8) in your editor or IDE is highly recommended, it **does yield productivity gains**.

Running pyflakes on the current edited file

You can bind a key to run pyflakes in the current buffer.

- **In kate** Menu: 'settings -> configure kate'
 - In plugins enable 'external tools'
 - In external Tools', add `pyflakes`:

```
kdiallog --title "pyflakes %filename" --msgbox "$(pyflakes %filename)"
```

- **In TextMate**

Menu: TextMate -> Preferences -> Advanced -> Shell variables, add a shell variable:

```
TM_PYCHECKER =  
/Library/Frameworks/Python.framework/Versions/Current/bin/pyflakes
```

Then `Ctrl-Shift-V` is binded to a pyflakes report

- **In vim** In your `.vimrc` (binds F5 to `pyflakes`):

```
autocmd FileType python let &mp = 'echo "*** running % ***" ; pyflakes %'  
autocmd FileType tex,mp,rst,python imap <Esc>[15~ <C-O>:make!^M  
autocmd FileType tex,mp,rst,python map <Esc>[15~ :make!^M  
autocmd FileType tex,mp,rst,python set autowrite
```

- **In emacs** In your `.emacs` (binds F5 to `pyflakes`):

```
(defun pyflakes-thisfile () (interactive)  
  (compile (format "pyflakes %s" (buffer-file-name))))  
  
(define-minor-mode pyflakes-mode  
  "Toggle pyflakes mode.  
  With no argument, this command toggles the mode.  
  Non-null prefix argument turns on the mode.  
  Null prefix argument turns off the mode."  
  ;; The initial value.  
  nil  
  ;; The indicator for the mode line.  
  " Pyflakes"  
  ;; The minor mode bindings.  
  '( ([f5] . pyflakes-thisfile) )  
)  
  
(add-hook 'python-mode-hook (lambda () (pyflakes-mode t)))
```

A type-as-go spell-checker like integration

- **In vim**

- Use the pyflakes.vim plugin:

1. download the zip file from https://www.vim.org/scripts/script.php?script_id=2441
2. extract the files in `~/ .vim/ftplugin/python`

3. make sure your vimrc has `filetype plugin indent on`

```
869
870     def _compute_log_likelihood(obs):
871         return self._log_emissionprob[:, obs].T
872
```

- Alternatively: use the [syntastic](#) plugin. This can be configured to use `flake8` too and also handles on-the-fly checking for many other languages.

```
17 ~
18 if __name__ == '__main__':
19     data = load_data('exercises/data.txt')
20     print('min: %f' % min(data)) # 10.20
21     print('max: %f' % max(data)) # 61.30
~
~
N debug_file.py
E261 at least two spaces before inline comment
```

- In emacs

Use the flymake mode with pyflakes, documented on

<https://www.emacswiki.org/emacs/FlyMake> and included in Emacs 26 and more recent. To activate it, use `M-x` (meta-key then x) and enter `flymake-mode` at the prompt. To enable it automatically when opening a Python file, add the following line to your .emacs file:

```
(add-hook 'python-mode-hook '(lambda () (flymake-mode)))
```

Debugging workflow

If you do have a non trivial bug, this is when debugging strategies kick in. There is no silver bullet. Yet, strategies help.

For debugging a given problem, the favorable situation is when the problem is isolated in a small number of lines of code, outside framework or application code, with short modify-run-fail cycles.

1. Make it fail reliably. Find a test case that makes the code fail every time.
2. Divide and Conquer. Once you have a failing test case, isolate the failing code.
 - Which module.
 - Which function.
 - Which line of code.

=> isolate a small reproducible failure: a test case

3. Change one thing at a time and re-run the failing test case.
4. Use the debugger to understand what is going wrong.
5. Take notes and be patient. It may take a while.

Note

Once you have gone through this process: isolated a tight piece of code reproducing the bug and fix the bug using this piece of code, add the corresponding code to your test suite.

Using the Python debugger

The python debugger, `pdb`: <https://docs.python.org/3/library/pdb.html>, allows you to inspect your code interactively.

Specifically it allows you to:

- View the source code.
- Walk up and down the call stack.
- Inspect values of variables.
- Modify values of variables.
- Set breakpoints.

print

Yes, `print` statements do work as a debugging tool. However to inspect runtime, it is often more efficient to use the debugger.

Invoking the debugger

Ways to launch the debugger:

1. Postmortem, launch debugger after module errors.
2. Launch the module with the debugger.
3. Call the debugger inside the module

Postmortem

Situation: You're working in IPython and you get a traceback.

Here we debug the file [index_error.py](#). When running it, an `IndexError` is raised. Type `%debug` and drop into the debugger.

```
In [1]: %run index_error.py
```

```
-----
IndexError                                Traceback (most recent call last)
File ~/src/scientific-python-lectures/advanced/debugging/index_error.py:10
      6     print(lst[len(lst)])
      9 if __name__ == "__main__":
--> 10     index_error()
```

```
File ~/src/scientific-python-lectures/advanced/debugging/index_error.py:6, in
index_error()
      4 def index_error():
      5     lst = list("foobar")
--> 6     print(lst[len(lst)])
```

```
IndexError: list index out of range
```

```
In [2]: %debug
```

```
> /home/jarrodd/src/scientific-python-
lectures/advanced/debugging/index_error.py(6)index_error()
      4 def index_error():
      5     lst = list("foobar")
--> 6     print(lst[len(lst)])
      7
      8
```

```
ipdb> list
1 """Small snippet to raise an IndexError."""
2
3
4 def index_error():
5     lst = list("foobar")
--> 6     print(lst[len(lst)])
      7
      8
      9 if __name__ == "__main__":
10     index_error()
```

```
ipdb> len(lst)
6
ipdb> print(lst[len(lst) - 1])
r
ipdb> quit
```

Post-mortem debugging without IPython

In some situations you cannot use IPython, for instance to debug a script that wants to be called from the command line. In this case, you can call the script with `python -m pdb script.py`:

```
$ python -m pdb index_error.py
> /home/jarrodd/src/scientific-python-lectures/advanced/debugging/index_error.py(1)<module>()
-> """Small snippet to raise an IndexError."""
(Pdb) continue
Traceback (most recent call last):
  File "/usr/lib64/python3.11/pdb.py", line 1793, in main
    pdb._run(target)
  File "/usr/lib64/python3.11/pdb.py", line 1659, in _run
    self.run(target.code)
  File "/usr/lib64/python3.11/bdb.py", line 600, in run
    exec(cmd, globals, locals)
  File "<string>", line 1, in <module>
  File "/home/jarrodd/src/scientific-python-lectures/advanced/debugging/index_error.py", line 10, in <module>
    index_error()
  File "/home/jarrodd/src/scientific-python-lectures/advanced/debugging/index_error.py", line 6, in index_error
    print(lst[len(lst)])
    ~~~^^^^^^^^^^^^^^
IndexError: list index out of range
Uncaught exception. Entering post mortem debugging
Running 'cont' or 'step' will restart the program
> /home/jarrodd/src/scientific-python-lectures/advanced/debugging/index_error.py(6)index_error()
-> print(lst[len(lst)])
(Pdb)
```

Step-by-step execution

Situation: You believe a bug exists in a module but are not sure where.

For instance we are trying to debug [!\[\]\(17f4ce09963cb0f17fafa50c06bebe9e_img.jpg\) `wiener\_filtering.py`](#). Indeed the code runs, but the filtering does not work well.

- Run the script in IPython with the debugger using `%run -d wiener_filtering.py`:

```

In [1]: %run -d wiener_filtering.py
*** Blank or comment
*** Blank or comment
*** Blank or comment
NOTE: Enter 'c' at the ipdb> prompt to continue execution.
> /home/jarrodd/src/scientific-python-lectures/advanced/debugging/wiener_filtering.py(1)<module>()
----> 1 """Wiener filtering a noisy raccoon face: this module is buggy"""
      2
      3 import numpy as np
      4 import scipy as sp
      5 import matplotlib.pyplot as plt

```

- Set a break point at line 29 using `b 29`:

```

ipdb> n
> /home/jarrodd/src/scientific-python-lectures/advanced/debugging/wiener_filtering.py(3)<module>()
      1 """Wiener filtering a noisy raccoon face: this module is buggy"""
      2
----> 3 import numpy as np
      4 import scipy as sp
      5 import matplotlib.pyplot as plt

ipdb> b 29
Breakpoint 1 at /home/jarrodd/src/scientific-python-lectures/advanced/debugging/wiener_filtering.py:29

```

- Continue execution to next breakpoint with `c(ontinue)`:

```

ipdb> c
> /home/jarrodd/src/scientific-python-lectures/advanced/debugging/wiener_filtering.py(29)iterated_wiener()
      27 Do not use this: this is crappy code to demo bugs!
      28 """
1--> 29 noisy_img = noisy_img
      30 denoised_img = local_mean(noisy_img, size=size)
      31 l_var = local_var(noisy_img, size=size)

```

- Step into code with `n(ext)` and `s(step)`: `next` jumps to the next statement in the current execution context, while `step` will go across execution contexts, i.e. enable exploring inside function calls:

```

ipdb> s
> /home/jarrold/src/scientific-python-
lectures/advanced/debugging/wiener_filtering.py(30)iterated_wiener()
    28      """
1    29      noisy_img = noisy_img
--> 30      denoised_img = local_mean(noisy_img, size=size)
    31      l_var = local_var(noisy_img, size=size)
    32      for i in range(3):

ipdb> n
> /home/jarrold/src/scientific-python-
lectures/advanced/debugging/wiener_filtering.py(31)iterated_wiener()
1    29      noisy_img = noisy_img
    30      denoised_img = local_mean(noisy_img, size=size)
--> 31      l_var = local_var(noisy_img, size=size)
    32      for i in range(3):
    33          res = noisy_img - denoised_img

```

- Step a few lines and explore the local variables:

```

ipdb> n
> /home/jarrold/src/scientific-python-
lectures/advanced/debugging/wiener_filtering.py(32)iterated_wiener()
    30      denoised_img = local_mean(noisy_img, size=size)
    31      l_var = local_var(noisy_img, size=size)
--> 32      for i in range(3):
    33          res = noisy_img - denoised_img
    34          noise = (res**2).sum() / res.size

ipdb> print(l_var)
[[2571 2782 3474 ... 3008 2922 3141]
 [2105  708  475 ...  469  354 2884]
 [1697  420  645 ...  273  236 2517]
 ...
 [2437  345  432 ...  413  387 4188]
 [2598  179  247 ...  367  441 3909]
 [2808 2525 3117 ... 4413 4454 4385]]
ipdb> print(l_var.min())
0

```

Oh dear, nothing but integers, and 0 variation. Here is our bug, we are doing integer arithmetic.

Raising exception on numerical errors

When we run the [wiener_filtering.py](#) file, the following warnings are raised:

```
In [2]: %run wiener_filtering.py
/home/jarrold/src/scientific-python-lectures/advanced/debugging/wiener_filtering.py:35: RuntimeWarning:
divide by zero encountered in divide
  noise_level = 1 - noise / l_var
```

We can turn these warnings in exception, which enables us to do post-mortem debugging on them, and find our problem more quickly:

```
In [3]: np.seterr(all='raise')
Out[3]: {'divide': 'warn', 'over': 'warn', 'under': 'ignore',
'invalid': 'warn'}

In [4]: %run wiener_filtering.py
-----
----
FloatingPointError                                Traceback (most recent call
last)
File ~/src/scientific-python-lectures/advanced/debugging/wiener_filtering.py:52
    49 plt.matshow(face[cut], cmap=plt.cm.gray)
    50 plt.matshow(noisy_face[cut], cmap=plt.cm.gray)
--> 52 denoised_face = iterated_wiener(noisy_face)
    53 plt.matshow(denoised_face[cut], cmap=plt.cm.gray)
    55 plt.show()

File ~/src/scientific-python-lectures/advanced/debugging/wiener_filtering.py:35, in
iterated_wiener(noisy_img, size)
    33 res = noisy_img - denoised_img
    34 noise = (res**2).sum() / res.size
--> 35 noise_level = 1 - noise / l_var
    36 noise_level[noise_level < 0] = 0
    37 denoised_img = np.int64(noise_level * res)

FloatingPointError: divide by zero encountered in divide
```

Other ways of starting a debugger

- **Raising an exception as a poor man break point**

If you find it tedious to note the line number to set a break point, you can simply raise an exception at the point that you want to inspect and use IPython's `%debug`. Note that in this case you cannot step or continue the execution.

- **Debugging test failures using nosetests**

You can run `nosetests --pdb` to drop in post-mortem debugging on exceptions, and `nosetests --pdb-failure` to inspect test failures using the debugger.

In addition, you can use the IPython interface for the debugger in nose by installing the nose plugin [ipdbplugin](#). You can then pass `--ipdb` and `--ipdb-failure` options to nosetests.

- **Calling the debugger explicitly**

Insert the following line where you want to drop in the debugger:

```
import pdb; pdb.set_trace()
```

Warning

When running `nosetests`, the output is captured, and thus it seems that the debugger does not work. Simply run the nosetests with the `-s` flag.

Graphical debuggers and alternatives

- [pudb](#) is a good semi-graphical debugger with a text user interface in the console.
- The [Visual Studio Code](#) integrated development environment includes a debugging mode.
- The [Mu editor](#) is a simple Python editor that includes a debugging mode.

Debugger commands and interaction

<code>l(list)</code>	Lists the code at the current position
<code>u(p)</code>	Walk up the call stack
<code>d(own)</code>	Walk down the call stack
<code>n(ext)</code>	Execute the next line (does not go down in new functions)
<code>s(tep)</code>	Execute the next statement (goes down in new functions)
<code>bt</code>	Print the call stack
<code>a</code>	Print the local variables
<code>!command</code>	Execute the given Python command (by opposition to pdb commands)

Warning

Debugger commands are not Python code

You cannot name the variables the way you want. For instance, if in you cannot override the variables in the current frame with the same name: **use different names than your local variable when typing code in the debugger.**

Getting help when in the debugger

Type `h` or `help` to access the interactive help:

```
ipdb> help
```

```
Documented commands (type help <topic>):
```

```
=====
```

EOF	commands	enable	ll	pp	s	until
a	condition	exceptions	longlist	psource	skip_hidden	up
alias	cont	exit	n	q	skip_predicates	w
args	context	h	next	quit	source	whatis
b	continue	help	p	r	step	where
break	d	ignore	pdef	restart	tbreak	
bt	debug	j	pdoc	return	u	
c	disable	jump	pfile	retval	unalias	
cl	display	l	pinfo	run	undisplay	
clear	down	list	pinfo2	rv	unt	

```
Miscellaneous help topics:
```

```
=====
```

```
exec  pdb
```

```
Undocumented commands:
```

```
=====
```

```
interact
```

Debugging segmentation faults using gdb

If you have a segmentation fault, you cannot debug it with pdb, as it crashes the Python interpreter before it can drop in the debugger. Similarly, if you have a bug in C code embedded in Python, pdb is useless. For this we turn to the gnu debugger, [gdb](#), available on Linux.

Before we start with gdb, let us add a few Python-specific tools to it. For this we add a few macros to our `~/gdbinit`. The optimal choice of macro depends on your Python version and your gdb version. I have added a simplified version in [gdbinit](#), but feel free to read [DebuggingWithGdb](#).

To debug with gdb the Python script [segfault.py](#), we can run the script in gdb as follows

```
$ gdb python
...
(gdb) run segfault.py
Starting program: /usr/bin/python segfault.py
[Thread debugging using libthread_db enabled]

Program received signal SIGSEGV, Segmentation fault.
_strided_byte_copy (dst=0x8537478 "\360\343G", outstrides=4, src=
    0x86c0690 <Address 0x86c0690 out of bounds>, instrides=32, N=3,
    elsize=4)
    at numpy/core/src/multiarray/ctors.c:365
365      _FAST_MOVE(Int32);
(gdb)
```

We get a segfault, and gdb captures it for post-mortem debugging in the C level stack (not the Python call stack). We can debug the C call stack using gdb's commands:

```
(gdb) up
#1  0x004af4f5 in _copy_from_same_shape (dest=<value optimized out>,
    src=<value optimized out>, myfunc=0x496780 <_strided_byte_copy>,
    swap=0)
    at numpy/core/src/multiarray/ctors.c:748
748      myfunc(dit->dataptr, dest->strides[maxaxis],
```

As you can see, right now, we are in the C code of numpy. We would like to know what is the Python code that triggers this segfault, so we go up the stack until we hit the Python execution loop:

```
(gdb) up
#8  0x080ddd23 in call_function (f=
    Frame 0x85371ec, for file /home/varoquau/usr/lib/python2.6/site-
    packages/numpy/core/arrayprint.py, line 156, in _leading_trailing (a=
    <numpy.ndarray at remote 0x85371b0>, _nc=<module at remote 0xb7f93a64>),
    throwflag=0)
    at ../Python/ceval.c:3750
3750  ../Python/ceval.c: No such file or directory.
    in ../Python/ceval.c

(gdb) up
#9  PyEval_EvalFrameEx (f=
    Frame 0x85371ec, for file /home/varoquau/usr/lib/python2.6/site-
    packages/numpy/core/arrayprint.py, line 156, in _leading_trailing (a=
    <numpy.ndarray at remote 0x85371b0>, _nc=<module at remote 0xb7f93a64>),
    throwflag=0)
    at ../Python/ceval.c:2412
2412  in ../Python/ceval.c
(gdb)
```

Once we are in the Python execution loop, we can use our special Python helper function. For instance we can find the corresponding Python code:

```
(gdb) pyframe
/home/varoquau/usr/lib/python2.6/site-packages/numpy/core/arrayprint.py (158):
_leading_trailing
(gdb)
```

This is numpy code, we need to go up until we find code that we have written:

```
(gdb) up
...
(gdb) up
#34 0x080dc97a in PyEval_EvalFrameEx (f=
    Frame 0x82f064c, for file segfault.py, line 11, in print_big_array
    (small_array=<numpy.ndarray at remote 0x853ecf0>, big_array=<numpy.ndarray at
    remote 0x853ed20>), throwflag=0) at ../Python/ceval.c:1630
1630     ../Python/ceval.c: No such file or directory.
    in ../Python/ceval.c
(gdb) pyframe
segfault.py (12): print_big_array
```

The corresponding code is:

```
def make_big_array(small_array):
    big_array = stride_tricks.as_strided(
        small_array, shape=(int(2e6), int(2e6)), strides=(32, 32)
    )
    return big_array
```

Thus the segfault happens when printing `big_array[-10:]`. The reason is simply that `big_array` has been allocated with its end outside the program memory.

Note

For a list of Python-specific commands defined in the `gdbinit`, read the source of this file.

Exercise 50



The following script is well documented and hopefully legible. It seeks to answer a problem of actual interest for numerical computing, but it does not work... Can you debug it?

Python source code:  [to_debug.py](#)

```

"""
A script to compare different root-finding algorithms.

This version of the script is buggy and does not execute. It is your
task
to find and fix these bugs.

The output of the script should look like:

    Benching 1D root-finder optimizers from scipy.optimize:
                brentth: 604678 total function calls
                brentq: 594454 total function calls
                ridder: 778394 total function calls
                bisect: 2148380 total function calls
"""

from itertools import product

import numpy as np
import scipy as sp

rng = np.random.default_rng(27446968)

FUNCTIONS = (
    np.tan, # Dilating map
    np.tanh, # Contracting map
    lambda x: x**3 + 1e-4 * x, # Almost null gradient at the root
    lambda x: x + np.sin(2 * x), # Non monotonous function
    lambda x: 1.1 * x + np.sin(4 * x), # Function with several local
maxima
)

OPTIMIZERS = (
    sp.optimize.brentth,
    sp.optimize.brentq,
    sp.optimize.ridder,
    sp.optimize.bisect,
)

def apply_optimizer(optimizer, func, a, b):
    """Return the number of function calls given an root-finding
optimizer,
a function and upper and lower bounds.
    """
    return optimizer(func, a, b, full_output=True)[1].function_calls

def bench_optimizer(optimizer, param_grid):
    """Find roots for all the functions, and upper and lower bounds
given and return the total number of function calls.
    """
    return sum(apply_optimizer(optimizer, func, a, b) for func, a, b in
param_grid)

def compare_optimizers(optimizers):

```

```

"""Compare all the optimizers given on a grid of a few different
functions all admitting a single root in zero and a upper and
lower bounds.
"""
random_a = -1.3 + rng.random(size=100)
random_b = 0.3 + rng.random(size=100)
param_grid = list(product(FUNCTIONS, random_a, random_b))
print("Benching 1D root-finder optimizers from scipy.optimize:")
for optimizer in OPTIMIZERS:
    print(
        f"{optimizer.__name__:>20s}: {bench_optimizer(optimizer,
param_grid):>8d} total function calls"
    )

if __name__ == "__main__":
    compare_optimizers(OPTIMIZERS)

```



```

"""
A script to compare different root-finding algorithms.

This version of the script is buggy and does not execute. It is your
task
to find and fix these bugs.

The output of the script should look like:

    Benching 1D root-finder optimizers from scipy.optimize:
                brentth: 604678 total function calls
                brentq: 594454 total function calls
                ridder: 778394 total function calls
                bisect: 2148380 total function calls
"""

from itertools import product

import numpy as np
import scipy as sp

rng = np.random.default_rng(27446968)

FUNCTIONS = (
    np.tan, # Dilating map
    np.tanh, # Contracting map
    lambda x: x**3 + 1e-4 * x, # Almost null gradient at the root
    lambda x: x + np.sin(2 * x), # Non monotonous function
    lambda x: 1.1 * x + np.sin(4 * x), # Function with several local
maxima
)

OPTIMIZERS = (
    sp.optimize.brentth,
    sp.optimize.brentq,
    sp.optimize.ridder,
    sp.optimize.bisect,
)

def apply_optimizer(optimizer, func, a, b):
    """Return the number of function calls given an root-finding
optimizer,
a function and upper and lower bounds.
    """
    return optimizer(func, a, b, full_output=True)[1].function_calls

def bench_optimizer(optimizer, param_grid):
    """Find roots for all the functions, and upper and lower bounds
given and return the total number of function calls.
    """
    return sum(apply_optimizer(optimizer, func, a, b) for func, a, b in
param_grid)

def compare_optimizers(optimizers):

```

```

"""Compare all the optimizers given on a grid of a few different
functions all admitting a single root in zero and a upper and
lower bounds.
"""
random_a = -1.3 + rng.random(size=100)
random_b = 0.3 + rng.random(size=100)
param_grid = list(product(FUNCTIONS, random_a, random_b))
print("Benching 1D root-finder optimizers from scipy.optimize:")
for optimizer in OPTIMIZERS:
    print(
        f"{optimizer.__name__:>20s}: {bench_optimizer(optimizer,
param_grid):>8d} total function calls"
    )

if __name__ == "__main__":
    compare_optimizers(OPTIMIZERS)

```

Optimizing code

Contents

- Optimization workflow
- Profiling Python code
- Making code go faster
- Writing faster numerical code
- Additional Links

Author: *Gaël Varoquaux*

This chapter deals with strategies to make Python code go faster.

Donald Knuth

“Premature optimization is the root of all evil”

Prerequisites

- [line_profiler](#)

Optimization workflow

1. Make it work: write the code in a simple **legible** ways.
2. Make it work reliably: write automated test cases, make really sure that your algorithm is right and that if you break it, the tests will capture the breakage.
3. Optimize the code by profiling simple use-cases to find the bottlenecks and speeding up these bottleneck, finding a better algorithm or implementation. Keep in mind that a trade off should be found between profiling on a realistic example and the simplicity and speed of execution of the code. For efficient work, it is best to work with profiling runs lasting around 10s.

Profiling Python code

No optimization without measuring!

- **Measure:** profiling, timing
- You'll have surprises: the fastest code is not always what you think

Timeit

In Jupyter or IPython, use `timeit` (<https://docs.python.org/3/library/timeit.html>) to time elementary operations:

```
import numpy as np  
a = np.arange(1000)  
%timeit a ** 2
```

822 ns $\pm$ 2.13 ns per loop (mean $\pm$ std. dev. of 7 runs, 1,000,000 loops each)

```
%timeit a ** 2.1
```

5.06 μ s $\pm$ 74.5 ns per loop (mean $\pm$ std. dev. of 7 runs, 100,000 loops each)

```
%timeit a * a
```

871 ns $\pm$ 2.94 ns per loop (mean $\pm$ std. dev. of 7 runs, 1,000,000 loops each)

Use this to guide your choice between strategies.

Note

For long running calls, using `%time` instead of `%timeit`; it is less precise but faster.

Profiler

Useful when you have a large program to profile, for example the  [following file](#):

```
# For this example to run, you also need the 'ica.py' file

import numpy as np
import scipy as sp

from ica import fastica

# @profile # uncomment this line to run with line_profiler
def test():
    rng = np.random.default_rng()
    data = rng.random((5000, 100))
    u, _s, _v = sp.linalg.svd(data)
    pca = u[:, :10].T @ data
    results = fastica(pca.T, whiten=False)

if __name__ == "__main__":
    test()
```

Note

This is a combination of two unsupervised learning techniques, principal component analysis ([PCA](#)) and independent component analysis ([ICA](#)). PCA is a technique for dimensionality reduction, i.e. an algorithm to explain the observed variance in your data using less dimensions. ICA is a source separation technique, for example to unmix multiple signals that have been recorded through multiple sensors. Doing a PCA first and then an ICA can be useful if you have more sensors than signals. For more information see: [the FastICA example from scikits-learn](#).

To run it, you also need to download the  [ica module](#). In IPython we can time the script:

```
In [1]: %run -t demo.py
IPython CPU timings (estimated):
  User   :    14.3929 s.
  System:    0.256016 s.
```

and profile it:

```
In [2]: %run -p demo.py
        916 function calls in 14.551 CPU seconds
Ordered by: internal time
ncalls  tottime  percall  cumtime  percall filename:lineno (function)
   1    14.457    14.457    14.479    14.479 decomp.py:849 (svd)
   1     0.054     0.054     0.054     0.054 {method 'random_sample' of
'mtrand.RandomState' objects}
   1     0.017     0.017     0.021     0.021 function_base.py:645
(asarray_chkfinite)
  54     0.011     0.000     0.011     0.000 {numpy.core._dotblas.dot}
   2     0.005     0.002     0.005     0.002 {method 'any' of 'numpy.ndarray'
objects}
   6     0.001     0.000     0.001     0.000 ica.py:195 (gprime)
   6     0.001     0.000     0.001     0.000 ica.py:192 (g)
  14     0.001     0.000     0.001     0.000 {numpy.linalg.lapack_lite.dsyevd}
  19     0.001     0.000     0.001     0.000 twodim_base.py:204 (diag)
   1     0.001     0.001     0.008     0.008 ica.py:69 (_ica_par)
   1     0.001     0.001    14.551    14.551 {execfile}
 107     0.000     0.000     0.001     0.000 defmatrix.py:239
(__array_finalize__)
   7     0.000     0.000     0.004     0.001 ica.py:58 (_sym_decorrelation)
   7     0.000     0.000     0.002     0.000 linalg.py:841 (eigh)
 172     0.000     0.000     0.000     0.000 {isinstance}
   1     0.000     0.000    14.551    14.551 demo.py:1 (<module>)
  29     0.000     0.000     0.000     0.000 numeric.py:180 (asarray)
  35     0.000     0.000     0.000     0.000 defmatrix.py:193 (__new__)
  35     0.000     0.000     0.001     0.000 defmatrix.py:43 (asmatrix)
  21     0.000     0.000     0.001     0.000 defmatrix.py:287 (__mul__)
  41     0.000     0.000     0.000     0.000 {numpy.core.multiarray.zeros}
  28     0.000     0.000     0.000     0.000 {method 'transpose' of
'numpy.ndarray' objects}
   1     0.000     0.000     0.008     0.008 ica.py:97 (fastica)
...
```

Clearly the `svd` (in `decomp.py`) is what takes most of our time, a.k.a. the bottleneck. We have to find a way to make this step go faster, or to avoid this step (algorithmic optimization). Spending time on the rest of the code is useless.

Profiling outside of IPython, running ``cProfile``

Similar profiling can be done outside of IPython, simply calling the built-in [Python profilers](#) `cProfile` and `profile`.

```
$ python -m cProfile -o demo.prof demo.py
```

Using the `-o` switch will output the profiler results to the file `demo.prof` to view with an external tool. This can be useful if you wish to process the profiler output with a visualization tool.

Line-profiler

The profiler tells us which function takes most of the time, but not where it is called.

For this, we use the [line_profiler](#): in the source file, we decorate a few functions that we want to inspect with `@profile` (no need to import it)

```
@profile
def test():
    rng = np.random.default_rng()
    data = rng.random((5000, 100))
    u, s, v = linalg.svd(data)
    pca = u[:, :10] @ data
    results = fastica(pca.T, whiten=False)
```

Then we run the script using the [kernprof](#) command, with switches `-l, --line-by-line` and `-v, --view` to use the line-by-line profiler and view the results in addition to saving them:

```
$ kernprof -l -v demo.py
```

```
Wrote profile results to demo.py.lprof
Timer unit: 1e-06 s
```

```
Total time: 1.27874 s
File: demo.py
Function: test at line 9
```

Line #	Hits	Time	Per Hit	% Time	Line Contents
9					@profile
10					def test():
11	1	69.0	69.0	0.0	rng =
12	1	2453.0	2453.0	0.2	np.random.default_rng()
13	1	1274715.0	1274715.0	99.7	data = rng.random((5000,
14	1	413.0	413.0	0.0	100))
15	1	1094.0	1094.0	0.1	u, s, v =
					sp.linalg.svd(data)
					pca = u[:, :10].T @ data
					results = fastica(pca.T,
					whiten=False)

The SVD is taking all the time. We need to optimise this line.

Making code go faster

Once we have identified the bottlenecks, we need to make the corresponding code go faster.

Algorithmic optimization

The first thing to look for is algorithmic optimization: are there ways to compute less, or better?

For a high-level view of the problem, a good understanding of the maths behind the algorithm helps. However, it is not uncommon to find simple changes, like **moving computation or memory allocation outside a for loop**, that bring in big gains.

Example of the SVD

In both examples above, the SVD - [Singular Value Decomposition](#) - is what takes most of the time. Indeed, the computational cost of this algorithm is roughly n^3 in the size of the input matrix.

However, in both of these example, we are not using all the output of the SVD, but only the first few rows of its first return argument. If we use the `svd` implementation of SciPy, we can ask for an incomplete version of the SVD. Note that implementations of linear algebra in SciPy are richer than those in NumPy and should be preferred.

```
In [3]: %timeit np.linalg.svd(data)
1 loops, best of 3: 14.5 s per loop

In [4]: import scipy as sp

In [5]: %timeit sp.linalg.svd(data)
1 loops, best of 3: 14.2 s per loop

In [6]: %timeit sp.linalg.svd(data, full_matrices=False)
1 loops, best of 3: 295 ms per loop

In [7]: %timeit np.linalg.svd(data, full_matrices=False)
1 loops, best of 3: 293 ms per loop
```

We can then use this insight to [optimize the previous code](#):

```
def test():
    rng = np.random.default_rng()
    data = rng.random((5000, 100))
    u, _s, _v = sp.linalg.svd(data, full_matrices=False)
    pca = u[:, :10].T @ data
    results = fastica(pca.T, whiten=False)
```

```

In [1]: import demo

In [2]: %timeit demo.
demo.fastica    demo.np          demo.prof.pdf  demo.py          demo.pyc
demo.linalg     demo.prof        demo.prof.png  demo.py.lprof    demo.test

In [2]: %timeit demo.test()
ica.py:65: RuntimeWarning: invalid value encountered in sqrt
  W = (u * np.diag(1.0/np.sqrt(s)) * u.T) * W # W = (W * W.T) ^{-1/2} * W
1 loops, best of 3: 17.5 s per loop

In [3]: import demo_opt

In [4]: %timeit demo_opt.test()
1 loops, best of 3: 208 ms per loop

```

Real incomplete SVDs, e.g. computing only the first 10 eigenvectors, can be computed with ARPACK, available in `scipy.sparse.linalg.eigsh`.

Computational linear algebra

For certain algorithms, many of the bottlenecks will be linear algebra computations. In this case, using the right function to solve the right problem is key. For instance, an eigenvalue problem with a symmetric matrix is easier to solve than with a general matrix. Also, most often, you can avoid inverting a matrix and use a less costly (and more numerically stable) operation.

Know your computational linear algebra. When in doubt, explore `scipy.linalg`, and use `%timeit` to try out different alternatives on your data.

Writing faster numerical code

A complete discussion on advanced use of NumPy is found in chapter [Advanced NumPy](#), or in the article [The NumPy array: a structure for efficient numerical computation](#). by van der Walt *et al*. Here we discuss only some commonly encountered tricks to make code faster.

Vectorizing for loops

Find tricks to avoid for loops using NumPy arrays. For this, masks and indices arrays can be useful.

Broadcasting

Use [broadcasting](#) to do operations on arrays as small as possible before combining them.

In place operations

```
a = np.zeros(10_000_000)
%timeit global a ; a = 0*a
```

22.4 ms $\pm$ 166 μ s per loop (mean $\pm$ std. dev. of 7 runs, 10 loops each)

```
%timeit global a ; a *= 0
```

5.79 ms $\pm$ 116 μ s per loop (mean $\pm$ std. dev. of 7 runs, 100 loops each)

note: we need `global a` in the `timeit` so that it works as expected, as otherwise it is assigning to `a`, and thus considers it as a local variable.

Be easy on the memory: use views, and not copies

Copying big arrays is as costly as making simple numerical operations on them:

```
a = np.zeros(10_000_000)
%timeit a.copy()
```

23.2 ms $\pm$ 226 μ s per loop (mean $\pm$ std. dev. of 7 runs, 10 loops each)

```
%timeit a + 1
```

18.6 ms $\pm$ 372 μ s per loop (mean $\pm$ std. dev. of 7 runs, 100 loops each)

Beware of cache effects

Memory access is cheaper when it is grouped: accessing a big array in a continuous way is much faster than random access. This implies amongst other things that **smaller strides are faster** (see [CPU cache effects](#)):

```
c = np.zeros((5000, 5000), order='C')

# Row elements are far apart in memory, for C ordering.
%timeit np.median(c, axis=0)
```

161 ms $\pm$ 1.16 ms per loop (mean $\pm$ std. dev. of 7 runs, 10 loops each)

```
# Column elements are contiguous in memory, for C ordering.
%timeit np.median(c, axis=1)
```

110 ms $\pm$ 765 μ s per loop (mean $\pm$ std. dev. of 7 runs, 10 loops each)

```
c.strides
```

```
(40000, 8)
```

This is the reason why Fortran ordering or C ordering may make a big difference on speed of operations:

```
rng = np.random.default_rng()
a = rng.random((20, 2**18))
b = rng.random((20, 2**18))
%timeit b @ a.T
```

11.1 ms $\pm$ 131 μ s per loop (mean $\pm$ std. dev. of 7 runs, 100 loops each)

```
c = np.ascontiguousarray(a.T)
%timeit b @ c
```

10.4 ms $\pm$ 71.6 μ s per loop (mean $\pm$ std. dev. of 7 runs, 100 loops each)

Note that copying the data to work around this effect may not be worth it:

```
%timeit c = np.ascontiguousarray(a.T)
```

12.4 ms $\pm$ 50.5 μ s per loop (mean $\pm$ std. dev. of 7 runs, 100 loops each)

Using [numexpr](#) can be useful to automatically optimize code for such effects.

Use compiled code

The last resort, once you are sure that all the high-level optimizations have been explored, is to transfer the hot spots, i.e. the few lines or functions in which most of the time is spent, to compiled code. For compiled code, the preferred option is to use [Cython](#): it is easy to transform existing Python code in compiled code, and with a good use of the [NumPy support](#) yields efficient code on NumPy arrays, for instance by unrolling loops.

Warning

For all the above: profile and time your choices. Don't base your optimization on theoretical considerations.

Additional Links

- If you need to profile memory usage, you could try the [memory profiler](#)
- If you need to profile down into C extensions, you could try using [gperftools](#) from Python with [yep](#).
- If you would like to track performance of your code across time, i.e. as you make new commits to your repository, you could try: [asv](#)
- If you need some interactive visualization why not try [RunSnakeRun](#)

Scipy sparse arrays

Contents

- Why Sparse Matrices?
- Sparse Matrices vs. Sparse Matrix Storage Schemes
- Typical Applications
- Sparsity Structure Visualization

Section author: *Robert Cimrman*

(Dense) matrix is:

- mathematical object
- data structure for storing a 2D array of values

Important features:

- memory allocated once for all items
 - usually a contiguous chunk, think NumPy ndarray
- *fast* access to individual items (*)

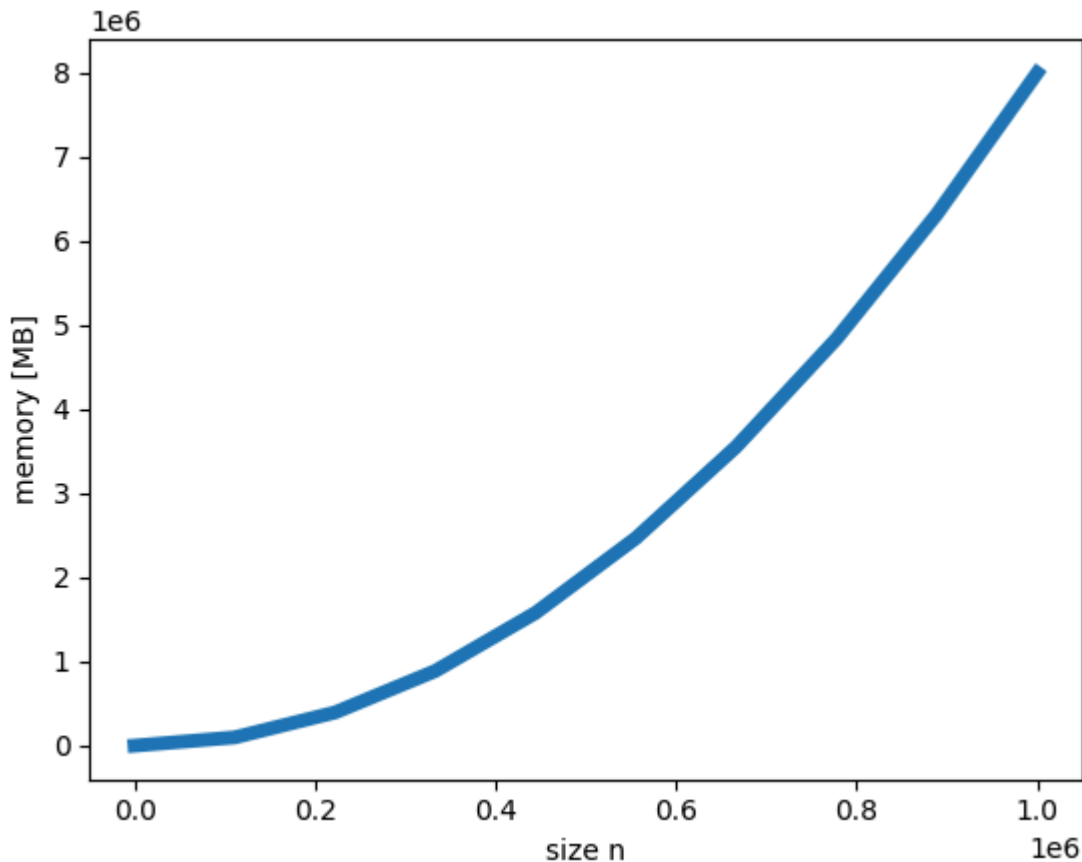
Why Sparse Matrices?

- the memory grows like `n**2` for dense matrix
- small example (double precision matrix):

```
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 1e6, 10)
plt.plot(x, 8.0 * (x**2) / 1e6, lw=5)
plt.xlabel('size n')
plt.ylabel('memory [MB]')
```

```
Text(0, 0.5, 'memory [MB]')
```



Sparse Matrices vs. Sparse Matrix Storage Schemes

- sparse matrix is a matrix, which is *almost empty*
- storing all the zeros is wasteful -> store only nonzero items
- think **compression**
- pros: huge memory savings
- cons: slow access to individual items, but it depends on actual storage scheme.

Typical Applications

- solution of partial differential equations (PDEs)
 - the *finite element method*
 - mechanical engineering, electrotechnics, physics, ...
- graph theory

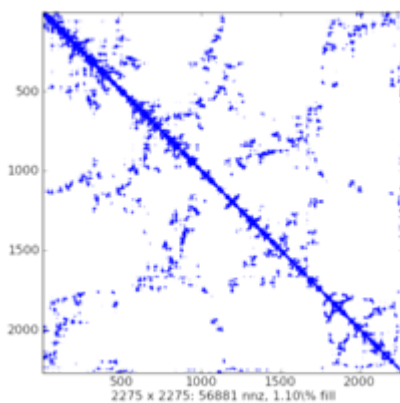
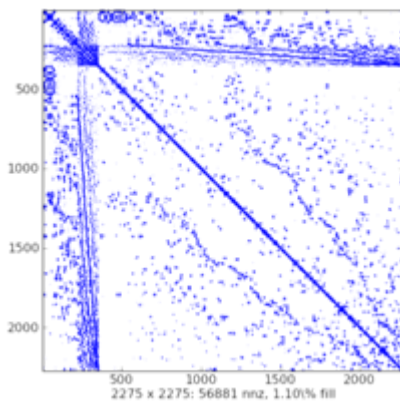
- nonzero at `(i, j)` means that node `i` is connected to node `j`
- natural language processing
 - nonzero at `(i, j)` means that the document `i` contains the word `j`
- ...

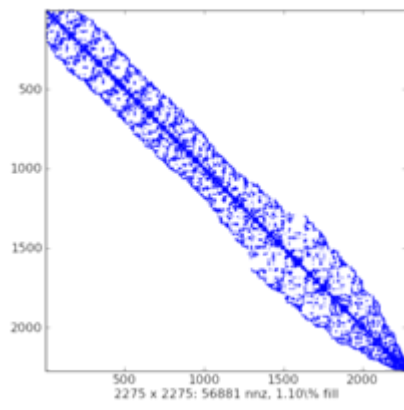
Prerequisites

- [numpy](#)
- [scipy](#)
- [matplotlib \(optional\)](#)
- [ipython \(the enhancements come handy\)](#)

Sparsity Structure Visualization

- `spy()` from `matplotlib`
- example plots:





Storage Schemes

Contents

- Sparse Array Classes
- Common Methods
- Summary

Sparse Array Classes

- There are seven sparse array types in `scipy.sparse`:
 1. [`csc\_array`](#): Compressed Sparse Row format
 2. [`csc\_array`](#): Compressed Sparse Column format
 3. [`bsr\_array`](#): Block Sparse Row format
 4. [`lil\_array`](#): List of Lists format
 5. [`dok\_array`](#): Dictionary of Keys format
 6. [`coo\_array`](#): COOrdinate format (aka IJV, triplet format)
 7. [`dia\_array`](#): DIAGONal format
- each suitable for some tasks
- many employ `sparsetools` C++ module by Nathan Bell
- assume the following is imported:

```
import numpy as np
import scipy as sp
import matplotlib.pyplot as plt
```

- **warning** for Numpy users:
 - passing a sparse array object to NumPy functions that expect `ndarray/matrix` does not work. Use sparse functions.
 - the older `csc_matrix` classes use `*` for matrix multiplication (dot product) and `'A.multiply(B)'` for elementwise multiplication.
 - the newer `csc_array` uses `'@'` for dot product and `*` for elementwise multiplication

- sparse arrays can be 1D or 2D, but not nD for $n > 2$ (unlike NumPy arrays).

Common Methods

- all `scipy.sparse` array classes are subclasses of `sparray`
 - default implementation of arithmetic operations
 - always converts to CSR
 - subclasses override for efficiency
 - shape, data type, set/get
 - indices of nonzero values in the array
 - format conversion, interaction with NumPy (`toarray()`)
 - ...
- attributes:
 - `mtx.T` - transpose (same as `mtx.transpose()`)
 - `mtx.real` - real part of complex matrix
 - `mtx.imag` - imaginary part of complex matrix
 - `mtx.size` - the number of nonzeros (same as `self.getnnz()`)
 - `mtx.shape` - the number of rows and columns (tuple)
- data and indices usually stored in 1D NumPy arrays

Summary

for mat	matrix * vector	get item	fancy get	set item	fancy set	solver s	note
CSR	sparseto ols	yes	yes	slow	.	any	has data array, fast row-wise ops
CSC	sparseto ols	yes	yes	slow	.	any	has data array, fast column-wise ops
BSR	sparseto ols	.	.	.	.	specia lized	has data array, specialized
CO O	sparseto ols	.	.	.	.	iterativ e	has data array, facilitates fast conversion
DIA	sparseto ols	.	.	.	.	iterativ e	has data array, specialized
LIL	via CSR	yes	yes	yes	yes	iterativ e	arithmetic via CSR, incremental construction
DO K	Python	yes	one axis only	yes	yes	iterativ e	O(1) item access, incremental construction, slow arithmetic

Linear System Solvers

Contents

- Sparse Direct Solvers
 - Iterative Solvers
 - Eigenvalue Problem Solvers
-
- sparse matrix/eigenvalue problem solvers live in `scipy.sparse.linalg`
 - the submodules:
 - `dsolve`: direct factorization methods for solving linear systems
 - `isolve`: iterative methods for solving linear systems
 - `eigen`: sparse eigenvalue problem solvers

All solvers are accessible from:

```
import scipy as sp
sp.sparse.linalg.__all__
```

```
['ArpackError',  
 'ArpackNoConvergence',  
 'LaplacianNd',  
 'LinearOperator',  
 'MatrixRankWarning',  
 'SuperLU',  
 'aslinearoperator',  
 'bicg',  
 'bicgstab',  
 'cg',  
 'cgs',  
 'dsolve',  
 'eigen',  
 'eigs',  
 'eigsh',  
 'expm',  
 'expm_multiply',  
 'factorized',  
 'funm_multiply_krylov',  
 'gcrotmk',  
 'gmres',  
 'interface',  
 'inv',  
 'is_sptriangular',  
 'isolve',  
 'lgmres',  
 'lobpcg',  
 'lsmr',  
 'lsqr',  
 'matfuncs',  
 'matrix_power',  
 'minres',  
 'norm',  
 'onenormest',  
 'qmr',  
 'spbandwidth',  
 'spilu',  
 'splu',  
 'spsolve',  
 'spsolve_triangular',  
 'svds',  
 'tfqmr',  
 'use_solver']
```

Sparse Direct Solvers

- default solver: SuperLU
 - included in SciPy
 - real and complex systems
 - both single and double precision

- optional: umfpack
 - real and complex systems
 - double precision only
 - recommended for performance
 - wrappers now live in `scikits.umfpack`
 - check-out the new `scikits.suitesparse` by Nathaniel Smith

Examples

Import the whole module, and see its docstring:

```
help(sp.sparse.linalg.spsolve)
```

Help on function spsolve in module scipy.sparse.linalg._dsolve.linsolve:

spsolve(A, b, permc_spec=None, use_umfpack=True)

Solve the sparse linear system $Ax=b$, where b may be a vector or a matrix.

Parameters

A : ndarray or sparse array or matrix

The square matrix A will be converted into CSC or CSR form

b : ndarray or sparse array or matrix

The matrix or vector representing the right hand side of the equation.

If a vector, b.shape must be (n,) or (n, 1).

permc_spec : str, optional

How to permute the columns of the matrix for sparsity preservation.

(default: 'COLAMD')

- ``NATURAL``: natural ordering.

- ``MMD\_ATA``: minimum degree ordering on the structure of $A^T A$.

- ``MMD\_AT\_PLUS\_A``: minimum degree ordering on the structure of $A^T A + A$.

- ``COLAMD``: approximate minimum degree column ordering [1]_, [2]_.

use_umfpack : bool, optional

if True (default) then use UMFPACK for the solution [3]_, [4]_, [5]_, [6]_. This is only referenced if b is a vector and

``scikits.umfpack`` is installed.

Returns

x : ndarray or sparse array or matrix

the solution of the sparse linear equation.

If b is a vector, then x is a vector of size A.shape[1]

If b is a matrix, then x is a matrix of size (A.shape[1], b.shape[1])

Notes

For solving the matrix expression $AX = B$, this solver assumes the resulting matrix X is sparse, as is often the case for very sparse inputs. If the resulting X is dense, the construction of this sparse result will be relatively expensive. In that case, consider converting A to a dense matrix and using `scipy.linalg.solve` or its variants.

References

.. [1] T. A. Davis, J. R. Gilbert, S. Larimore, E. Ng, Algorithm 836: COLAMD, an approximate column minimum degree ordering algorithm, ACM Trans. on Mathematical Software, 30(3), 2004, pp. 377--380. :doi:`10.1145/1024074.1024080`

.. [2] T. A. Davis, J. R. Gilbert, S. Larimore, E. Ng, A column approximate minimum degree ordering algorithm, ACM Trans. on Mathematical Software, 30(3), 2004, pp. 353--376. :doi:`10.1145/1024074.1024079`

.. [3] T. A. Davis, Algorithm 832: UMFPACK - an unsymmetric-pattern multifrontal method with a column pre-ordering strategy, ACM Trans. on Mathematical Software, 30(2), 2004, pp. 196--199. <https://dl.acm.org/doi/abs/10.1145/992200.992206>

- .. [4] T. A. Davis, A column pre-ordering strategy for the unsymmetric-pattern multifrontal method, ACM Trans. on Mathematical Software, 30(2), 2004, pp. 165--195.
<https://dl.acm.org/doi/abs/10.1145/992200.992205>
- .. [5] T. A. Davis and I. S. Duff, A combined unifrontal/multifrontal method for unsymmetric sparse matrices, ACM Trans. on Mathematical Software, 25(1), 1999, pp. 1--19.
<https://doi.org/10.1145/305658.287640>
- .. [6] T. A. Davis and I. S. Duff, An unsymmetric-pattern multifrontal method for sparse LU factorization, SIAM J. Matrix Analysis and Computations, 18(1), 1997, pp. 140--158.
<https://doi.org/10.1137/S0895479894246905T>.

Examples

```
>>> import numpy as np
>>> from scipy.sparse import csc_array
>>> from scipy.sparse.linalg import spsolve
>>> A = csc_array([[3, 2, 0], [1, -1, 0], [0, 5, 1]], dtype=float)
>>> B = csc_array([[2, 0], [-1, 0], [2, 0]], dtype=float)
>>> x = spsolve(A, B)
>>> np.allclose(A.dot(x).toarray(), B.toarray())
True
```

Both superlu and umfpack can be used (if the latter is installed) as follows.

Prepare a linear system:

```
import numpy as np
mtx = sp.sparse.spdiags([[1, 2, 3, 4, 5], [6, 5, 8, 9, 10]], [0, 1], 5, 5,
                        "csc")
mtx.toarray()
```

```
array([[ 1,  5,  0,  0,  0],
       [ 0,  2,  8,  0,  0],
       [ 0,  0,  3,  9,  0],
       [ 0,  0,  0,  4, 10],
       [ 0,  0,  0,  0,  5]])
```

```
rhs = np.array([1, 2, 3, 4, 5], dtype=np.float32)
```

Solve as single precision real:

```
mtx1 = mtx.astype(np.float32)
x = sp.sparse.linalg.spsolve(mtx1, rhs, use_umfpack=False)
print(x)
```

```
[106.  -21.    5.5  -1.5   1. ]
```

```
print("Error: %s" % (mtx1 * x - rhs))
```

```
Error: [0. 0. 0. 0. 0.]
```

Solve as double precision real:

```
mtx2 = mtx.astype(np.float64)
x = sp.sparse.linalg.spsolve(mtx2, rhs, use_umfpack=True)
print(x)
```

```
[106.  -21.    5.5  -1.5   1. ]
```

```
print("Error: %s" % (mtx2 * x - rhs))
```

```
Error: [0. 0. 0. 0. 0.]
```

Solve as single precision complex:

```
mtx1 = mtx.astype(np.complex64)
x = sp.sparse.linalg.spsolve(mtx1, rhs, use_umfpack=False)
print(x)
```

```
[106. +0.j -21. +0.j   5.5+0.j  -1.5+0.j   1. +0.j]
```

```
print("Error: %s" % (mtx1 * x - rhs))
```

```
Error: [0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j]
```

Solve as double precision complex:

```
mtx2 = mtx.astype(np.complex128)
x = sp.sparse.linalg.spsolve(mtx2, rhs, use_umfpack=True)
print(x)
```

```
[106. +0.j -21. +0.j   5.5+0.j  -1.5+0.j   1. +0.j]
```

```
print("Error: %s" % (mtx2 * x - rhs))
```

```
Error: [0.+0.j 0.+0.j 0.+0.j 0.+0.j 0.+0.j]
```

 [examples/direct_solve.py](#)

Iterative Solvers

- the `isolve` module contains the following solvers:
 - `bicg` (BIConjugate Gradient)
 - `bicgstab` (BIConjugate Gradient STABilized)
 - `cg` (Conjugate Gradient) - symmetric positive definite matrices only
 - `cgs` (Conjugate Gradient Squared)
 - `gmres` (Generalized Minimal RESidual)
 - `minres` (MINimum RESidual)
 - `qmr` (Quasi-Minimal Residual)

Common Parameters

- mandatory:
 - `A` : The N-by-N matrix of the linear system.
 - `b` : Right hand side of the linear system. Has shape (N,) or (N,1).
- optional:
 - `x0` : Starting guess for the solution.
 - `tol` : Relative tolerance to achieve before terminating.
 - `maxiter` : Maximum number of iterations. Iteration will stop after maxiter steps even if the specified tolerance has not been achieved.
 - `M` : Preconditioner for A. The preconditioner should approximate the inverse of A. Effective preconditioning dramatically improves the rate of convergence, which implies that fewer iterations are needed to reach a given error tolerance.
 - `callback` : User-supplied function to call after each iteration. It is called as `callback(xk)`, where `xk` is the current solution vector.

LinearOperator Class

- common interface for performing matrix vector products
- useful abstraction that enables using dense and sparse matrices within the solvers, as well as *matrix-free* solutions
- has `shape` and `matvec()` (+ some optional parameters)

Here is an example:

```
import numpy as np
import scipy as sp

def mv(v):
    return np.array([2 * v[0], 3 * v[1]])
```

```
A = sp.sparse.linalg.LinearOperator((2, 2), matvec=mv)
A
```

```
<2x2 _CustomLinearOperator with dtype=int8>
```

```
A.matvec(np.ones(2))
```

```
array([2., 3.])
```

```
A * np.ones(2)
```

```
array([2., 3.])
```

A Few Notes on Preconditioning

- problem specific
- often hard to develop
- if not sure, try ILU
 - available in `scipy.sparse.linalg` as `spilu()`

Eigenvalue Problem Solvers

The `eigen` module

- `arpack`: a collection of Fortran77 subroutines designed to solve large scale eigenvalue problems
- `lobpcg`: (Locally Optimal Block Preconditioned Conjugate Gradient Method); * works very well in combination with [PyAMG](#)
 - example by Nathan Bell:

↓ [examples/pyamg_with_lobpcg.py](#)

Another example by Nils Wagner:

↓ [examples/lobpcg_sakurai.py](#)

Output:

```
$ python examples/lobpcg_sakurai.py
Results by LOBPCG for n=2500

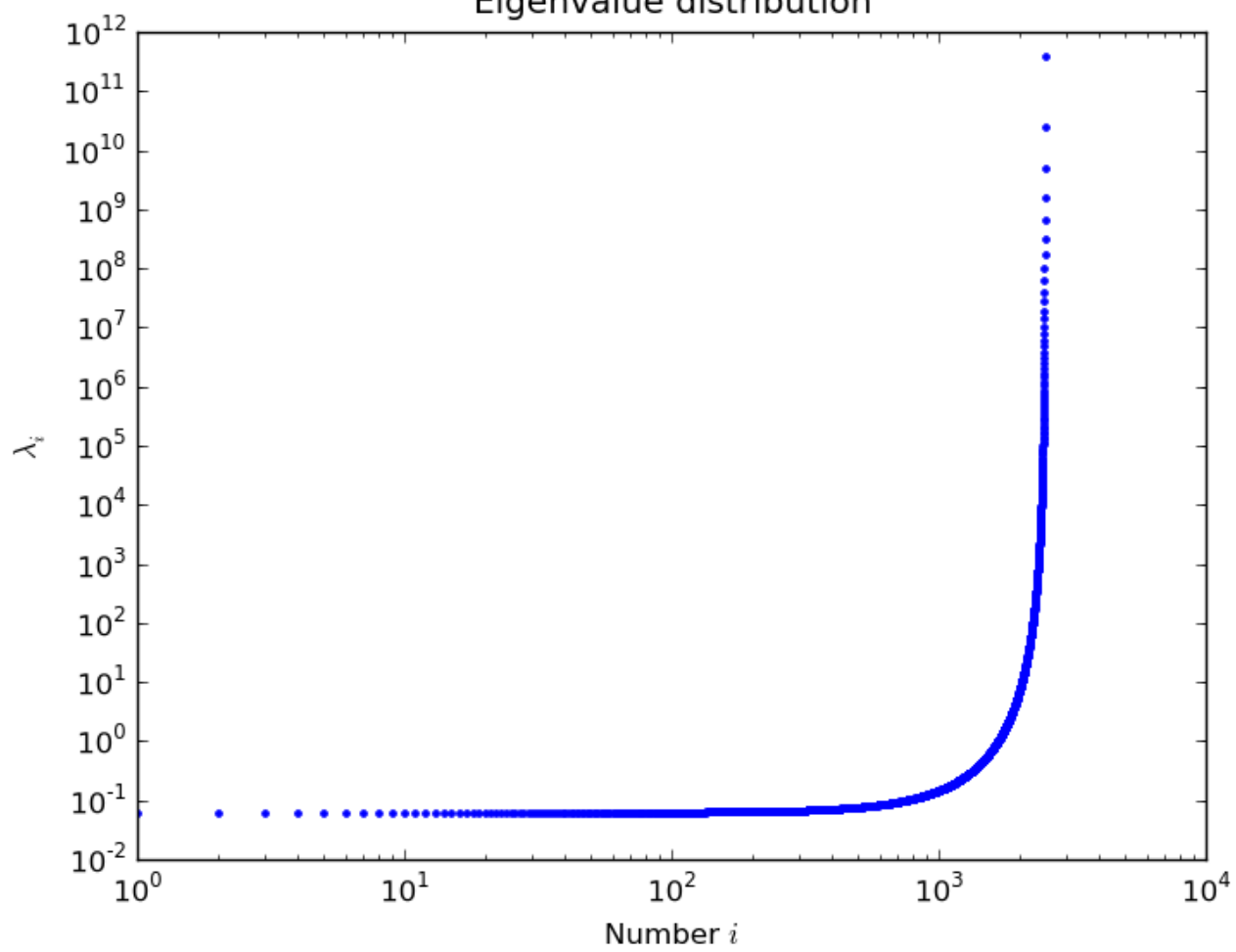
[ 0.06250083  0.06250028  0.06250007]

Exact eigenvalues

[ 0.06250005  0.0625002  0.06250044]

Elapsed time 7.01
```

Eigenvalue distribution



Other Interesting Packages

- PyAMG : - algebraic multigrid solvers
 -  [pyamg/pyamg](https://github.com/pyamg/pyamg)
- Pysparse : - own sparse matrix classes
 - matrix and eigenvalue problem solvers
 - <https://pysparse.sourceforge.net/>

Mathematical optimization: finding minima of functions

Contents

- Knowing your problem
- A review of the different optimizers
- Practical guide to optimization with SciPy
- Special case: non-linear least-squares
- Optimization with constraints

► Show code cell source

Authors: *Gaël Varoquaux*

[Mathematical optimization](#) deals with the problem of finding numerically minimums (or maximums or zeros) of a function. In this context, the function is called *cost function*, or *objective function*, or *energy*.

Here, we are interested in using `scipy.optimize` for black-box optimization: we do not rely on the mathematical expression of the function that we are optimizing. Note that this expression can often be used for more efficient, non black-box, optimization.

Prerequisites

- [NumPy](#)
- [SciPy](#)
- [Matplotlib](#)

See also

References

Mathematical optimization is very ... mathematical. If you want performance, it really pays to read the books:

- [Convex Optimization](#) by Boyd and Vandenberghe (pdf available free online).
- [Numerical Optimization](#) by Nocedal and Wright. Detailed reference on gradient descent methods.
- [Practical Methods of Optimization](#) by Fletcher. Good at hand-waving explanations.

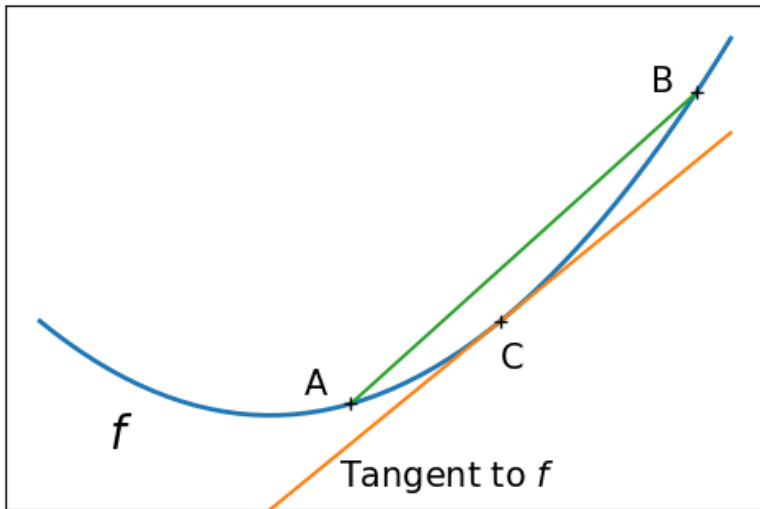
Knowing your problem

Not all optimization problems are equal. Knowing your problem enables you to choose the right tool.

Dimensionality of the problem

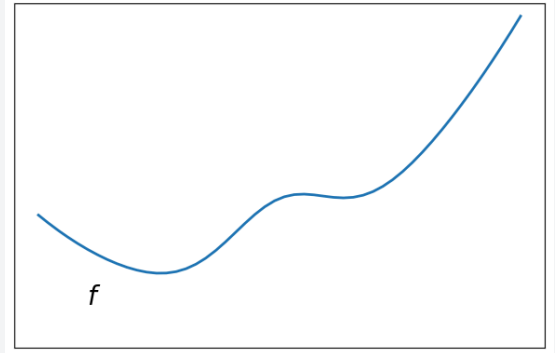
The scale of an optimization problem is pretty much set by the *dimensionality of the problem*, i.e. the number of scalar variables on which the search is performed.

Convex versus non-convex optimization



A convex function:

- f is above all its tangents.
- Equivalently, for two points A, B , $f(C)$ lies below the segment $[f(A), f(B)]$, if $A < C < B$.



A non-convex function

Plot code



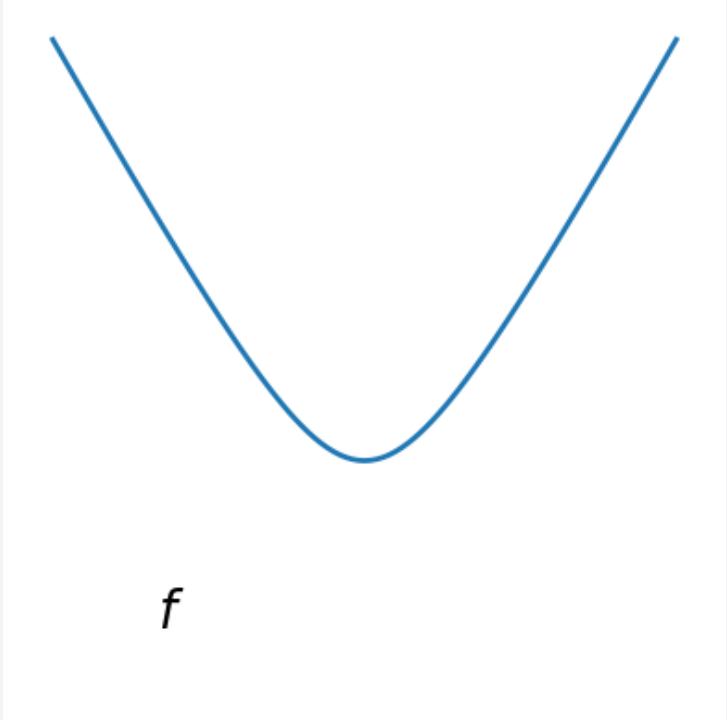
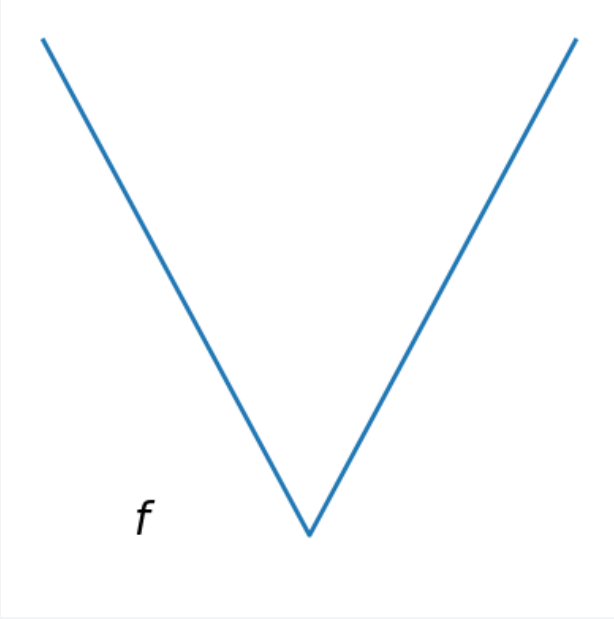
See [convex, non-convex function plots](#).

Optimizing convex functions is easy. Optimizing non-convex functions can be very hard.

Note

It can be proven that for a convex function a local minimum is also a global minimum. Then, in some sense, the minimum is unique.

Smooth and non-smooth problems

 A plot of a smooth function f, represented by a blue parabolic curve opening upwards. The curve is continuous and differentiable everywhere.	 A plot of a non-smooth function f, represented by a blue V-shaped curve opening upwards. The function has a sharp corner at the minimum, where the gradient is not uniquely defined.
A smooth function: The gradient is defined everywhere, and is a continuous function	A non-smooth function

Plot code

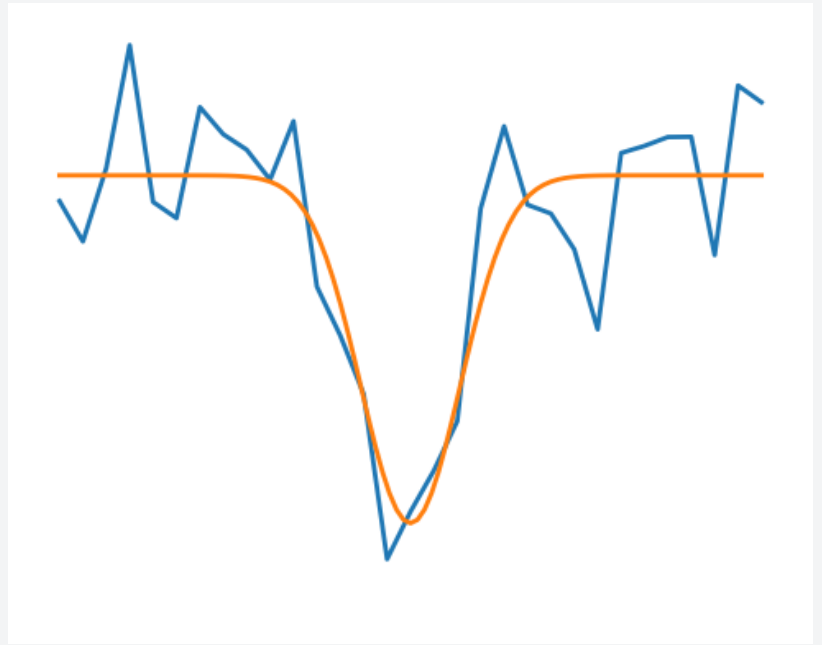


See [smooth, non-smooth function plots](#).

Optimizing smooth functions is easier (true in the context of *black-box* optimization, otherwise [Linear Programming](#) is an example of methods which deal very efficiently with piece-wise linear functions).

Noisy versus exact cost functions

Noisy (blue) and non-noisy (orange) functions



Plot code

See [noisy, non-noisy function plots](#).

Noisy gradients

Many optimization methods rely on gradients of the objective function. If the gradient function is not given, they are computed numerically, which induces errors. In such situation, even if the objective function is not noisy, a gradient-based optimization may be a noisy optimization.

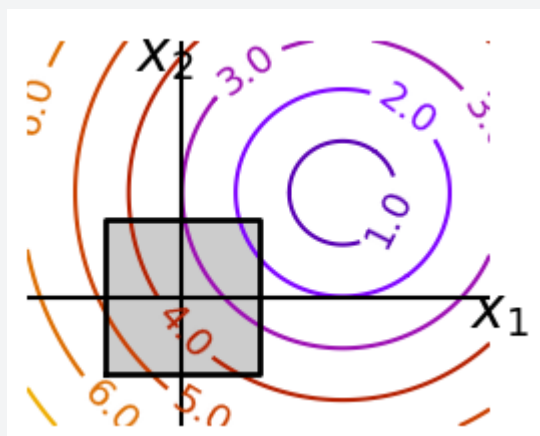
Constraints

Optimizations under constraints

Here:

$$-1 < x_1 < 1$$

$$-1 < x_2 < 1$$



See [constraint plots](#).

A review of the different optimizers

Getting started: 1D optimization

Let's get started by finding the minimum of the scalar function $f(x) = \exp[(x - 0.5)^2]$.

`scipy.optimize.minimize_scalar()` uses Brent's method to find the minimum of a function:

```
def f(x):
    return -np.exp(-(x - 0.5)**2)

result = sp.optimize.minimize_scalar(f)
result.success # check if solver was successful
```

True

```
x_min = result.x
x_min
```

np.float64(0.50000000058670102)

```
x_min - 0.5
```

np.float64(5.867010210991452e-09)

Table 7 **Brent's method on a quadratic function**: it converges in 3 iterations, as the quadratic approximation is then exact.

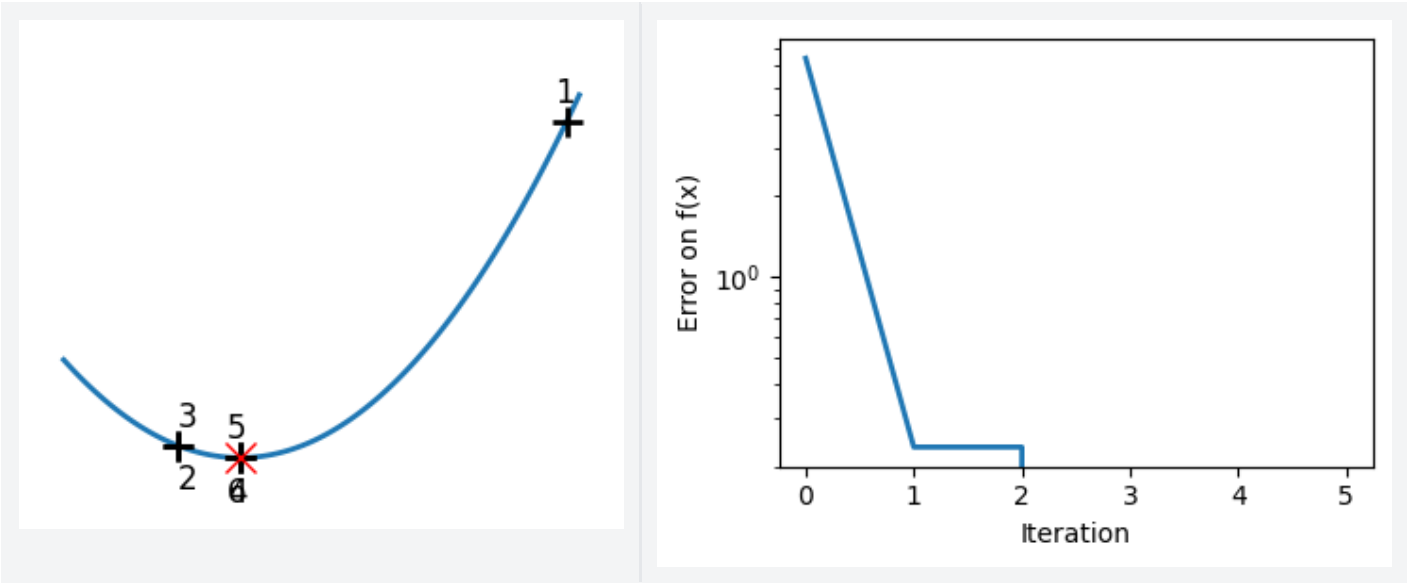
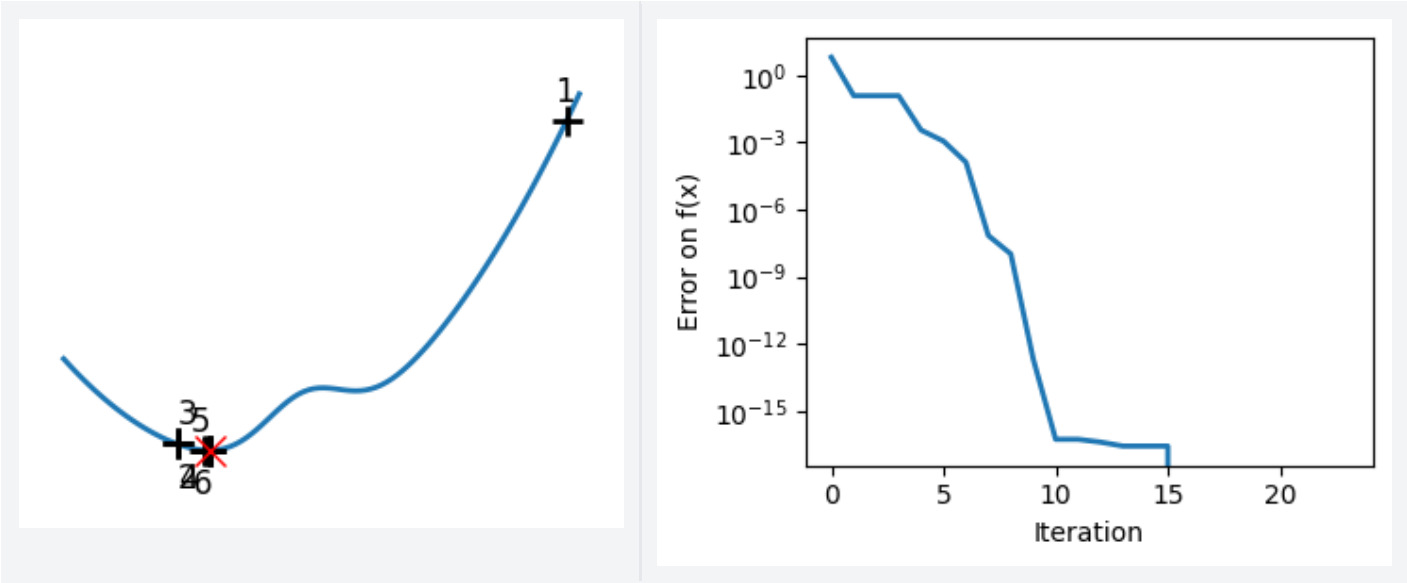


Table 8 **Brent's method on a non-convex function**: note that the fact that the optimizer avoided the local minimum is a matter of luck.



Plot code ^

See [Brent's method figures](#).

Note

You can use different solvers using the parameter `method`.

Note

`scipy.optimize.minimize_scalar()` can also be used for optimization constrained to an interval using the parameter `bounds`.

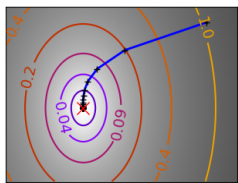
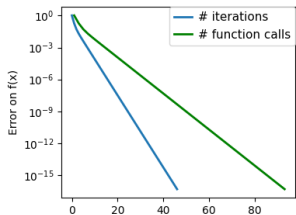
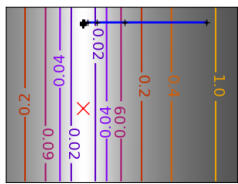
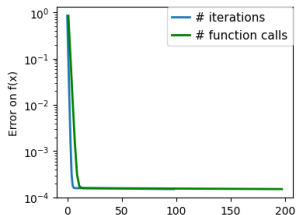
Gradient based methods

Some intuitions about gradient descent

Here we focus on **intuitions**, not code. Code will follow.

[Gradient descent](#) basically consists in taking small steps in the direction of the gradient, that is the direction of the *steepest descent*.

Table 9 Fixed step gradient descent

A well-conditioned quadratic function.		
An ill-conditioned quadratic function. The core problem of gradient-methods on ill-conditioned problems is that the gradient tends not to point in the direction of the minimum.		

Plot code

See [gradient descent plots](#).

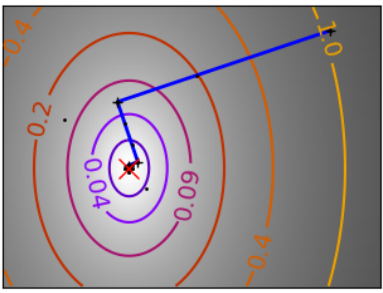
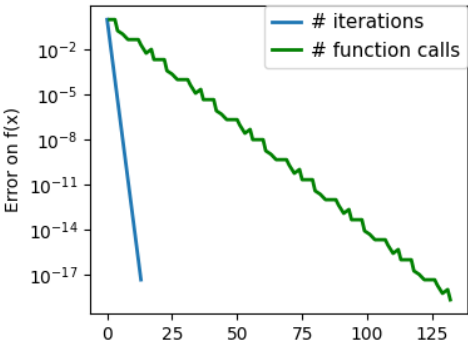
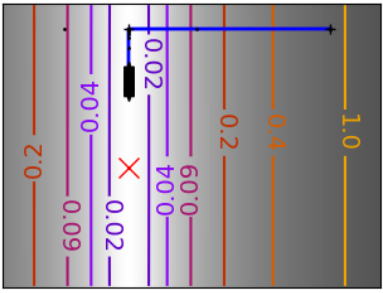
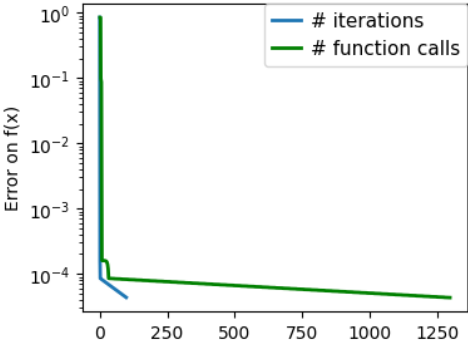
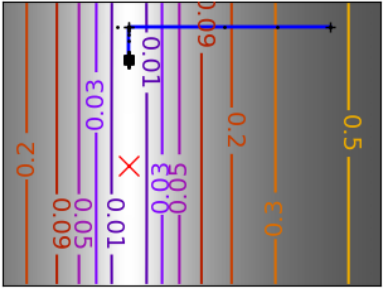
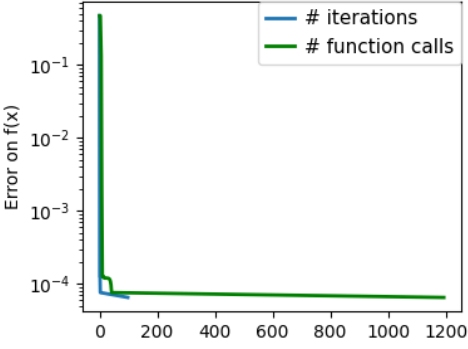
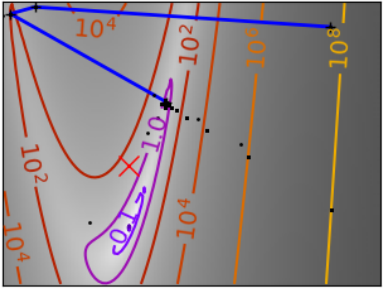
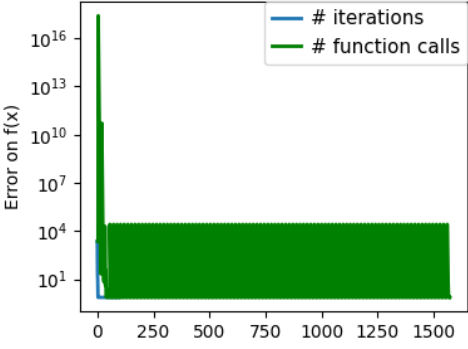
We can see that very anisotropic ([ill-conditioned](#)) functions are harder to optimize.

Take home message: conditioning number and preconditioning

If you know natural scaling for your variables, prescale them so that they behave similarly. This is related to [preconditioning](#).

Also, it clearly can be advantageous to take bigger steps. This is done in gradient descent code using a [line search](#).

Table 10 Adaptive step gradient descent

A well-conditioned quadratic function.		
An ill-conditioned quadratic function.		
An ill-conditioned non-quadratic function.		
An ill-conditioned very non-quadratic function.		

See [gradient descent plots](#).

The more a function looks like a quadratic function (elliptic iso-curves), the easier it is to optimize.

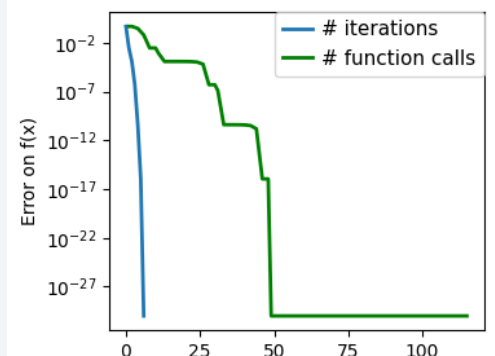
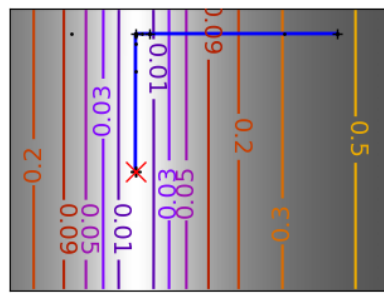
Conjugate gradient descent

The gradient descent algorithms above are toys not to be used on real problems.

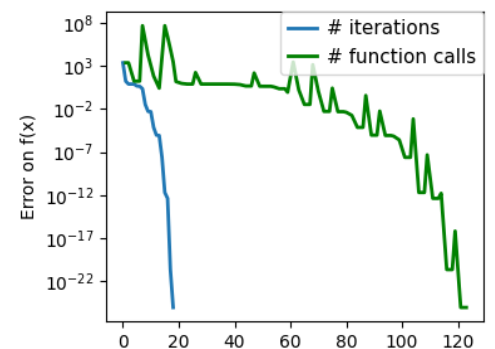
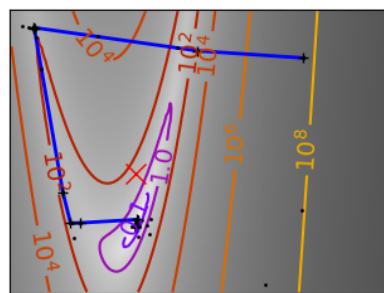
As can be seen from the above experiments, one of the problems of the simple gradient descent algorithms, is that it tends to oscillate across a valley, each time following the direction of the gradient, that makes it cross the valley. The conjugate gradient solves this problem by adding a *friction* term: each step depends on the two last values of the gradient and sharp turns are reduced.

Table 11 Conjugate gradient descent

An ill-conditioned non-quadratic function.



An ill-conditioned very non-quadratic function.



See [gradient descent plots](#).

SciPy provides `scipy.optimize.minimize()` to find the minimum of scalar functions of one or more variables. The simple conjugate gradient method can be used by setting the parameter `method` to CG

```
def f(x): # The rosenbrock function
    return .5*(1 - x[0])**2 + (x[1] - x[0]**2)**2

sp.optimize.minimize(f, [2, -1], method="CG")
```

```
message: Optimization terminated successfully.
success: True
status: 0
  fun: 1.650917861123459e-11
   x: [ 1.000e+00  1.000e+00]
  nit: 13
  jac: [-6.154e-06  2.538e-07]
 nfev: 81
 njev: 27
```

Gradient methods need the Jacobian (gradient) of the function. They can compute it numerically, but will perform better if you can pass them the gradient:

```
def jacobian(x):
    return np.array((-2*.5*(1 - x[0]) - 4*x[0]*(x[1] - x[0]**2), 2*(x[1] -
x[0]**2)))

sp.optimize.minimize(f, [2, 1], method="CG", jac=jacobian)
```

```
message: Optimization terminated successfully.
success: True
status: 0
  fun: 2.95786590125255e-14
   x: [ 1.000e+00  1.000e+00]
  nit: 8
  jac: [ 7.183e-07 -2.990e-07]
 nfev: 16
 njev: 16
```

Note that the function has only been evaluated 27 times, compared to 108 without the gradient.

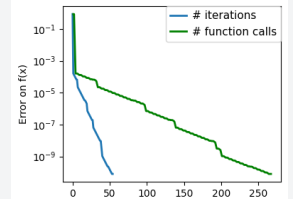
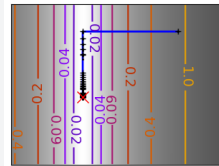
Newton and quasi-newton methods

Newton methods: using the Hessian (2nd differential)

[Newton methods](#) use a local quadratic approximation to compute the jump direction. For this purpose, they rely on the 2 first derivative of the function: the *gradient* and the [Hessian](#).

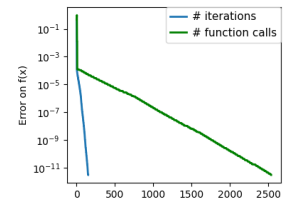
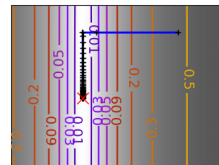
An ill-conditioned quadratic function:

Note that, as the quadratic approximation is exact, the Newton method is blazing fast

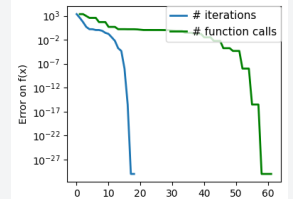
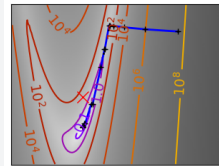


An ill-conditioned non-quadratic function:

Here we are optimizing a Gaussian, which is always below its quadratic approximation. As a result, the Newton method overshoots and leads to oscillations.



An ill-conditioned very non-quadratic function:



Plot code



See [gradient descent plots](#).

In SciPy, you can use the Newton method by setting `method` to Newton-CG in

`scipy.optimize.minimize()`. Here, CG refers to the fact that an internal inversion of the Hessian is performed by conjugate gradient.

```
def f(x):    # The rosenbrock function
    return .5*(1 - x[0])**2 + (x[1] - x[0]**2)**2

def jacobian(x):
    return np.array((-2*.5*(1 - x[0]) - 4*x[0]*(x[1] - x[0]**2), 2*(x[1] - x[0]**2)))

sp.optimize.minimize(f, [2, -1], method="Newton-CG", jac=jacobian)
```

```
message: Optimization terminated successfully.
success: True
status: 0
  fun: 1.5601357400786612e-15
   x: [ 1.000e+00  1.000e+00]
  nit: 10
 jac: [ 1.058e-07 -7.483e-08]
nfev: 11
njev: 33
nhev: 0
```

Note that compared to a conjugate gradient (above), Newton's method has required less function evaluations, but more gradient evaluations, as it uses it to approximate the Hessian. Let's compute the Hessian and pass it to the algorithm:

```
def hessian(x): # Computed with sympy
    return np.array(((1 - 4*x[1] + 12*x[0]**2, -4*x[0]), (-4*x[0], 2)))

sp.optimize.minimize(f, [2, -1], method="Newton-CG", jac=jacobian, hess=hessian)
```

```
message: Optimization terminated successfully.
success: True
status: 0
  fun: 1.6277298383706738e-15
   x: [ 1.000e+00  1.000e+00]
  nit: 10
 jac: [ 1.110e-07 -7.781e-08]
nfev: 11
njev: 11
nhev: 10
```

Note

At very high-dimension, the inversion of the Hessian can be costly and unstable (large scale > 250).

Note

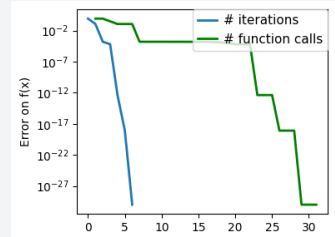
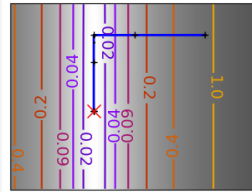
Newton optimizers should not to be confused with Newton's root finding method, based on the same principles, `scipy.optimize.newton\(\)`.

Quasi-Newton methods: approximating the Hessian on the fly

BFGS: BFGS (Broyden-Fletcher-Goldfarb-Shanno algorithm) refines at each step an approximation of the Hessian.

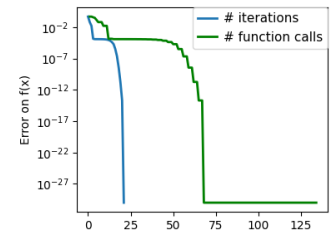
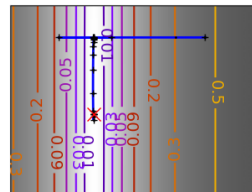
An ill-conditioned quadratic function:

On an exactly quadratic function, BFGS is not as fast as Newton's method, but still very fast.

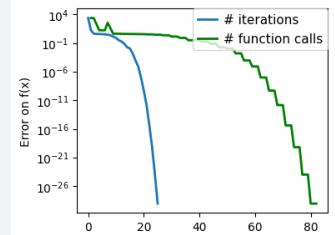
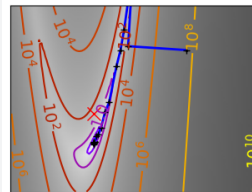


An ill-conditioned non-quadratic function:

Here BFGS does better than Newton, as its empirical estimate of the curvature is better than that given by the Hessian.



An ill-conditioned very non-quadratic function:



Plot code



See [gradient descent plots](#).

```
def f(x):    # The rosenbrock function
    return .5*(1 - x[0])**2 + (x[1] - x[0]**2)**2

def jacobian(x):
    return np.array((-2*.5*(1 - x[0]) - 4*x[0]*(x[1] - x[0]**2), 2*(x[1] - x[0]**2)))

sp.optimize.minimize(f, [2, -1], method="BFGS", jac=jacobian)
```

```
message: Optimization terminated successfully.
success: True
status: 0
  fun: 2.630637192365927e-16
   x: [ 1.000e+00  1.000e+00]
  nit: 8
  jac: [ 6.709e-08 -3.222e-08]
hess_inv: [[ 9.999e-01  2.000e+00]
            [ 2.000e+00  4.499e+00]]
  nfev: 10
  njev: 10
```

L-BFGS: Limited-memory BFGS sits between BFGS and conjugate gradient: in very high dimensions (> 250) the Hessian matrix is too costly to compute and invert. L-BFGS keeps a low-rank version. In addition, box bounds are also supported by L-BFGS-B:

```
def f(x):  # The rosenbrock function
    return .5*(1 - x[0])**2 + (x[1] - x[0]**2)**2

def jacobian(x):
    return np.array((-2*.5*(1 - x[0]) - 4*x[0]*(x[1] - x[0]**2), 2*(x[1] -
x[0]**2)))

sp.optimize.minimize(f, [2, 2], method="L-BFGS-B", jac=jacobian)
```

```
message: CONVERGENCE: NORM OF PROJECTED GRADIENT <= PGTOL
success: True
status: 0
  fun: 1.4417677473011861e-15
   x: [ 1.000e+00  1.000e+00]
  nit: 16
  jac: [ 1.023e-07 -2.593e-08]
  nfev: 17
  njev: 17
hess_inv: <2x2 LbfgsInvHessProduct with dtype=float64>
```

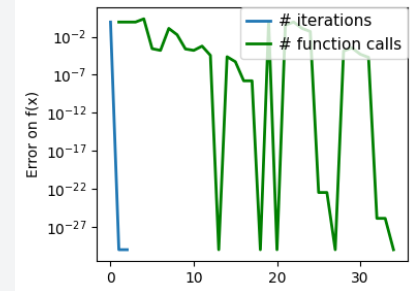
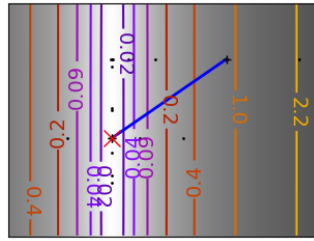
Gradient-less methods

A shooting method: the Powell algorithm

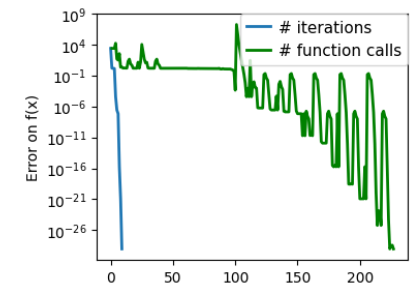
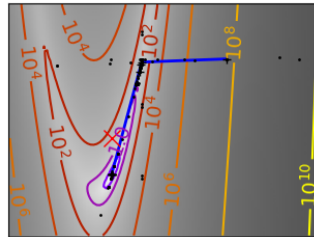
Almost a gradient approach:

An ill-conditioned quadratic function:

Powell's method isn't too sensitive to local ill-conditioning in low dimensions



An ill-conditioned very non-quadratic function:



Plot code



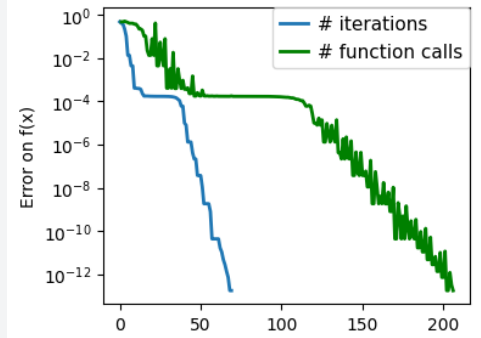
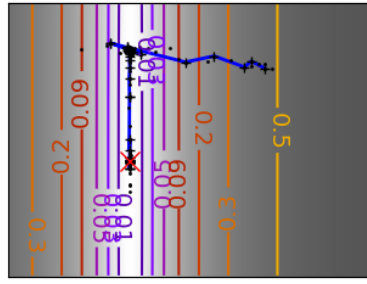
See [gradient descent plots](#).

Simplex method: the Nelder-Mead

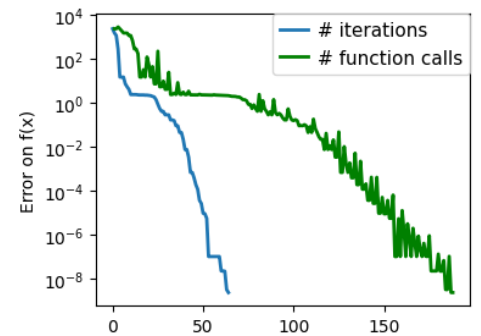
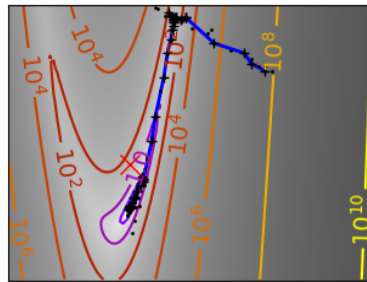
The Nelder-Mead algorithms are a generalization of dichotomy approaches to high-dimensional spaces. The algorithm works by refining a [simplex](#), the generalization of intervals and triangles to high-dimensional spaces, to bracket the minimum.

Strong points: it is robust to noise, as it does not rely on computing gradients. Thus it can work on functions that are not locally smooth such as experimental data points, as long as they display a large-scale bell-shape behavior. However it is slower than gradient-based methods on smooth, non-noisy functions.

An ill-conditioned non-quadratic function:



An ill-conditioned very non-quadratic function:



Plot code

See [gradient descent plots](#).

Using the Nelder-Mead solver in `scipy.optimize.minimize()`:

```
def f(x): # The rosenbrock function
    return .5*(1 - x[0])**2 + (x[1] - x[0]**2)**2

sp.optimize.minimize(f, [2, -1], method="Nelder-Mead")
```

```
message: Optimization terminated successfully.
success: True
status: 0
  fun: 1.11527915993744e-10
   x: [ 1.000e+00  1.000e+00]
  nit: 58
 nfev: 111
final_simplex: (array([[ 1.000e+00,  1.000e+00],
                        [ 1.000e+00,  1.000e+00],
                        [ 1.000e+00,  1.000e+00]]), array([ 1.115e-10,  1.537e-
10,  4.988e-10]))
```

Global optimizers

If your problem does not admit a unique local minimum (which can be hard to test unless the function is convex), and you do not have prior information to initialize the optimization close to the solution, you may need a global optimizer.

Brute force: a grid search

`scipy.optimize.brute()` evaluates the function on a given grid of parameters and returns the parameters corresponding to the minimum value. The parameters are specified with ranges given to `numpy.mgrid`. By default, 20 steps are taken in each direction:

```
def f(x): # The rosenbrock function
    return .5*(1 - x[0])**2 + (x[1] - x[0]**2)**2

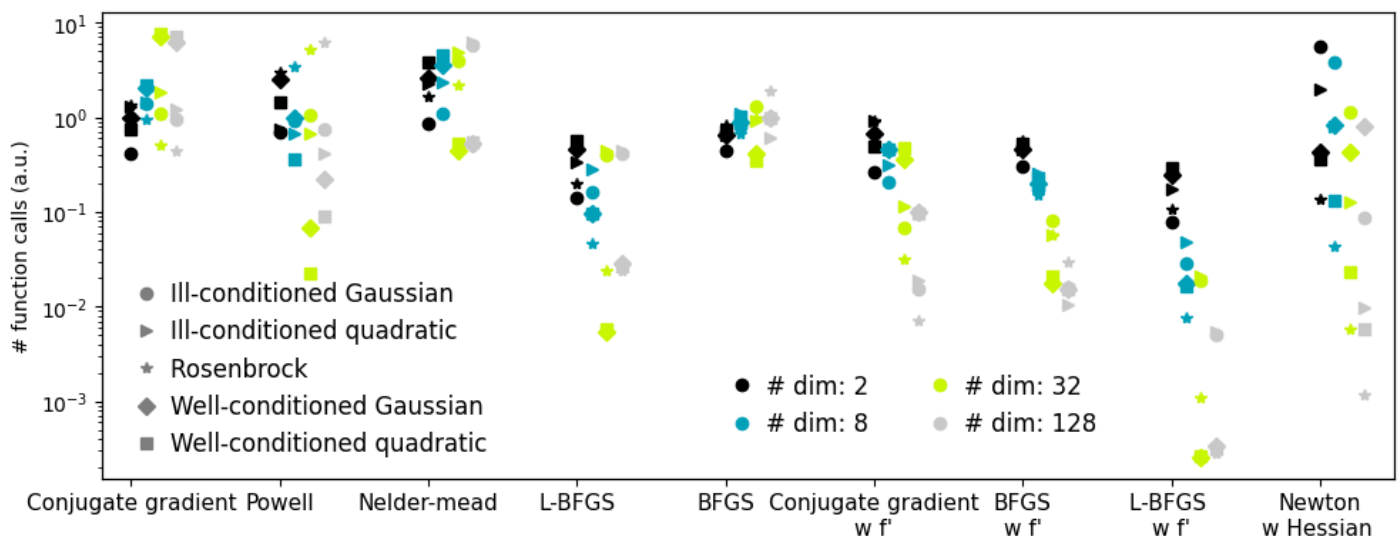
sp.optimize.brute(f, ((-1, 2), (-1, 2)))
```

```
array([1.00001462, 1.00001547])
```

Practical guide to optimization with SciPy

Choosing a method

All methods are exposed as the `method` argument of `scipy.optimize.minimize()`.



See [compare optimizers](#).

Table 12 Rules of thumb for choosing a method

Without knowledge of the gradient	- In general, prefer BFGS or L-BFGS, even if you have to approximate numerically gradients. These are also the default if you omit the parameter <code>method</code> - depending if the problem has constraints or bounds. - On well-conditioned problems, Powell and Nelder-Mead, both gradient-free methods, work well in high dimension, but they collapse for ill-conditioned problems.
With knowledge of the gradient	BFGS or L-BFGS. - Computational overhead of BFGS is larger than that L-BFGS, itself larger than that of conjugate gradient. On the other side, BFGS usually needs less function evaluations than CG. Thus conjugate gradient method is better than BFGS at optimizing computationally cheap functions.
With the Hessian	If you can compute the Hessian, prefer the Newton method (Newton-CG or TCG).
If you have noisy measurements	Use Nelder-Mead or Powell.

Making your optimizer faster

- Choose the right method (see above), do compute analytically the gradient and Hessian, if you can.
- Use [preconditionning](#) when possible.
- Choose your initialization points wisely. For instance, if you are running many similar optimizations, warm-restart one with the results of another.
- Relax the tolerance if you don't need precision using the parameter `tol`.

Computing gradients

Computing gradients, and even more Hessians, is very tedious but worth the effort. Symbolic computation with [SymPy](#) may come in handy.

Warning

A very common source of optimization not converging well is human error in the computation of the gradient. You can use `scipy.optimize.check_grad()` to check that your gradient is correct. It returns the norm of the different between the gradient given, and a gradient computed numerically:

```
sp.optimize.check_grad(f, jacobian, [2, -1])
```

```
np.float64(2.384185791015625e-07)
```

See also `scipy.optimize.approx_fprime()` to find your errors.

Synthetic exercises

A simple (?) quadratic function

Exercise 55



Optimize the following function, using `K[0]` as a starting point:

```
rng = np.random.default_rng(27446968)
K = rng.normal(size=(100, 100))

def f(x):
    return np.sum((K @ (x - 1))**2) + np.sum(x**2)**2
```

Time your approach. Find the fastest approach. Why is BFGS not working well?

Solution to [Exercise 55](#)



Alternating optimization

The challenge here is that Hessian of the problem is a very ill-conditioned matrix. This can easily be seen, as the Hessian of the first term is simply $2 * K.T @ K$. Thus the conditioning of the problem can be judged from looking at the conditioning of K .

```
import time

rng = np.random.default_rng(27446968)

K = rng.normal(size=(100, 100))

def f(x):
    return np.sum((K @ (x - 1)) ** 2) + np.sum(x**2) ** 2

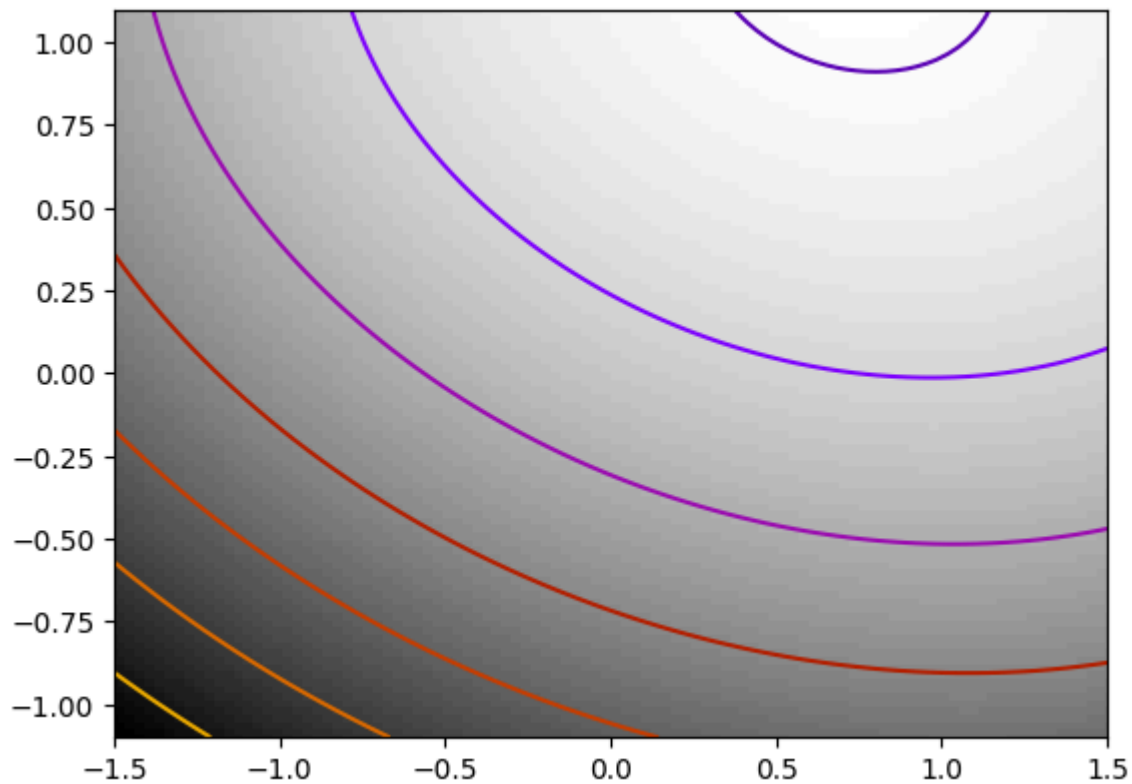
def f_prime(x):
    return 2 * K.T @ K @ (x - 1) + 4 * np.sum(x**2) * x

def hessian(x):
    H = 2 * K.T @ K + 4 * 2 * x * x[:, np.newaxis]
    return H + 4 * np.eye(H.shape[0]) * np.sum(x**2)
```

Some pretty plotting

```
plt.figure()
Z = X, Y = np.mgrid[-1.5:1.5:100j, -1.1:1.1:100j] # type: ignore[misc]
# Complete in the additional dimensions with zeros
Z = np.reshape(Z, (2, -1)).copy()
Z.resize((100, Z.shape[-1]))
Z = np.apply_along_axis(f, 0, Z)
Z = np.reshape(Z, X.shape)
plt.imshow(Z.T, cmap="gray_r", extent=(-1.5, 1.5, -1.1, 1.1),
origin="lower")
plt.contour(X, Y, Z, cmap="gnuplot")
```

```
<matplotlib.contour.QuadContourSet at 0x7f7a09ef1370>
```



A reference but slow solution:

```
t0 = time.time()
x_ref = sp.optimize.minimize(f, K[0], method="Powell").x
print(f"    Powell: time {time.time() - t0:.2f}s")
f_ref = f(x_ref)
```

Powell: time 0.10s

Compare different approaches

```
t0 = time.time()
x_bfgs = sp.optimize.minimize(f, K[0], method="BFGS").x
print(
    f"    BFGS: time {time.time() - t0:.2f}s, x error"
    {np.sqrt(np.sum((x_bfgs - x_ref) ** 2)):.2f}, f error {f(x_bfgs) -"
    f_ref:.2f}"
)

t0 = time.time()
x_l_bfgs = sp.optimize.minimize(f, K[0], method="L-BFGS-B").x
print(
    f"    L-BFGS: time {time.time() - t0:.2f}s, x error"
    {np.sqrt(np.sum((x_l_bfgs - x_ref) ** 2)):.2f}, f error {f(x_l_bfgs) -"
    f_ref:.2f}"
)
```

```
BFGS: time 0.51s, x error 0.02, f error -0.03
L-BFGS: time 0.06s, x error 0.02, f error -0.03
```

```
t0 = time.time()
x_bfgs = sp.optimize.minimize(f, K[0], jac=f_prime, method="BFGS").x
print(
    f" BFGS w f': time {time.time() - t0:.2f}s, x error {np.sqrt(np.sum((x_bfgs - x_ref) ** 2)):.2f}, f error {f(x_bfgs) - f_ref:.2f}"
)

t0 = time.time()
x_l_bfgs = sp.optimize.minimize(f, K[0], jac=f_prime, method="L-BFGS-B").x
print(
    f"L-BFGS w f': time {time.time() - t0:.2f}s, x error {np.sqrt(np.sum((x_l_bfgs - x_ref) ** 2)):.2f}, f error {f(x_l_bfgs) - f_ref:.2f}"
)
```

```
BFGS w f': time 0.06s, x error 0.02, f error -0.03
L-BFGS w f': time 0.00s, x error 0.02, f error -0.03
```

```
t0 = time.time()
x_newton = sp.optimize.minimize(
    f, K[0], jac=f_prime, hess=hessian, method="Newton-CG"
).x
print(
    f" Newton: time {time.time() - t0:.2f}s, x error {np.sqrt(np.sum((x_newton - x_ref) ** 2)):.2f}, f error {f(x_newton) - f_ref:.2f}"
)
```

```
Newton: time 0.01s, x error 0.02, f error -0.03
```

A locally flat minimum

Exercise 56

Consider the function $\exp(-1/(.1x^{**2} + y^{**2}))$. This function admits a minimum in (0, 0). Starting from an initialization at (1, 1), try to get within $1e-8$ of this minimum point.

This exercise is hard because the function is very flat around the minimum (all its derivatives are zero). Thus gradient information is unreliable.

Solution to [Exercise 56](#)

Finding a minimum in a flat neighborhood

The function admits a minimum in $[0, 0]$. The challenge is to get within $1e-7$ of this minimum, starting at $x_0 = [1, 1]$.

The solution that we adopt here is to give up on using gradient or information based on local differences, and to rely on the Powell algorithm. With 162 function evaluations, we get to $1e-8$ of the solution.

```
def f(x):  
    return np.exp(-1 / (0.01 * x[0] ** 2 + x[1] ** 2))
```

A well-conditioned version of f :

```
def g(x):  
    return f([10 * x[0], x[1]])
```

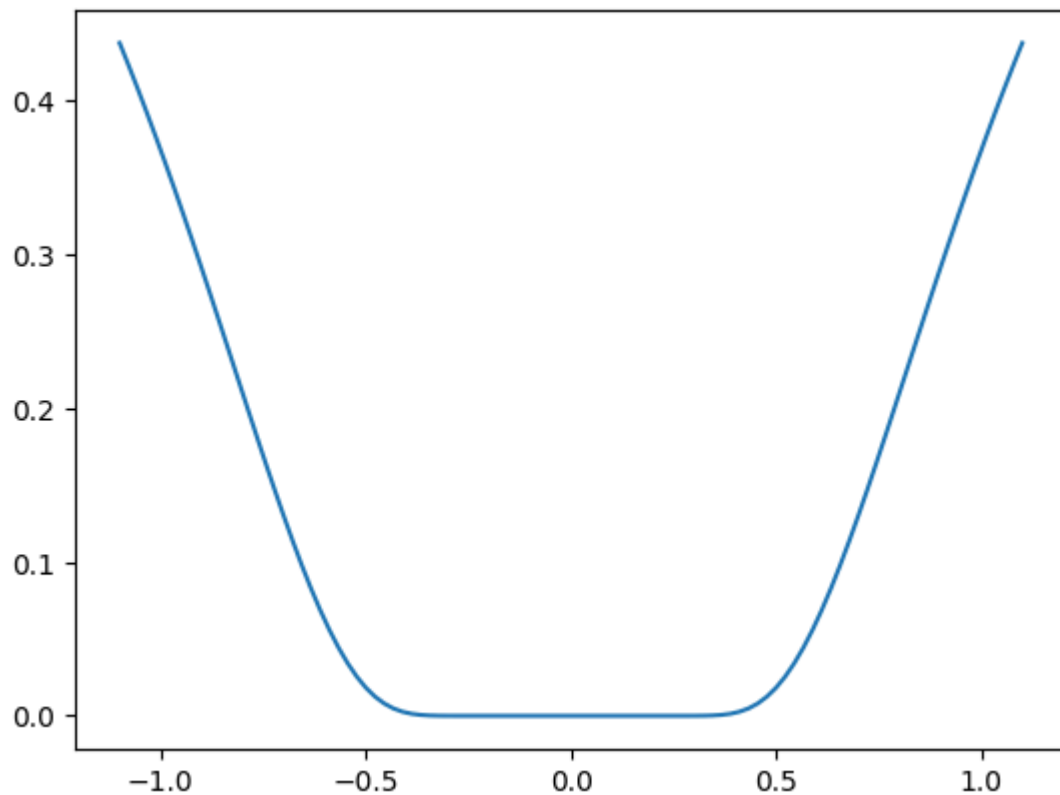
The gradient of g . We won't use it here for the optimization.

```
def g_prime(x):  
    r = np.sqrt(x[0] ** 2 + x[1] ** 2)  
    return 2 / r**3 * g(x) * x / r  
  
result = sp.optimize.minimize(g, [1, 1], method="Powell", tol=1e-10)  
x_min = result.x  
x_min
```

```
array([ 1.00165881e-08, -9.59695391e-09])
```

Some pretty plotting:

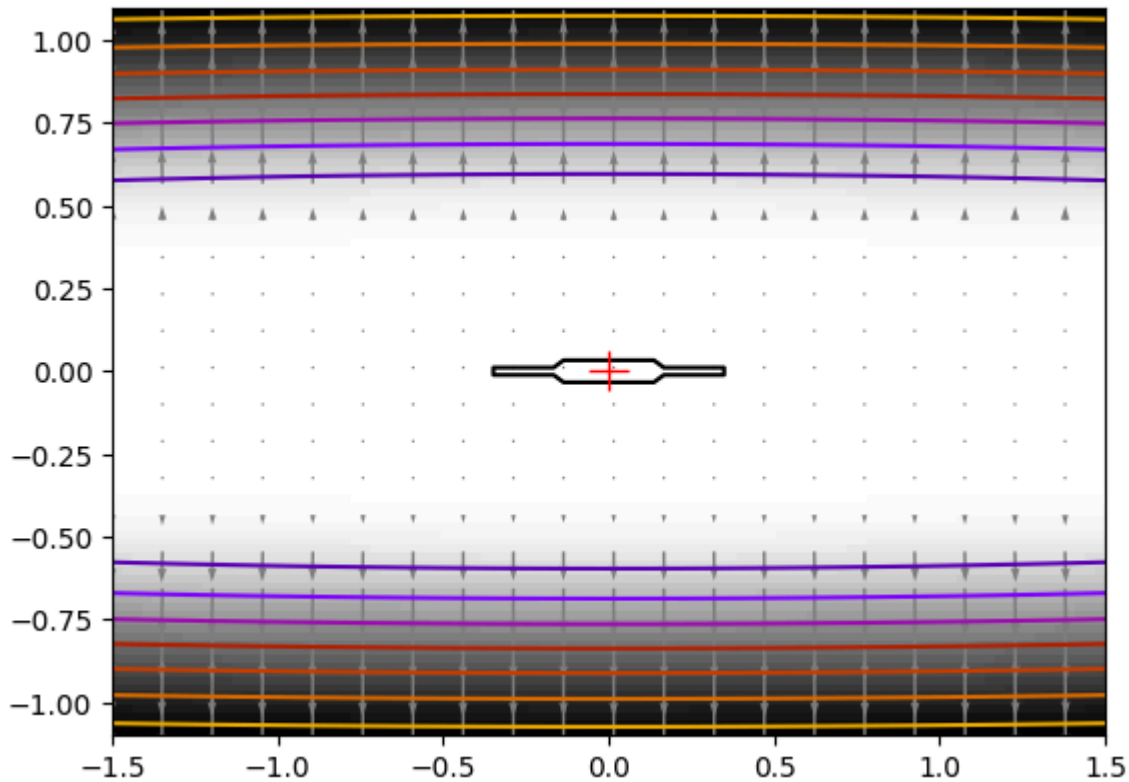
```
t = np.linspace(-1.1, 1.1, 100)  
plt.plot(t, f([0, t]));
```



```
X, Y = np.mgrid[-1.5:1.5:100j, -1.1:1.1:100j] # type: ignore[misc]
plt.imshow(f([X, Y]).T, cmap="gray_r", extent=(-1.5, 1.5, -1.1, 1.1),
origin="lower")
plt.contour(X, Y, f([X, Y]), cmap="gnuplot")

# Plot the gradient
dX, dY = g_prime([0.1 * X[:, ::5], Y[:, ::5]])
# Adjust for our preconditioning
dX *= 0.1
plt.quiver(X[:, ::5], Y[:, ::5], dX, dY, color=".5")

# Plot our solution
plt.plot(x_min[0], x_min[1], "r+", markersize=15);
```



Special case: non-linear least-squares

Minimizing the norm of a vector function

Least square problems, minimizing the norm of a vector function, have a specific structure that can be used in the [Levenberg–Marquardt algorithm](#) implemented in `scipy.optimize.leastsq()`.

Lets try to minimize the norm of the following vectorial function:

```
def f(x):
    return np.arctan(x) - np.arctan(np.linspace(0, 1, len(x)))
```

```
x0 = np.zeros(10)
sp.optimize.leastsq(f, x0)
```

```
(array([0.          , 0.11111111, 0.22222222, 0.33333333, 0.44444444,
        0.55555556, 0.66666667, 0.77777778, 0.88888889, 1.          ]),
 2)
```

This took 67 function evaluations (check it with 'full_output=True'). What if we compute the norm ourselves and use a good generic optimizer (BFGS):

```
def g(x):
    return np.sum(f(x)**2)

result = sp.optimize.minimize(g, x0, method="BFGS")
result.fun
```

```
np.float64(2.6940809156585398e-11)
```

BFGS needs more function calls, and gives a less precise result.

Note

`leastsq` is interesting compared to BFGS only if the dimensionality of the output vector is large, and larger than the number of parameters to optimize.

Warning

If the function is linear, this is a linear-algebra problem, and should be solved with

`scipy.linalg.lstsq()`.

Curve fitting

Least square problems occur often when fitting a non-linear to data. While it is possible to construct our optimization problem ourselves, SciPy provides a helper function for this purpose:

`scipy.optimize.curve_fit()`:

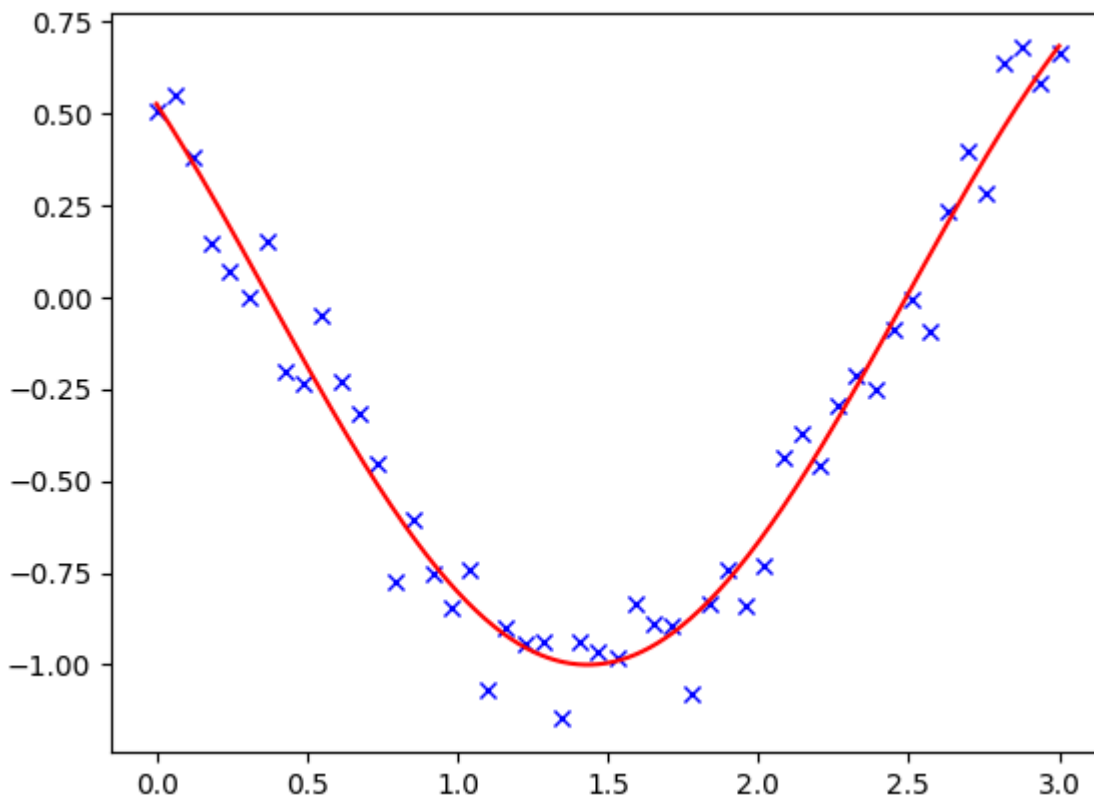
```
def f(t, omega, phi):
    return np.cos(omega * t + phi)

x = np.linspace(0, 3, 50)
rng = np.random.default_rng(27446968)
y = f(x, 1.5, 1) + .1*rng.normal(size=50)

sp.optimize.curve_fit(f, x, y)
```

```
(array([1.48121235, 0.99992781]),
 array([[ 0.00033369, -0.00049995],
        [-0.00049995,  0.00109915]]))
```

► Show code cell source



Exercise 57

Do the same with $\omega = 3$. What is the difficulty?

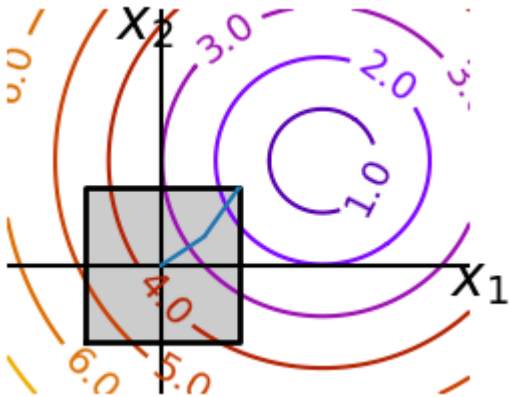
Optimization with constraints

Box bounds

Box bounds correspond to limiting each of the individual parameters of the optimization. Note that some problems that are not originally written as box bounds can be rewritten as such via change of variables. Both `scipy.optimize.minimize_scalar()` and `scipy.optimize.minimize()` support bound constraints with the parameter `bounds`:

```
def f(x):  
    return np.sqrt((x[0] - 3)**2 + (x[1] - 2)**2)  
  
sp.optimize.minimize(f, np.array([0, 0]), bounds=((-1.5, 1.5), (-1.5, 1.5)))
```

```
message: CONVERGENCE: NORM OF PROJECTED GRADIENT <= PGTOL
success: True
status: 0
  fun: 1.5811388300841898
   x: [ 1.500e+00  1.500e+00]
  nit: 2
  jac: [-9.487e-01 -3.162e-01]
 nfev: 9
 njev: 3
hess_inv: <2x2 LbfgsInvHessProduct with dtype=float64>
```



 **Plot code**

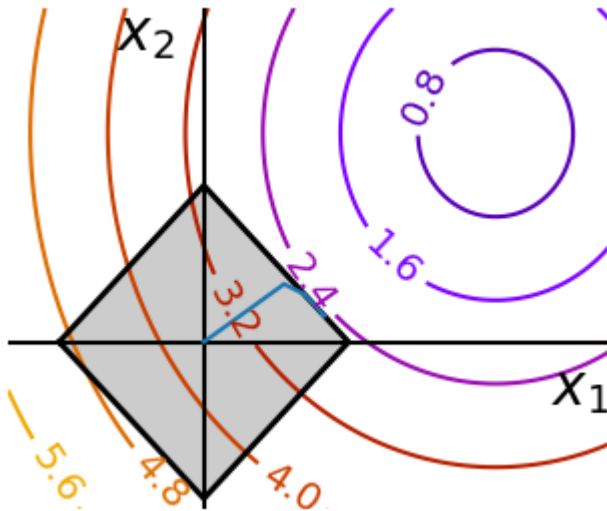
See [constraint plots](#).

General constraints

Equality and inequality constraints specified as functions: $f(x) = 0$ and $g(x) < 0$.

`scipy.optimize.fmin\_slsqp\(\)` Sequential least square

programming: equality and inequality constraints



Plot code

See [constraint non-bounds](#).

```
def f(x):  
    return np.sqrt((x[0] - 3)**2 + (x[1] - 2)**2)
```

```
def constraint(x):  
    return np.atleast_1d(1.5 - np.sum(np.abs(x)))
```

```
x0 = np.array([0, 0])  
sp.optimize.minimize(f, x0, constraints={"fun": constraint, "type": "ineq"})
```

```
message: Optimization terminated successfully  
success: True  
status: 0  
    fun: 2.474873735043833  
     x: [ 1.250e+00  2.500e-01]  
    nit: 5  
   jac: [-7.071e-01 -7.071e-01]  
  nfev: 15  
  njev: 5  
multipliers: [ 7.071e-01]
```

Warning

The above problem is known as the [Lasso](#) problem in statistics, and there exist very efficient solvers for it (for instance in [scikit-learn](#)). In general do not use generic solvers when specific ones exist.

Lagrange multipliers

If you are ready to do a bit of math, many constrained optimization problems can be converted to non-constrained optimization problems using a mathematical trick known as [Lagrange multipliers](#).

See also

Other Software

SciPy tries to include the best well-established, general-use, and permissively-licensed optimization algorithms available. However, even better options for a given task may be available in other libraries; please also see [IPOPT](#) and [PyGMO](#).

Interfacing with C

Contents

- Introduction
- Python-C-API
- Ctypes
- SWIG
- Cython
- Summary
- Further Reading and References
- Exercises

Author: *Valentin Haenel*

This chapter contains an *introduction* to the many different routes for making your native code (primarily `C/C++`) available from Python, a process commonly referred to *wrapping*. The goal of this chapter is to give you a flavour of what technologies exist and what their respective merits and shortcomings are, so that you can select the appropriate one for your specific needs. In any case, once you do start wrapping, you almost certainly will want to consult the respective documentation for your selected technique.

Introduction

This chapter covers the following techniques:

- [Python-C-API](#)
- [Ctypes](#)
- [SWIG \(Simplified Wrapper and Interface Generator\)](#)
- [Cython](#)

These four techniques are perhaps the most well known ones, of which Cython is probably the most advanced one and the one you should consider using first. The others are also important, if

you want to understand the wrapping problem from different angles. Having said that, there are other alternatives out there, but having understood the basics of the ones above, you will be in a position to evaluate the technique of your choice to see if it fits your needs.

The following criteria may be useful when evaluating a technology:

- Are additional libraries required?
- Is the code autogenerated?
- Does it need to be compiled?
- Is there good support for interacting with NumPy arrays?
- Does it support C++?

Before you set out, you should consider your use case. When interfacing with native code, there are usually two use-cases that come up:

- Existing code in C/C++ that needs to be leveraged, either because it already exists, or because it is faster.
- Python code too slow, push inner loops to native code

Each technology is demonstrated by wrapping the `cos` function from `math.h`. While this is a mostly a trivial example, it should serve us well to demonstrate the basics of the wrapping solution. Since each technique also includes some form of NumPy support, this is also demonstrated using an example where the cosine is computed on some kind of array.

Last but not least, two small warnings:

- All of these techniques may crash (segmentation fault) the Python interpreter, which is (usually) due to bugs in the C code.
- All the examples have been done on Linux, they *should* be possible on other operating systems.
- You will need a C compiler for most of the examples.

Python-C-API

The [Python-C-API](#) is the backbone of the standard Python interpreter (a.k.a *CPython*). Using this API it is possible to write Python extension module in C and C++. Obviously, these extension modules can, by virtue of language compatibility, call any function written in C or C++.

When using the Python-C-API, one usually writes much boilerplate code, first to parse the arguments that were given to a function, and later to construct the return type.

Advantages

- Requires no additional libraries
- Lots of low-level control
- Entirely usable from C++

Disadvantages

- May require a substantial amount of effort
- Much overhead in the code
- Must be compiled
- High maintenance cost
- No forward compatibility across Python versions as C-API changes
- Reference count bugs are easy to create and very hard to track down.

Note

The Python-C-API example here serves mainly for didactic reasons. Many of the other techniques actually depend on this, so it is good to have a high-level understanding of how it works. In 99% of the use-cases you will be better off, using an alternative technique.

Note

Since reference counting bugs are easy to create and hard to track down, anyone really needing to use the Python C-API should read the [section about objects, types and reference counts](#) from the official python documentation. Additionally, there is a tool by the name of [cpychecker](#) which can help discover common errors with reference counting.

Example

The following C-extension module, make the `cos` function from the standard math library available to Python:

```

/* Example of wrapping cos function from math.h with the Python-C-API. */

#include <Python.h>
#include <math.h>

/* wrapped cosine function */
static PyObject* cos_func(PyObject* self, PyObject* args)
{
    double value;
    double answer;

    /* parse the input, from python float to c double */
    if (!PyArg_ParseTuple(args, "d", &value))
        return NULL;
    /* if the above function returns -1, an appropriate Python exception will
     * have been set, and the function simply returns NULL
     */

    /* call cos from libm */
    answer = cos(value);

    /* construct the output from cos, from c double to python float */
    return Py_BuildValue("f", answer);
}

/* define functions in module */
static PyMethodDef CosMethods[] =
{
    {"cos_func", cos_func, METH_VARARGS, "evaluate the cosine"},
    {NULL, NULL, 0, NULL}
};

#if PY_MAJOR_VERSION >= 3
/* module initialization */
/* Python version 3*/
static struct PyModuleDef cModPyDem =
{
    PyModuleDef_HEAD_INIT,
    "cos_module", "Some documentation",
    -1,
    CosMethods
};

PyMODINIT_FUNC
PyInit_cos_module(void)
{
    return PyModule_Create(&cModPyDem);
}

#else

/* module initialization */
/* Python version 2 */
PyMODINIT_FUNC
initscos_module(void)
{
    (void) Py_InitModule("cos_module", CosMethods);
}

```

```
}  
  
#endif
```

As you can see, there is much boilerplate, both to «massage» the arguments and return types into place and for the module initialisation. Although some of this is amortised, as the extension grows, the boilerplate required for each function(s) remains.

The standard python build system, `setuptools`, supports compiling C-extensions via a `setup.py` file:

```
from setuptools import setup, Extension  
  
# define the extension module  
cos_module = Extension("cos_module", sources=["cos_module.c"])  
  
# run the setup  
setup(ext_modules=[cos_module])
```

The setup file is called as follows:

```
$ cd advanced/interfacing_with_c/python_c_api  
  
$ ls  
cos_module.c  setup.py  
  
$ python setup.py build_ext --inplace  
running build_ext  
building 'cos_module' extension  
creating build  
creating build/temp.linux-x86_64-2.7  
gcc -pthread -fno-strict-aliasing -g -O2 -DNDEBUG -g -fwrapv -O3 -Wall -  
Wstrict-prototypes -fPIC -I/home/esc/anaconda/include/python2.7 -c cos_module.c  
-o build/temp.linux-x86_64-2.7/cos_module.o  
gcc -pthread -shared build/temp.linux-x86_64-2.7/cos_module.o -  
L/home/esc/anaconda/lib -lpython2.7 -o /home/esc/git-working/scientific-python-  
lectures/advanced/interfacing_with_c/python_c_api/cos_module.so  
  
$ ls  
build/  cos_module.c  cos_module.so  setup.py
```

- `build_ext` is to build extension modules
- `--inplace` will output the compiled extension module into the current directory

The file `cos_module.so` contains the compiled extension, which we can now load in the IPython interpreter:

Note

In Python 3, the filename for compiled modules includes metadata on the Python interpreter (see [PEP 3149](#)) and is thus longer. The import statement is not affected by this.

```
In [1]: import cos_module

In [2]: cos_module?
Type:      module
String Form:<module 'cos_module' from 'cos_module.so'>
File:      /home/esc/git-working/scientific-python-lectures/advanced/interfacing_with_c/python_c_api/cos_module.so
Docstring: <no docstring>

In [3]: dir(cos_module)
Out[3]: ['__doc__', '__file__', '__name__', '__package__', 'cos_func']

In [4]: cos_module.cos_func(1.0)
Out[4]: 0.5403023058681398

In [5]: cos_module.cos_func(0.0)
Out[5]: 1.0

In [6]: cos_module.cos_func(3.14159265359)
Out[7]: -1.0
```

Now let's see how robust this is:

```
In [10]: cos_module.cos_func('foo')
-----
TypeError                                Traceback (most recent call last)
<ipython-input-10-11bee483665d> in <module>()
----> 1 cos_module.cos_func('foo')
TypeError: a float is required
```

NumPy Support

Analog to the Python-C-API, NumPy, which is itself implemented as a C-extension, comes with the [NumPy-C-API](#). This API can be used to create and manipulate NumPy arrays from C, when writing a custom C-extension. See also: [Advanced NumPy](#).

Note

If you do ever need to use the NumPy C-API refer to the documentation about [Arrays](#) and [Iterators](#).

The following example shows how to pass NumPy arrays as arguments to functions and how to iterate over NumPy arrays using the (old) NumPy-C-API. It simply takes an array as argument applies the cosine function from the `math.h` and returns a resulting new array.

```

/* Example of wrapping the cos function from math.h using the NumPy-C-API. */

#include <Python.h>
#include <numpy/arrayobject.h>
#include <math.h>

/* wrapped cosine function */
static PyObject* cos_func_np(PyObject* self, PyObject* args)
{
    PyArrayObject *arrays[2]; /* holds input and output array */
    PyObject *ret;
    NpyIter *iter;
    npy_uint32 op_flags[2];
    npy_uint32 iterator_flags;
    PyArray_Descr *op_dtypes[2];

    NpyIter_IterNextFunc *iternext;

    /* parse single NumPy array argument */
    if (!PyArg_ParseTuple(args, "O!", &PyArray_Type, &arrays[0])) {
        return NULL;
    }

    arrays[1] = NULL; /* The result will be allocated by the iterator */

    /* Set up and create the iterator */
    iterator_flags = (NPY_ITER_ZEROSIZE_OK |
        /*
         * Enable buffering in case the input is not behaved
         * (native byte order or not aligned),
         * disabling may speed up some cases when it is known to
         * be unnecessary.
         */
        NPY_ITER_BUFFERED |
        /* Manually handle innermost iteration for speed: */
        NPY_ITER_EXTERNAL_LOOP |
        NPY_ITER_GROWINNER);

    op_flags[0] = (NPY_ITER_READONLY |
        /*
         * Required that the arrays are well behaved, since the cos
         * call below requires this.
         */
        NPY_ITER_NBO |
        NPY_ITER_ALIGNED);

    /* Ask the iterator to allocate an array to write the output to */
    op_flags[1] = NPY_ITER_WRITEONLY | NPY_ITER_ALLOCATE;

    /*
     * Ensure the iteration has the correct type, could be checked
     * specifically here.
     */
    op_dtypes[0] = PyArray_DescrFromType(NPY_DOUBLE);
    op_dtypes[1] = op_dtypes[0];

```

```

/* Create the NumPy iterator object: */
iter = NpyIter_MultiNew(2, arrays, iterator_flags,
                        /* Use input order for output and iteration */
                        NPY_KEEPOORDER,
                        /* Allow only byte-swapping of input */
                        NPY_EQUIV_CASTING, op_flags, op_dtypes);
Py_DECREF(op_dtypes[0]); /* The second one is identical. */

if (iter == NULL)
    return NULL;

iternext = NpyIter_GetIterNext(iter, NULL);
if (iternext == NULL) {
    NpyIter_Deallocate(iter);
    return NULL;
}

/* Fetch the output array which was allocated by the iterator: */
ret = (PyObject *)NpyIter_GetOperandArray(iter)[1];
Py_INCREF(ret);

if (NpyIter_GetIterSize(iter) == 0) {
    /*
     * If there are no elements, the loop cannot be iterated.
     * This check is necessary with NPY_ITER_ZEROSIZE_OK.
     */
    NpyIter_Deallocate(iter);
    return ret;
}

/* The location of the data pointer which the iterator may update */
char **dataptr = NpyIter_GetDataPtrArray(iter);
/* The location of the stride which the iterator may update */
numpy_intp *strideptr = NpyIter_GetInnerStrideArray(iter);
/* The location of the inner loop size which the iterator may update */
numpy_intp *innersizeptr = NpyIter_GetInnerLoopSizePtr(iter);

/* iterate over the arrays */
do {
    numpy_intp stride = strideptr[0];
    numpy_intp count = *innersizeptr;
    /* out is always contiguous, so use double */
    double *out = (double *)dataptr[1];
    char *in = dataptr[0];

    /* The output is allocated and guaranteed contiguous (out++ works): */
    assert(strideptr[1] == sizeof(double));

    /*
     * For optimization it can make sense to add a check for
     * stride == sizeof(double) to allow the compiler to optimize for that.
     */
    while (count--) {
        *out = cos(*(double *)in);
        out++;
        in += stride;
    }
} while (iternext(iter));

```

```

    /* Clean up and return the result */
    NpyIter_Deallocate(iter);
    return ret;
}

/* define functions in module */
static PyMethodDef CosMethods[] =
{
    {"cos_func_np", cos_func_np, METH_VARARGS,
     "evaluate the cosine on a NumPy array"},
    {NULL, NULL, 0, NULL}
};

#if PY_MAJOR_VERSION >= 3
/* module initialization */
/* Python version 3*/
static struct PyModuleDef cModPyDem = {
    PyModuleDef_HEAD_INIT,
    "cos_module", "Some documentation",
    -1,
    CosMethods
};
PyMODINIT_FUNC PyInit_cos_module_np(void) {
    PyObject *module;
    module = PyModule_Create(&cModPyDem);
    if(module==NULL) return NULL;
    /* IMPORTANT: this must be called */
    import_array();
    if (PyErr_Occurred()) return NULL;
    return module;
}

#else
/* module initialization */
/* Python version 2 */
PyMODINIT_FUNC initempty_module_np(void) {
    PyObject *module;
    module = Py_InitModule("cos_module_np", CosMethods);
    if(module==NULL) return;
    /* IMPORTANT: this must be called */
    import_array();
    return;
}

#endif

```

To compile this we can use `setuptools` again. However we need to be sure to include the NumPy headers by using `numpy.get_include()`.

```
from setuptools import setup, Extension
import numpy

# define the extension module
cos_module_np = Extension(
    "cos_module_np", sources=["cos_module_np.c"], include_dirs=
[numpy.get_include()]
)

# run the setup
setup(ext_modules=[cos_module_np])
```

To convince ourselves if this does actually works, we run the following test script:

```

import cos_module_np
import numpy as np
import matplotlib.pyplot as plt

x = np.arange(0, 2 * np.pi, 0.1)
y = cos_module_np.cos_func_np(x)
plt.plot(x, y)
plt.show()

# Below are more specific tests for less common usage
# -----

# The function is OK with `x` not having any elements:
x_empty = np.array([], dtype=np.float64)
y_empty = cos_module_np.cos_func_np(x_empty)
assert np.array_equal(y_empty, np.array([], dtype=np.float64))

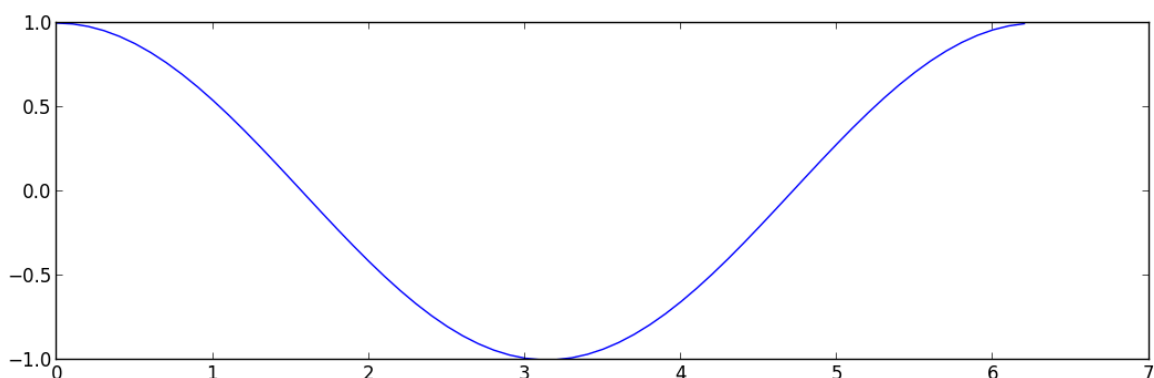
# The function can handle arbitrary dimensions and non-contiguous data.
# `x_2d` contains the same values, but has a different shape.
# Note: `x_2d.flags` shows it is not contiguous and `x_2d.ravel() == x`
x_2d = x.repeat(2)[::2].reshape(-1, 3)
y_2d = cos_module_np.cos_func_np(x_2d)
# When reshaped back, the same result is given:
assert np.array_equal(y_2d.ravel(), y)

# The function handles incorrect byte-order fine:
x_not_native_byteorder = x.astype(x.dtype.newbyteorder())
y_not_native_byteorder = cos_module_np.cos_func_np(x_not_native_byteorder)
assert np.array_equal(y_not_native_byteorder, y)

# The function fails if the data type is incorrect:
x_incorrect_dtype = x.astype(np.float32)
try:
    cos_module_np.cos_func_np(x_incorrect_dtype)
    assert 0, "This cannot be reached."
except TypeError:
    # A TypeError will be raised, this can be changed by changing the
    # casting rule.
    pass

```

And this should result in the following figure:



Ctypes

[Ctypes](#) is a *foreign function library* for Python. It provides C compatible data types, and allows calling functions in DLLs or shared libraries. It can be used to wrap these libraries in pure Python.

Advantages

- Part of the Python standard library
- Does not need to be compiled
- Wrapping code entirely in Python

Disadvantages

- Requires code to be wrapped to be available as a shared library (roughly speaking `*.dll` in Windows `*.so` in Linux and `*.dylib` in Mac OSX.)
- No good support for C++

Example

As advertised, the wrapper code is in pure Python.

```

"""Example of wrapping cos function from math.h using ctypes."""

import ctypes

# find and load the library

# OSX or linux
from ctypes.util import find_library

libm_name = find_library("m")
assert libm_name is not None, "Cannot find libm (math) on this system :/ That's bad."

libm = ctypes.cdll.LoadLibrary(libm_name)

# Windows
# from ctypes import windll
# libm = cdll.msvcrt

# set the argument type
libm.cos.argtypes = [ctypes.c_double]
# set the return type
libm.cos.restype = ctypes.c_double

def cos_func(arg):
    """Wrapper for cos from math.h"""
    return libm.cos(arg)

```

- Finding and loading the library may vary depending on your operating system, check [the documentation](#) for details
- This may be somewhat deceptive, since the math library exists in compiled form on the system already. If you were to wrap a in-house library, you would have to compile it first, which may or may not require some additional effort.

We may now use this, as before:

```

In [1]: import cos_module

In [2]: cos_module?
Type:      module
String Form:<module 'cos_module' from 'cos_module.py'>
File:      /home/esc/git-working/scientific-python-lectures/advanced/interfacing_with_c/ctypes/cos_module.py
Docstring: <no docstring>

In [3]: dir(cos_module)
Out[3]:
['__builtins__',
 '__doc__',
 '__file__',
 '__name__',
 '__package__',
 'cos_func',
 'ctypes',
 'find_library',
 'libm']

In [4]: cos_module.cos_func(1.0)
Out[4]: 0.5403023058681398

In [5]: cos_module.cos_func(0.0)
Out[5]: 1.0

In [6]: cos_module.cos_func(3.14159265359)
Out[6]: -1.0

```

As with the previous example, this code is somewhat robust, although the error message is not quite as helpful, since it does not tell us what the type should be.

```

cos_module.cos_func('foo')
-----
ArgumentError                                Traceback (most recent call last)
<ipython-input-7-11bee483665d> in <module>()
----> 1 cos_module.cos_func('foo')
/home/esc/git-working/scientific-python-lectures/advanced/interfacing_with_c/ctypes/cos_module.py in cos_func(arg)
     12 def cos_func(arg):
     13     ''' Wrapper for cos from math.h '''
----> 14     return libm.cos(arg)
ArgumentError: argument 1: <type 'exceptions.TypeError'>: wrong type

```

NumPy Support

NumPy contains some support for interfacing with ctypes. In particular there is support for exporting certain attributes of a NumPy array as ctypes data-types and there are functions to convert from C arrays to NumPy arrays and back.

For more information, consult the corresponding section in the [NumPy Cookbook](#) and the API documentation for [numpy.ndarray.ctypes](#) and [numpy.ctypeslib](#).

For the following example, let's consider a C function in a library that takes an input and an output array, computes the cosine of the input array and stores the result in the output array.

The library consists of the following header file (although this is not strictly needed for this example, we list it for completeness):

```
void cos_doubles(double * in_array, double * out_array, int size);
```

The function implementation resides in the following C source file:

```
#include <math.h>

/* Compute the cosine of each element in in_array, storing the result in
 * out_array. */
void cos_doubles(double * in_array, double * out_array, int size){
    int i;
    for(i=0;i<size;i++){
        out_array[i] = cos(in_array[i]);
    }
}
```

And since the library is pure C, we can't use `setuptools` to compile it, but must use a combination of `make` and `gcc`:

```
m.PHONY : clean

libcos_doubles.so : cos_doubles.o
    gcc -shared -Wl,-soname,libcos_doubles.so -o libcos_doubles.so
cos_doubles.o

cos_doubles.o : cos_doubles.c
    gcc -c -fPIC cos_doubles.c -o cos_doubles.o

clean :
    -rm -vf libcos_doubles.so cos_doubles.o cos_doubles.pyc
```

We can then compile this (on Linux) into the shared library `libcos_doubles.so`:

```
$ ls
cos_doubles.c  cos_doubles.h  cos_doubles.py  makefile  test_cos_doubles.py
$ make
gcc -c -fPIC cos_doubles.c -o cos_doubles.o
gcc -shared -Wl,-soname,libcos_doubles.so -o libcos_doubles.so cos_doubles.o
$ ls
cos_doubles.c  cos_doubles.o  libcos_doubles.so*  test_cos_doubles.py
cos_doubles.h  cos_doubles.py  makefile
```

Now we can proceed to wrap this library via ctypes with direct support for (certain kinds of) NumPy arrays:

```
"""Example of wrapping a C library function that accepts a C double array as
input using the numpy.ctypeslib."""

import numpy as np
import numpy.ctypeslib as npct
from ctypes import c_int

# input type for the cos_doubles function
# must be a double array, with single dimension that is contiguous
array_1d_double = npct.ndpointer(dtype=np.double, ndim=1, flags="CONTIGUOUS")

# load the library, using NumPy mechanisms
libcd = npct.load_library("libcos_doubles", ".")

# setup the return types and argument types
libcd.cos_doubles.restype = None
libcd.cos_doubles.argtypes = [array_1d_double, array_1d_double, c_int]

def cos_doubles_func(in_array, out_array):
    return libcd.cos_doubles(in_array, out_array, len(in_array))
```

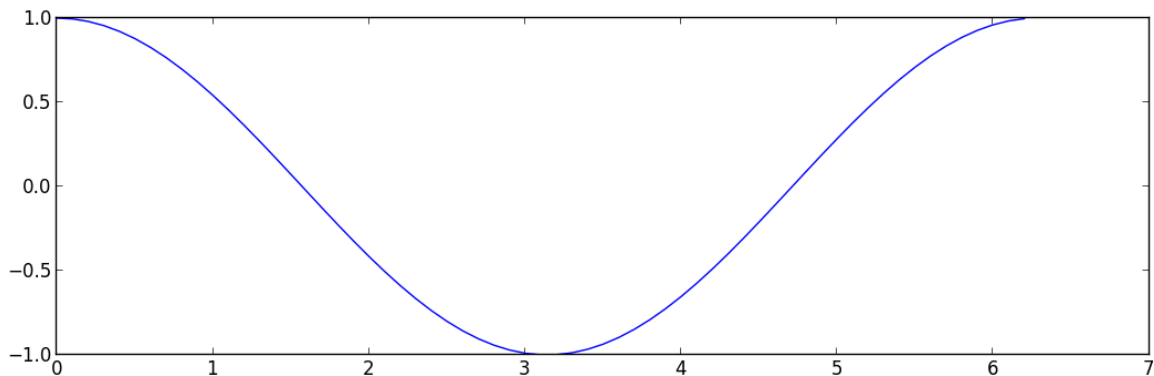
- Note the inherent limitation of contiguous single dimensional NumPy arrays, since the C functions requires this kind of buffer.
- Also note that the output array must be preallocated, for example with `numpy.zeros()` and the function will write into it's buffer.
- Although the original signature of the `cos_doubles` function is `ARRAY, ARRAY, int` the final `cos_doubles_func` takes only two NumPy arrays as arguments.

And, as before, we convince ourselves that it worked:

```
import numpy as np
import matplotlib.pyplot as plt
import cos_doubles

x = np.arange(0, 2 * np.pi, 0.1)
y = np.empty_like(x)

cos_doubles.cos_doubles_func(x, y)
plt.plot(x, y)
plt.show()
```



SWIG

[SWIG](#), the Simplified Wrapper Interface Generator, is a software development tool that connects programs written in C and C++ with a variety of high-level programming languages, including Python. The important thing with SWIG is, that it can autogenerate the wrapper code for you. While this is an advantage in terms of development time, it can also be a burden. The generated file tend to be quite large and may not be too human readable and the multiple levels of indirection which are a result of the wrapping process, may be a bit tricky to understand.

Note

The autogenerated C code uses the Python-C-API.

Advantages

- Can automatically wrap entire libraries given the headers
- Works nicely with C++

Disadvantages

- Autogenerates enormous files

- Hard to debug if something goes wrong
- Steep learning curve

Example

Let's imagine that our `cos` function lives in a `cos_module` which has been written in `C` and consists of the source file `cos_module.c`:

```
#include <math.h>

double cos_func(double arg){
    return cos(arg);
}
```

and the header file `cos_module.h`:

```
double cos_func(double arg);
```

And our goal is to expose the `cos_func` to Python. To achieve this with SWIG, we must write an *interface file* which contains the instructions for SWIG.

```
/* Example of wrapping cos function from math.h using SWIG. */

%module cos_module
%{
    /* the resulting C file should be built as a python extension */
    #define SWIG_FILE_WITH_INIT
    /* Includes the header in the wrapper code */
    #include "cos_module.h"
%}
/* Parse the header file to generate wrappers */
#include "cos_module.h"
```

As you can see, not too much code is needed here. For this simple example it is enough to simply include the header file in the interface file, to expose the function to Python. However, SWIG does allow for more fine grained inclusion/exclusion of functions found in header files, check the documentation for details.

Generating the compiled wrappers is a two stage process:

1. Run the `swig` executable on the interface file to generate the files `cos_module_wrap.c`, which is the source file for the autogenerated Python C-extension and `cos_module.py`, which

is the autogenerated pure python module.

2. Compile the `cos_module_wrap.c` into the `_cos_module.so`. Luckily, `setuptools` knows how to handle SWIG interface files, so that our `setup.py` is simply:

```
from setuptools import setup, Extension
```

```
setup(ext_modules=[Extension("_cos_module", sources=["cos_module.c",  
"cos_module.i"])]))
```

```
$ cd advanced/interfacing_with_c/swig
```

```
$ ls
```

```
cos_module.c  cos_module.h  cos_module.i  setup.py
```

```
$ python setup.py build_ext --inplace
```

```
running build_ext
```

```
building '_cos_module' extension
```

```
swigging cos_module.i to cos_module_wrap.c
```

```
swig -python -o cos_module_wrap.c cos_module.i
```

```
creating build
```

```
creating build/temp.linux-x86_64-2.7
```

```
gcc -pthread -fno-strict-aliasing -g -O2 -DNDEBUG -g -fwrapv -O3 -Wall -
```

```
Wstrict-prototypes -fPIC -I/home/esc/anaconda/include/python2.7 -c cos_module.c
```

```
-o build/temp.linux-x86_64-2.7/cos_module.o
```

```
gcc -pthread -fno-strict-aliasing -g -O2 -DNDEBUG -g -fwrapv -O3 -Wall -
```

```
Wstrict-prototypes -fPIC -I/home/esc/anaconda/include/python2.7 -c
```

```
cos_module_wrap.c -o build/temp.linux-x86_64-2.7/cos_module_wrap.o
```

```
gcc -pthread -shared build/temp.linux-x86_64-2.7/cos_module.o build/temp.linux-
```

```
x86_64-2.7/cos_module_wrap.o -L/home/esc/anaconda/lib -lpython2.7 -o
```

```
/home/esc/git-working/scientific-python-
```

```
lectures/advanced/interfacing_with_c/swig/_cos_module.so
```

```
$ ls
```

```
build/  cos_module.c  cos_module.h  cos_module.i  cos_module.py
```

```
_cos_module.so*  cos_module_wrap.c  setup.py
```

We can now load and execute the `cos_module` as we have done in the previous examples:

```

In [1]: import cos_module

In [2]: cos_module?
Type:      module
String Form:<module 'cos_module' from 'cos_module.py'>
File:      /home/esc/git-working/scientific-python-lectures/advanced/interfacing_with_c/swig/cos_module.py
Docstring: <no docstring>

In [3]: dir(cos_module)
Out[3]:
['__builtins__',
'__doc__',
'__file__',
'__name__',
'__package__',
'_cos_module',
'_newclass',
'_object',
'_swig_getattr',
'_swig_property',
'_swig_repr',
'_swig_setattr',
'_swig_setattr_nondynamic',
'cos_func']

In [4]: cos_module.cos_func(1.0)
Out[4]: 0.5403023058681398

In [5]: cos_module.cos_func(0.0)
Out[5]: 1.0

In [6]: cos_module.cos_func(3.14159265359)
Out[6]: -1.0

```

Again we test for robustness, and we see that we get a better error message (although, strictly speaking in Python there is no `double` type):

```

In [7]: cos_module.cos_func('foo')
-----
TypeError                                Traceback (most recent call last)
<ipython-input-7-11bee483665d> in <module>()
----> 1 cos_module.cos_func('foo')
TypeError: in method 'cos_func', argument 1 of type 'double'

```

NumPy Support

NumPy provides [support for SWIG](#) with the `numpy.i` file. This interface file defines various so-called *typemaps* which support conversion between NumPy arrays and C-Arrays. In the following

example we will take a quick look at how such typemaps work in practice.

We have the same `cos_doubles` function as in the ctypes example:

```
void cos_doubles(double * in_array, double * out_array, int size);
```

```
#include <math.h>

/* Compute the cosine of each element in in_array, storing the result in
 * out_array. */
void cos_doubles(double * in_array, double * out_array, int size){
    int i;
    for(i=0;i<size;i++){
        out_array[i] = cos(in_array[i]);
    }
}
```

This is wrapped as `cos_doubles_func` using the following SWIG interface file:

```

/* Example of wrapping a C function that takes a C double array as input using
 * NumPy typemaps for SWIG. */

%module cos_doubles
%{
    /* the resulting C file should be built as a python extension */
    #define SWIG_FILE_WITH_INIT
    /* Includes the header in the wrapper code */
    #include "cos_doubles.h"
%}

/* include the NumPy typemaps */
#include "numpy.i"
/* need this for correct module initialization */
%init %{
    import_array();
%}

/* typemaps for the two arrays, the second will be modified in-place */
%apply (double* IN_ARRAY1, int DIM1) {(double * in_array, int size_in)}
%apply (double* INPLACE_ARRAY1, int DIM1) {(double * out_array, int size_out)}

/* Wrapper for cos_doubles that massages the types */
%inline %{
    /* takes as input two NumPy arrays */
    void cos_doubles_func(double * in_array, int size_in, double * out_array,
int size_out) {
        /* calls the original function, providing only the size of the first
        */
        cos_doubles(in_array, out_array, size_in);
    }
%}

```

- To use the NumPy typemaps, we need include the `numpy.i` file.
- Observe the call to `import_array()` which we encountered already in the NumPy-C-API example.
- Since the type maps only support the signature `ARRAY, SIZE` we need to wrap the `cos_doubles` as `cos_doubles_func` which takes two arrays including sizes as input.
- As opposed to the simple SWIG example, we don't include the `cos_doubles.h` header, There is nothing there that we wish to expose to Python since we expose the functionality through `cos_doubles_func`.

And, as before we can use `setuptools` to wrap this:

```
from setuptools import setup, Extension
import numpy

setup(
    ext_modules=[
        Extension(
            "_cos_doubles",
            sources=["cos_doubles.c", "cos_doubles.i"],
            include_dirs=[numpy.get_include()],
        )
    ]
)
```

As previously, we need to use `include_dirs` to specify the location.

```

$ ls
cos_doubles.c cos_doubles.h cos_doubles.i numpy.i setup.py
test_cos_doubles.py
$ python setup.py build_ext -i
running build_ext
building '_cos_doubles' extension
swigging cos_doubles.i to cos_doubles_wrap.c
swig -python -o cos_doubles_wrap.c cos_doubles.i
cos_doubles.i:24: Warning(490): Fragment 'NumPy_Backward_Compatibility' not
found.
cos_doubles.i:24: Warning(490): Fragment 'NumPy_Backward_Compatibility' not
found.
cos_doubles.i:24: Warning(490): Fragment 'NumPy_Backward_Compatibility' not
found.
creating build
creating build/temp.linux-x86_64-2.7
gcc -pthread -fno-strict-aliasing -g -O2 -DNDEBUG -g -fwrapv -O3 -Wall -
Wstrict-prototypes -fPIC -I/home/esc/anaconda/lib/python2.7/site-
packages/numpy/core/include -I/home/esc/anaconda/include/python2.7 -c
cos_doubles.c -o build/temp.linux-x86_64-2.7/cos_doubles.o
gcc -pthread -fno-strict-aliasing -g -O2 -DNDEBUG -g -fwrapv -O3 -Wall -
Wstrict-prototypes -fPIC -I/home/esc/anaconda/lib/python2.7/site-
packages/numpy/core/include -I/home/esc/anaconda/include/python2.7 -c
cos_doubles_wrap.c -o build/temp.linux-x86_64-2.7/cos_doubles_wrap.o
In file included from /home/esc/anaconda/lib/python2.7/site-
packages/numpy/core/include/numpy/ndarraytypes.h:1722,
                 from /home/esc/anaconda/lib/python2.7/site-
packages/numpy/core/include/numpy/ndarrayobject.h:17,
                 from /home/esc/anaconda/lib/python2.7/site-
packages/numpy/core/include/numpy/arrayobject.h:15,
                 from cos_doubles_wrap.c:2706:
/home/esc/anaconda/lib/python2.7/site-
packages/numpy/core/include/numpy/npymath/npymath.h:11:2: warning: #warning
"Using deprecated NumPy API, disable it by #defining NPY_NO_DEPRECATED_API
NPY_1_7_API_VERSION"
gcc -pthread -shared build/temp.linux-x86_64-2.7/cos_doubles.o
build/temp.linux-x86_64-2.7/cos_doubles_wrap.o -L/home/esc/anaconda/lib -
lpython2.7 -o /home/esc/git-working/scientific-python-
lectures/advanced/interfacing_with_c/swig_numpy/_cos_doubles.so
$ ls
build/          cos_doubles.h  cos_doubles.py  cos_doubles_wrap.c  setup.py
cos_doubles.c  cos_doubles.i  _cos_doubles.so* numpy.i
test_cos_doubles.py

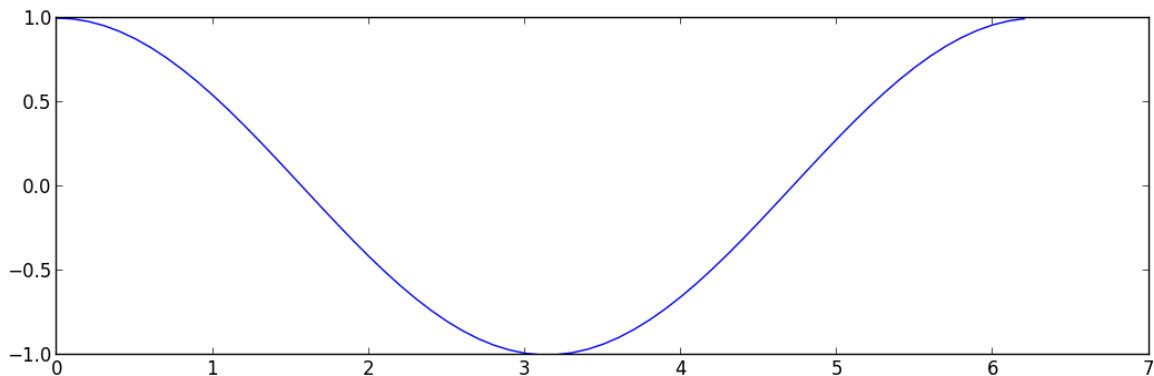
```

And, as before, we convince ourselves that it worked:

```
import numpy as np
import matplotlib.pyplot as plt
import cos_doubles

x = np.arange(0, 2 * np.pi, 0.1)
y = np.empty_like(x)

cos_doubles.cos_doubles_func(x, y)
plt.plot(x, y)
plt.show()
```



Cython

[Cython](#) is both a Python-like language for writing C-extensions and an advanced compiler for this language. The Cython *language* is a superset of Python, which comes with additional constructs that allow you call C functions and annotate variables and class attributes with c types. In this sense one could also call it a *Python with types*.

In addition to the basic use case of wrapping native code, Cython supports an additional use-case, namely interactive optimization. Basically, one starts out with a pure-Python script and incrementally adds Cython types to the bottleneck code to optimize only those code paths that really matter.

In this sense it is quite similar to SWIG, since the code can be autogenerated but in a sense it also quite similar to ctypes since the wrapping code can (almost) be written in Python.

While others solutions that autogenerated code can be quite difficult to debug (for example SWIG) Cython comes with an extension to the GNU debugger that helps debug Python, Cython and C code.

Note

The autogenerated C code uses the Python-C-API.

Advantages

- Python like language for writing C-extensions
- Autogenerated code
- Supports incremental optimization
- Includes a GNU debugger extension
- Support for C++ (Since version 0.13)

Disadvantages

- Must be compiled
- Requires an additional library (but only at build time, at this problem can be overcome by shipping the generated C files)

Example

The main Cython code for our `cos_module` is contained in the file `cos_module.pyx`:

```
""" Example of wrapping cos function from math.h using Cython. """

cdef extern from "math.h":
    double cos(double arg)

def cos_func(arg):
    return cos(arg)
```

Note the additional keywords such as `cdef` and `extern`. Also the `cos_func` is then pure Python.

Again we can use the standard `setuptools` module, but this time we need some additional pieces from `Cython.Build`:

```
from setuptools import setup, Extension
from Cython.Build import cythonize

extensions = [Extension("cos_module", sources=["cos_module.pyx"])]

setup(ext_modules=cythonize(extensions))
```

Compiling this:

```
$ cd advanced/interfacing_with_c/cython
$ ls
cos_module.pyx  setup.py
$ python setup.py build_ext --inplace
running build_ext
cythoning cos_module.pyx to cos_module.c
building 'cos_module' extension
creating build
creating build/temp.linux-x86_64-2.7
gcc -pthread -fno-strict-aliasing -g -O2 -DNDEBUG -g -fwrapv -O3 -Wall -
Wstrict-prototypes -fPIC -I/home/esc/anaconda/include/python2.7 -c cos_module.c
-o build/temp.linux-x86_64-2.7/cos_module.o
gcc -pthread -shared build/temp.linux-x86_64-2.7/cos_module.o -
L/home/esc/anaconda/lib -lpython2.7 -o /home/esc/git-working/scientific-python-
lectures/advanced/interfacing_with_c/cython/cos_module.so
$ ls
build/  cos_module.c  cos_module.pyx  cos_module.so*  setup.py
```

And running it:

```

In [1]: import cos_module

In [2]: cos_module?
Type:      module
String Form:<module 'cos_module' from 'cos_module.so'>
File:      /home/esc/git-working/scientific-python-lectures/advanced/interfacing_with_c/cython/cos_module.so
Docstring: <no docstring>

In [3]: dir(cos_module)
Out[3]:
['__builtins__',
 '__doc__',
 '__file__',
 '__name__',
 '__package__',
 '__test__',
 'cos_func']

In [4]: cos_module.cos_func(1.0)
Out[4]: 0.5403023058681398

In [5]: cos_module.cos_func(0.0)
Out[5]: 1.0

In [6]: cos_module.cos_func(3.14159265359)
Out[6]: -1.0

```

And, testing a little for robustness, we can see that we get good error messages:

```

In [7]: cos_module.cos_func('foo')
-----
TypeError                                Traceback (most recent call last)
<ipython-input-7-11bee483665d> in <module>()
----> 1 cos_module.cos_func('foo')
/home/esc/git-working/scientific-python-lectures/advanced/interfacing_with_c/cython/cos_module.so in
cos_module.cos_func (cos_module.c:506)()
TypeError: a float is required

```

Additionally, it is worth noting that **Cython** ships with complete declarations for the C math library, which simplifies the code above to become:

```

""" Simpler example of wrapping cos function from math.h using Cython. """

from libc.math cimport cos

def cos_func(arg):
    return cos(arg)

```

In this case the `cimport` statement is used to import the `cos` function.

NumPy Support

Cython has support for NumPy via the `numpy.pyx` file which allows you to add the NumPy array type to your Cython code. I.e. like specifying that variable `i` is of type `int`, you can specify that variable `a` is of type `numpy.ndarray` with a given `dtype`. Also, certain optimizations such as bounds checking are supported. Look at the corresponding section in the [Cython documentation](#). In case you want to pass NumPy arrays as C arrays to your Cython wrapped C functions, there is a [section about this in the Cython documentation](#).

In the following example, we will show how to wrap the familiar `cos_doubles` function using Cython.

```
void cos_doubles(double * in_array, double * out_array, int size);
```

```
#include <math.h>

/* Compute the cosine of each element in in_array, storing the result in
 * out_array. */
void cos_doubles(double * in_array, double * out_array, int size){
    int i;
    for(i=0;i<size;i++){
        out_array[i] = cos(in_array[i]);
    }
}
```

This is wrapped as `cos_doubles_func` using the following Cython code:

```

""" Example of wrapping a C function that takes C double arrays as input using
    the NumPy declarations from Cython """

# cimport the Cython declarations for NumPy
cimport numpy as np

# if you want to use the NumPy-C-API from Cython
# (not strictly necessary for this example, but good practice)
np.import_array()

# cdefine the signature of our c function
cdef extern from "cos_doubles.h":
    void cos_doubles (double * in_array, double * out_array, int size)

# create the wrapper code, with NumPy type annotations
def cos_doubles_func(np.ndarray[double, ndim=1, mode="c"] in_array not None,
                    np.ndarray[double, ndim=1, mode="c"] out_array not None):
    cos_doubles(<double*> np.PyArray_DATA(in_array),
               <double*> np.PyArray_DATA(out_array),
               in_array.shape[0])

```

And can be compiled using `setuptools`:

```

from setuptools import setup, Extension
from Cython.Build import cythonize
import numpy

extensions = [
    Extension(
        "cos_doubles",
        sources=["_cos_doubles.pyx", "cos_doubles.c"],
        include_dirs=[numpy.get_include()],
    )
]

setup(ext_modules=cythonize(extensions))

```

- As with the previous compiled NumPy examples, we need the `include_dirs` option.

```

$ ls
cos_doubles.c  cos_doubles.h  _cos_doubles.pyx  setup.py  test_cos_doubles.py
$ python setup.py build_ext -i
running build_ext
cythoning _cos_doubles.pyx to _cos_doubles.c
building 'cos_doubles' extension
creating build
creating build/temp.linux-x86_64-2.7
gcc -pthread -fno-strict-aliasing -g -O2 -DNDEBUG -g -fwrapv -O3 -Wall -Wstrict-prototypes -fPIC -I/home/esc/anaconda/lib/python2.7/site-packages/numpy/core/include -I/home/esc/anaconda/include/python2.7 -c _cos_doubles.c -o build/temp.linux-x86_64-2.7/_cos_doubles.o
In file included from /home/esc/anaconda/lib/python2.7/site-packages/numpy/core/include/numpy/ndarraytypes.h:1722,
                 from /home/esc/anaconda/lib/python2.7/site-packages/numpy/core/include/numpy/ndarrayobject.h:17,
                 from /home/esc/anaconda/lib/python2.7/site-packages/numpy/core/include/numpy/arrayobject.h:15,
                 from _cos_doubles.c:253:
/home/esc/anaconda/lib/python2.7/site-packages/numpy/core/include/numpy/npymath/npymath.h:11:2: warning: #warning "Using deprecated NumPy API, disable it by #defining NPY_NO_DEPRECATED_API NPY_1_7_API_VERSION"
/home/esc/anaconda/lib/python2.7/site-packages/numpy/core/include/numpy/__ufunc_api.h:236: warning: '_import_umath' defined but not used
gcc -pthread -fno-strict-aliasing -g -O2 -DNDEBUG -g -fwrapv -O3 -Wall -Wstrict-prototypes -fPIC -I/home/esc/anaconda/lib/python2.7/site-packages/numpy/core/include -I/home/esc/anaconda/include/python2.7 -c cos_doubles.c -o build/temp.linux-x86_64-2.7/cos_doubles.o
gcc -pthread -shared build/temp.linux-x86_64-2.7/_cos_doubles.o build/temp.linux-x86_64-2.7/cos_doubles.o -L/home/esc/anaconda/lib -lpython2.7 -o /home/esc/git-working/scientific-python-lectures/advanced/interfacing_with_c/cython_numpy/cos_doubles.so
$ ls
build/ _cos_doubles.c  cos_doubles.c  cos_doubles.h  _cos_doubles.pyx
cos_doubles.so*  setup.py  test_cos_doubles.py

```

And, as before, we convince ourselves that it worked:

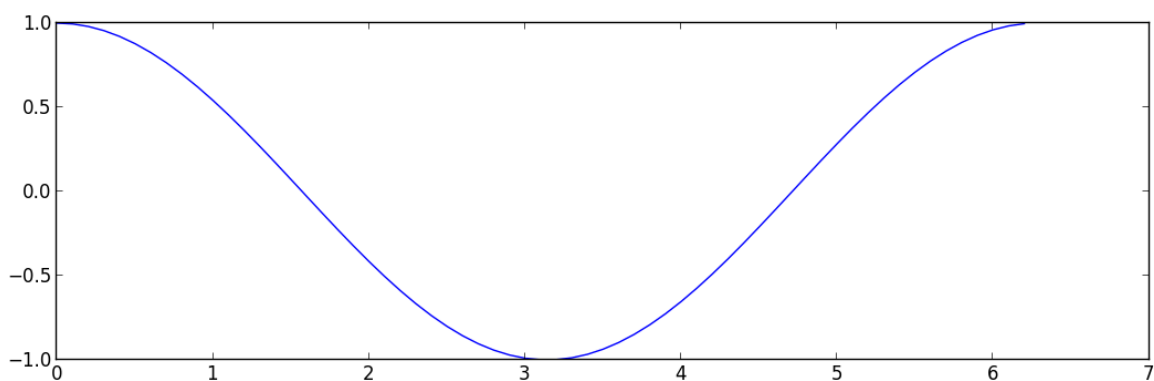
```

import numpy as np
import matplotlib.pyplot as plt
import cos_doubles

x = np.arange(0, 2 * np.pi, 0.1)
y = np.empty_like(x)

cos_doubles.cos_doubles_func(x, y)
plt.plot(x, y)
plt.show()

```



Summary

In this section four different techniques for interfacing with native code have been presented. The table below roughly summarizes some of the aspects of the techniques.

x	Part of CPython	Compiled	Autogenerated	NumPy Support
Python-C-API	True	True	False	True
Ctypes	True	False	False	True
Swig	False	True	True	True
Cython	False	True	True	True

Of all three presented techniques, Cython is the most modern and advanced. In particular, the ability to optimize code incrementally by adding types to your Python code is unique.

Further Reading and References

- [Gaël Varoquaux's blog post about avoiding data copies](#) provides some insight on how to handle memory management cleverly. If you ever run into issues with large datasets, this is a reference to come back to for some inspiration.

Exercises

Since this is a brand new section, the exercises are considered more as pointers as to what to look at next, so pick the ones that you find more interesting. If you have good ideas for exercises,

please let us know!

1. Download the source code for each example and compile and run them on your machine.
2. Make trivial changes to each example and convince yourself that this works. (E.g. change `cos` for `sin`.)
3. Most of the examples, especially the ones involving NumPy may still be fragile and respond badly to input errors. Look for ways to crash the examples, figure what the problem is and devise a potential solution. Here are some ideas:
 1. Numerical overflow.
 2. Input and output arrays that have different lengths.
 3. Multidimensional array.
 4. Empty array
 5. Arrays with non-`double` types
4. Use the `%timeit` IPython magic to measure the execution time of the various solutions

Python-C-API

1. Modify the NumPy example such that the function takes two input arguments, where the second is the preallocated output array, making it similar to the other NumPy examples.
2. Modify the example such that the function only takes a single input array and modifies this in place.
3. Try to fix the example to use the new [NumPy iterator protocol](#). If you manage to obtain a working solution, please submit a pull-request on github.
4. You may have noticed, that the NumPy-C-API example is the only NumPy example that does not wrap `cos_doubles` but instead applies the `cos` function directly to the elements of the NumPy array. Does this have any advantages over the other techniques.
5. Can you wrap `cos_doubles` using only the NumPy-C-API. You may need to ensure that the arrays have the correct type, are one dimensional and contiguous in memory.

Ctypes

1. Modify the NumPy example such that `cos_doubles_func` handles the preallocation for you, thus making it more like the NumPy-C-API example.

SWIG

1. Look at the code that SWIG autogenerated, how much of it do you understand?
2. Modify the NumPy example such that `cos_doubles_func` handles the preallocation for you, thus making it more like the NumPy-C-API example.
3. Modify the `cos_doubles` C function so that it returns an allocated array. Can you wrap this using SWIG typemaps? If not, why not? Is there a workaround for this specific situation? (Hint: you know the size of the output array, so it may be possible to construct a NumPy array from the returned `double *`.)

Cython

1. Look at the code that Cython autogenerated. Take a closer look at some of the comments that Cython inserts. What do you see?
2. Look at the section [Working with NumPy](#) from the Cython documentation to learn how to incrementally optimize a pure python script that uses NumPy.
3. Modify the NumPy example such that `cos_doubles_func` handles the preallocation for you, thus making it more like the NumPy-C-API example.